

Inteligență Artificială

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

Radu Ionescu

raducu.ionescu@gmail.com

Informatică, anul II, 2025-2026

< All teams



Inteligență Artificială - 2025-2026 F... ...

- Home page
- Class Notebook
- Classwork
- Assignments
- Grades
- Insights
- Reflect

▼ Main Channels

- General
- Învățare automată - curs
- Învățare automată - lab și sem**
- Rezolvarea problemelor și căutare - curs
- Rezolvarea problemelor și căutare - lab ...**

All documents

In messages

+ New ▾

↑ Upload ▾

📄 Edit in grid view

🔗 Share

🔗 Copy link

📄 Forms

New

🔗

Rezolvarea problemelor și căutare - curs > seria23

	Name ▾	Modified ▾	Modified By ▾	+
	Curs1.pdf	February 28	DUMITRU BOGDAN A	
	Curs1.pptx	February 28	DUMITRU BOGDAN A	
	Curs2.pdf	March 4	DUMITRU BOGDAN A	
	Curs2.pptx	March 4	DUMITRU BOGDAN A	
	Curs3.pdf	6 days ago	DUMITRU BOGDAN A	
	Curs3.pptx	Sunday at 5:01 PM	EVA MIHAELA GAVRI	
	📄 IA-INFO-PSS-2025-2026-BONUS-KAHOOT...	About a minute ago	DUMITRU BOGDAN A	

Recapitulare – cursul trecut

1. Căutare neinformată

- Căutare în lăţime (Bread-First Search)
- Căutare în adâncime (Depth-First Search)
- Căutare în adâncime limitată (Depth-Limited Search)
- Căutare ierativă în adâncime (Iterative Deepening Search)
- Căutare uniformă după cost (Uniform-Cost Search)
- Căutare bidirecţională (Bidirectional Search)

2. Căutare informată

- Căutare Greedy (Greedy Best-First Search)

Cuprinsul cursului de azi

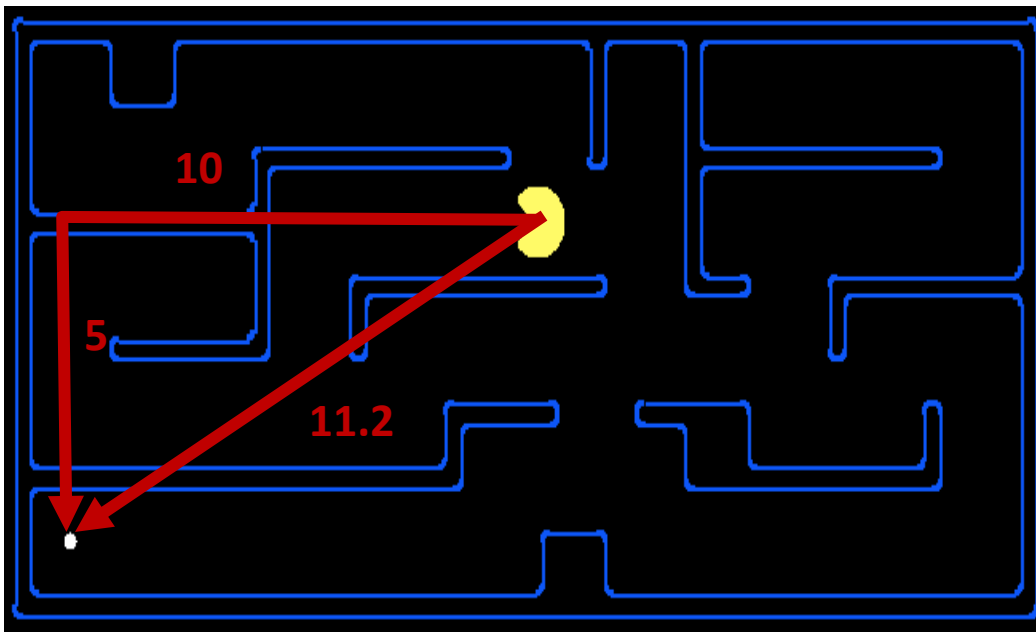
1. Căutare informată

- Căutare Greedy (Greedy Best-First Search)
- Euristici admisibile
- Căutare A* (A star Best-First Search)

Recapitulare: Funcții euristice

O euristică este:

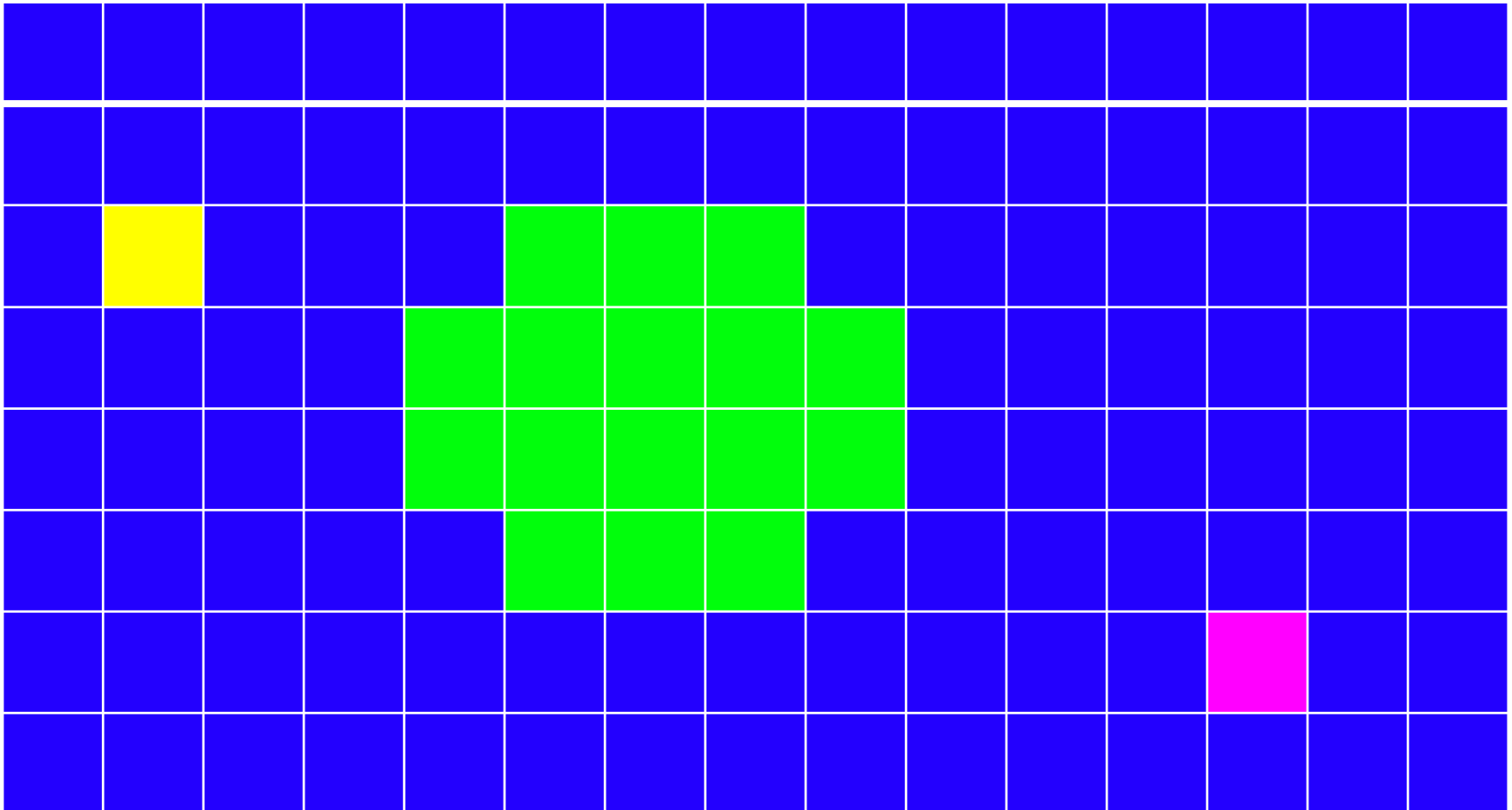
- o funcție h care *estimează* costul minim rămas până la o stare-scop;
- $h(n) \geq 0$, pentru orice stare n ;
- **$h(\text{stare-scop}) = 0$;**
- specifică pentru fiecare problemă;
- exemple: distanța Manhattan, distanța Euclideană pentru găsirea celui mai scurt drum;



Funcții euristice

Configurație mediu:

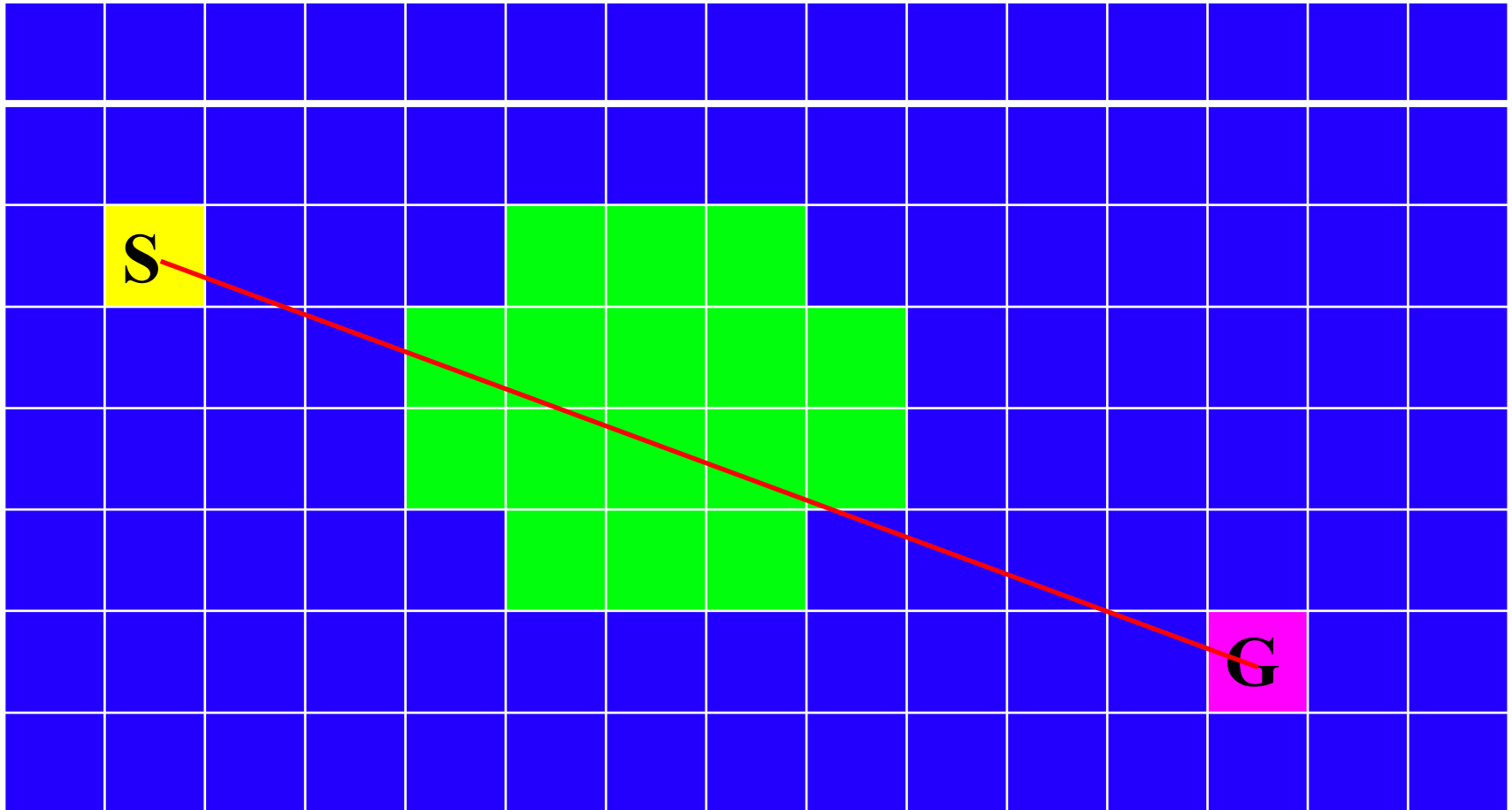
- **stare inițială:** celula marcată cu galben
- **stare scop:** celula marcată cu roz
- **obstacole:** celulele marcate cu verde (nu pot fi traversate)
- **spațiu navigabil:** celulele marcate cu albastru



Funcții euristice

$h(n)$ = distanța euclidiană între nodul curent n și nodul scop n_{goal}

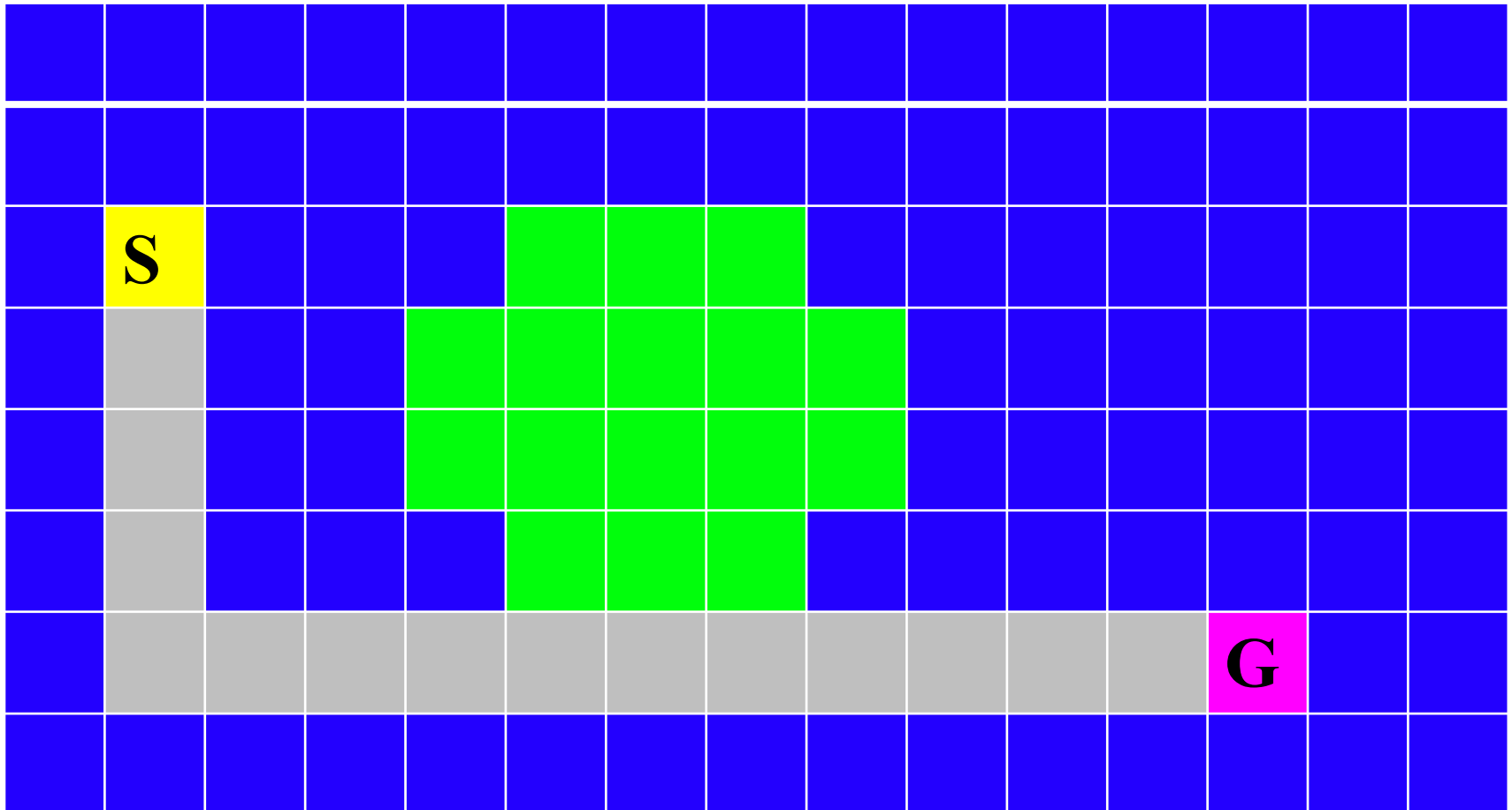
$$h(n) = d_{\text{euclid}}(n, n_{goal}) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$



Funcții euristice

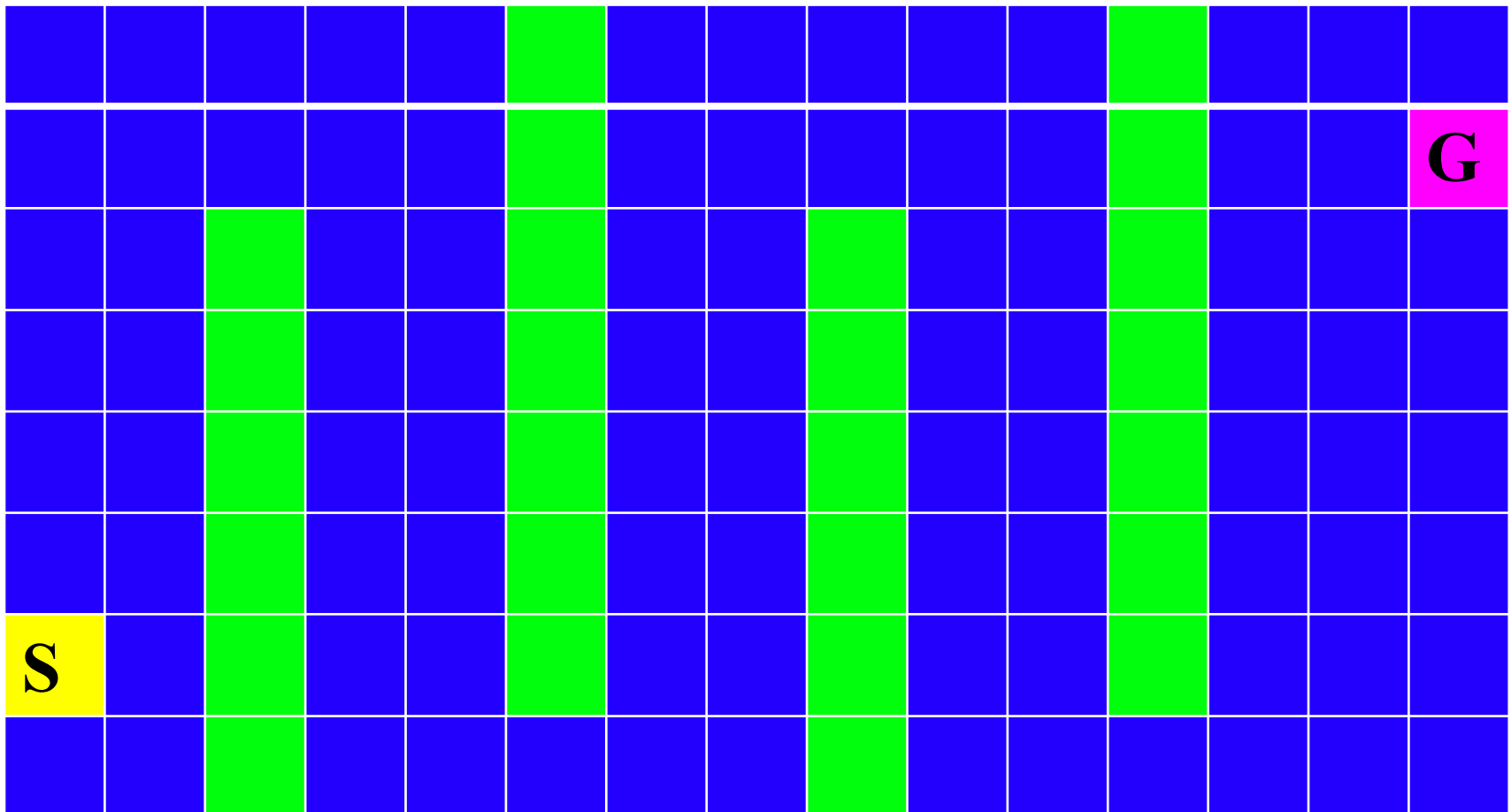
$$h(n) = \text{distanța Manhattan între nodul curent } n \text{ și nodul scop } n_{goal}$$

$$h(n) = d_{\text{Manhattan}}(n, n_{goal}) = |x_n - x_{goal}| + |y_n - y_{goal}|$$



Funcții euristice

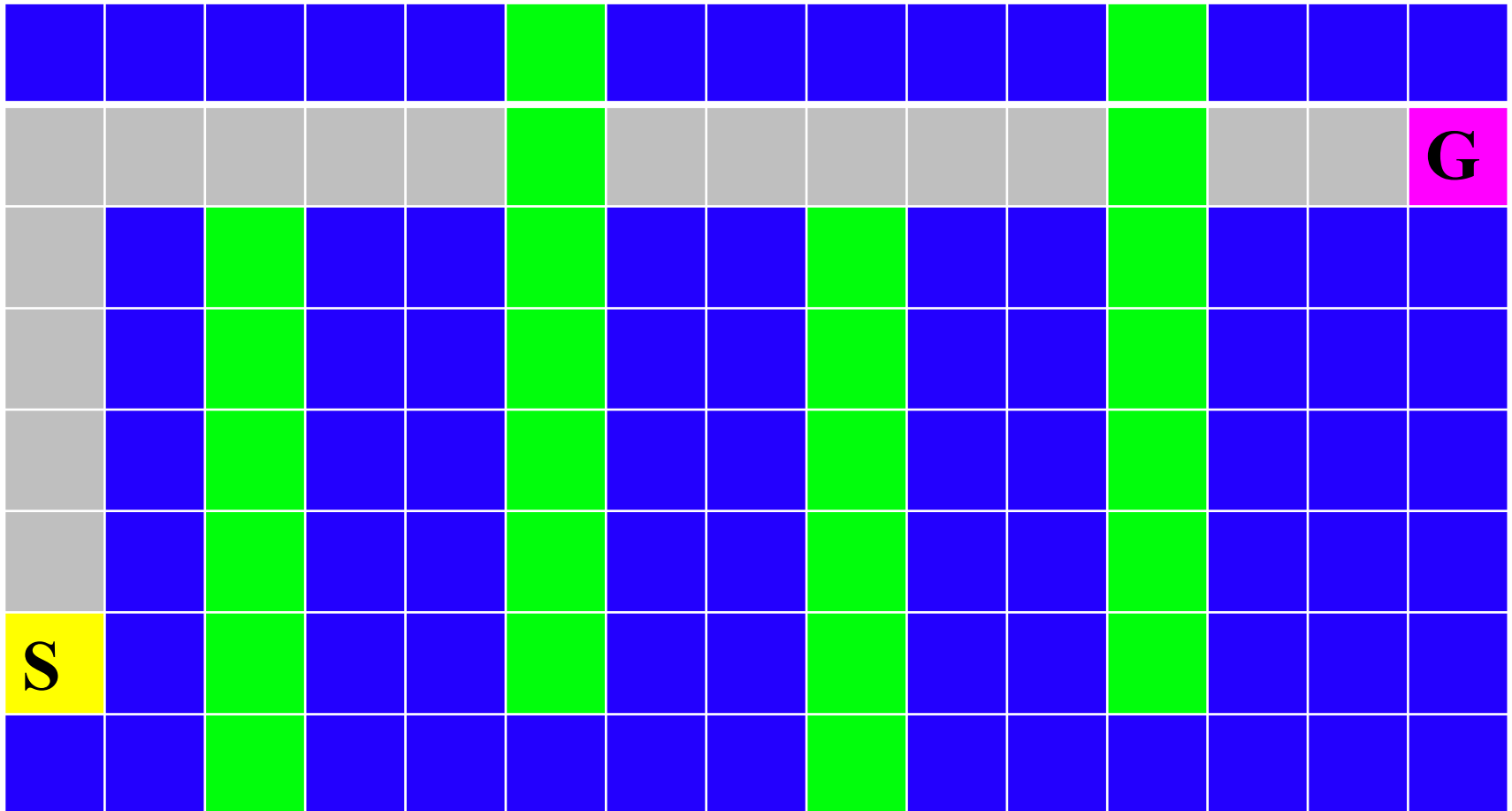
- Distanța Manhattan ignoră obstacolele
- În medii cu bariere → subestimare mare, euristică prea optimistă
- Euristica nu este foarte informativă, nu oferă o margine inferioară strânsă pentru $h^*(n)$



Funcții euristice

$h(n)$ = distanța Manhattan între nodul curent n și nodul scop n_{goal}

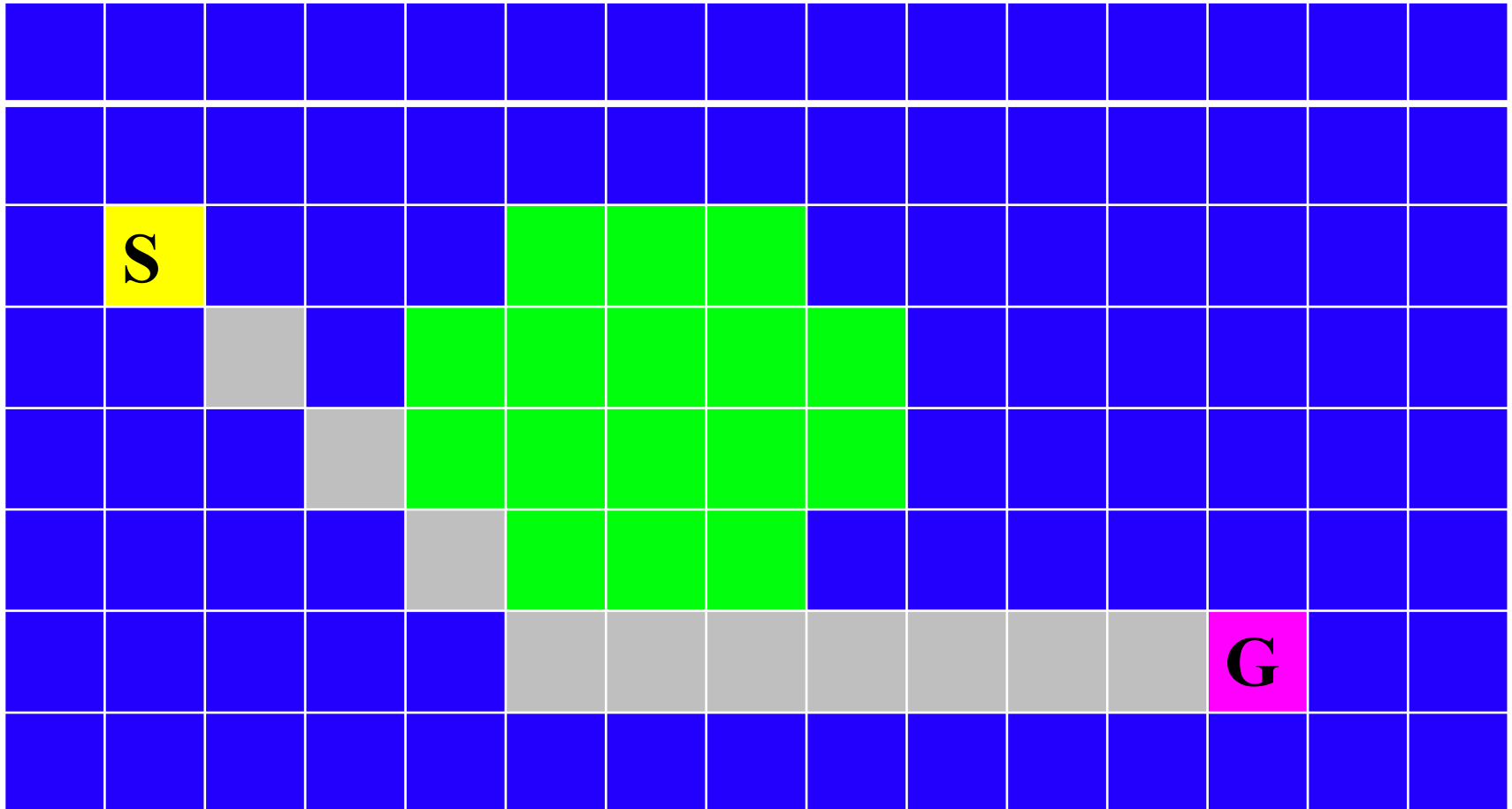
$$h(n) = d_{\text{Manhattan}}(n, n_{goal}) = |x_n - x_{goal}| + |y_n - y_{goal}|$$



Funcții euristice

Într-un grid cu mișcare în 8 direcții și cost unitar, euristica naturală este distanța Chebyshev.

$$h(n) = d_{\text{Chebyshev}}(n, n_{goal}) = \max(|x_n - x_{goal}|, |y_n - y_{goal}|)$$

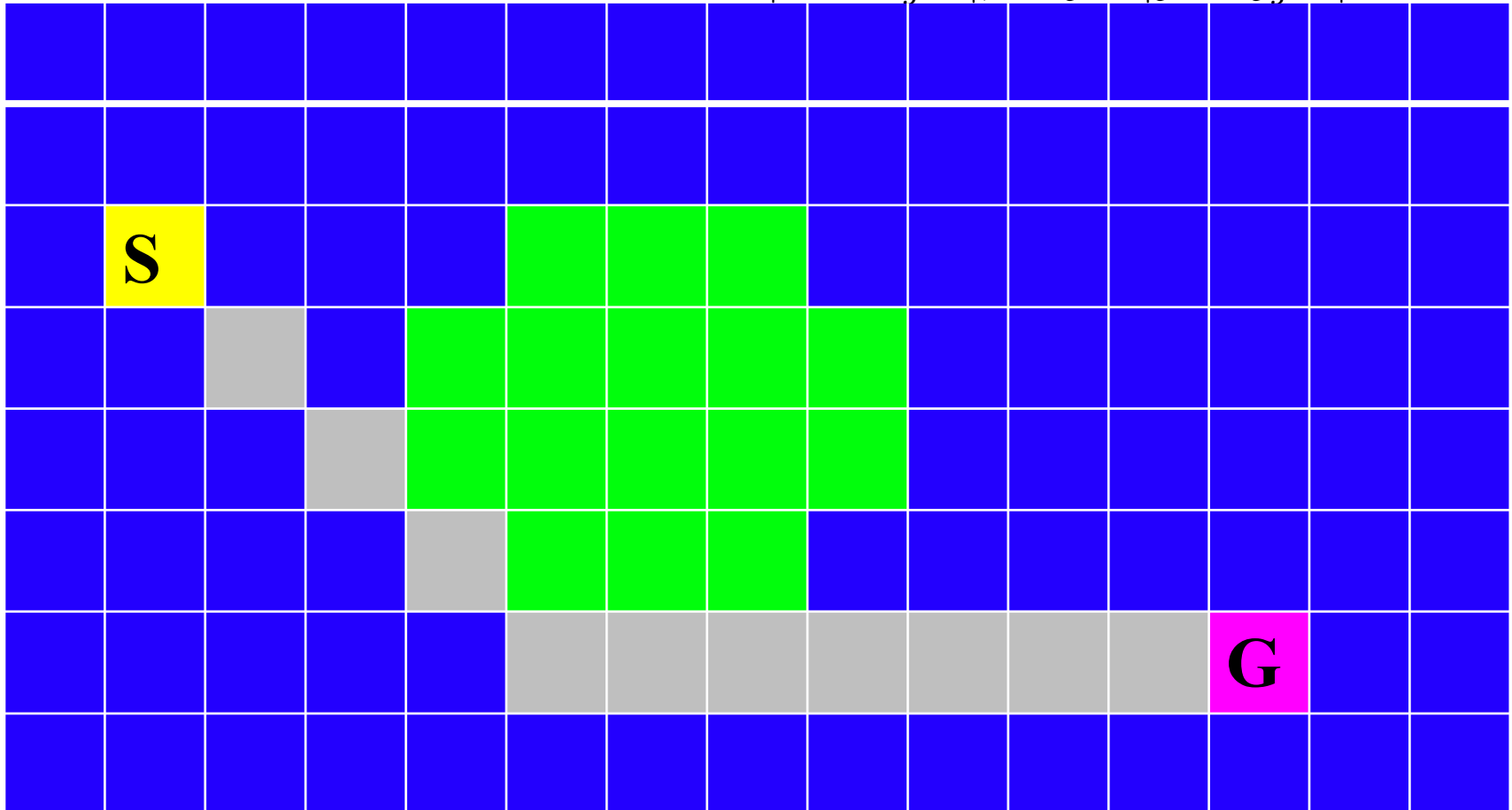


Funcții euristice

Într-un grid cu mișcare în 8 direcții și costuri neuniforme (1 și $\sqrt{2}$), euristica naturală este distanța Octile.

$$h(n) = d_{\text{Octile}}(n, n_{goal}) = \max(dx, dy) + (\sqrt{2} - 1) \cdot \min(dx, dy)$$

$$dx = |x_n - x_{goal}|, \quad dy = |y_n - y_{goal}|$$



Căutare Greedy folosind TREE-SEARCH

funcția **GBeFS-TREE-SEARCH** (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**

ciclează

dacă frontiera este vidă **atunci returnează** eșec

nod = **pop_min_f(Open List)**

dacă nodul conține o **stare scop**

atunci returnează soluția corespunzătoare

expandează nodul ales, adăugând nodurile succesori generate în **frontieră**

Frontiera (OPEN LIST) pentru Greedy Best First Search:

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a euristicii**

Căutare Greedy folosind GRAPH-SEARCH

funcția GBeFS-GRAPH-SEARCH (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**
inițializează mulțimea (Closed List) de stări explorate (vizitate) cu mulțimea vidă
ciclează

dacă frontiera este vidă **atunci returnează** eșec

nod = pop_min_f(Open List)

dacă nodul conține o **stare scop**

atunci returnează soluția corespunzătoare

dacă starea nodului curent este în Closed List

atunci continuă

adaugă starea nodului curent la Closed List

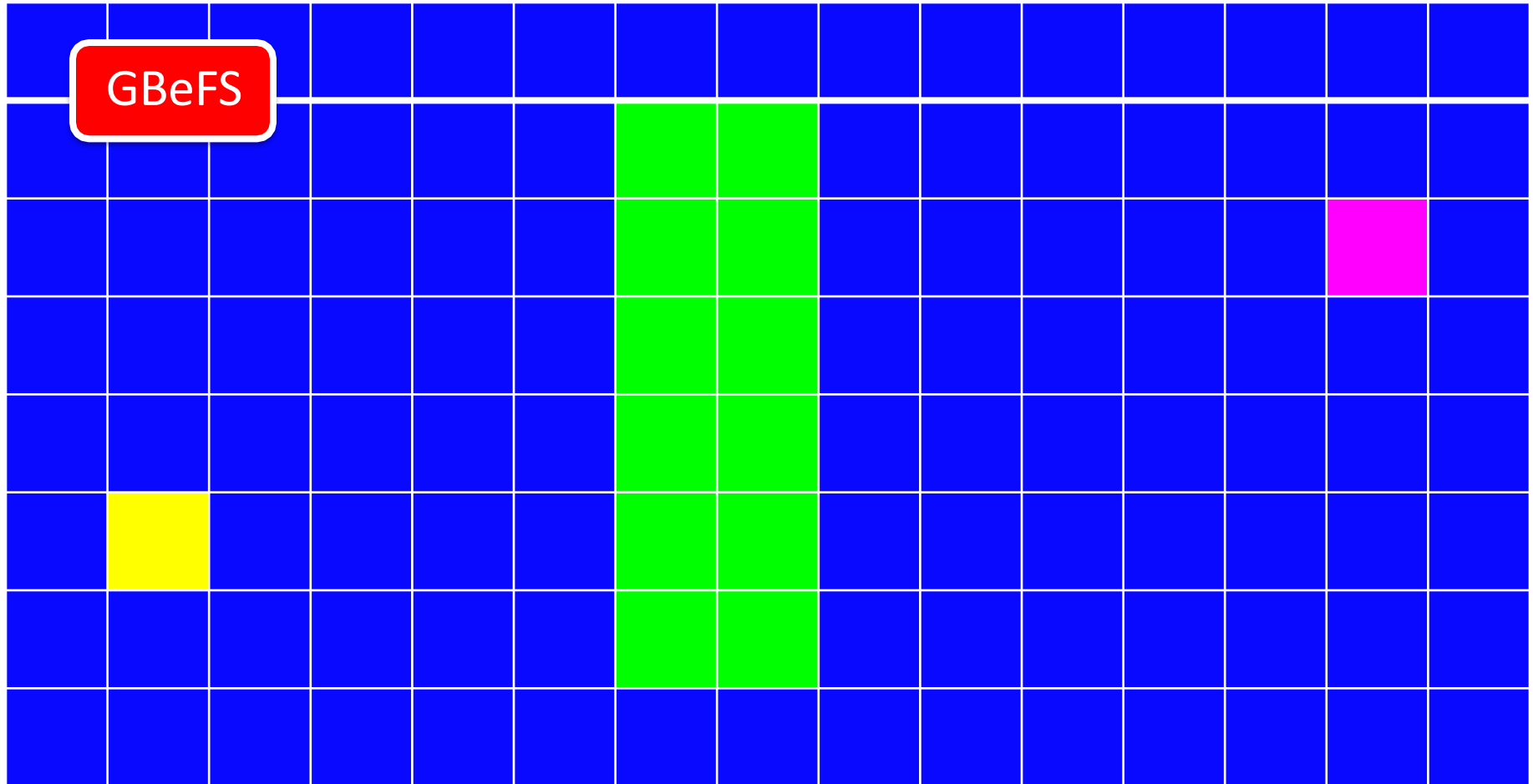
expandează nodul ales, adăugând nodurile succesori generate în frontieră

Frontiera (OPEN LIST) pentru Greedy Best First Search:

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a euristicii**

Căutare Greedy = Greedy Best First Search

Grid cu mișcare în 8 direcții.

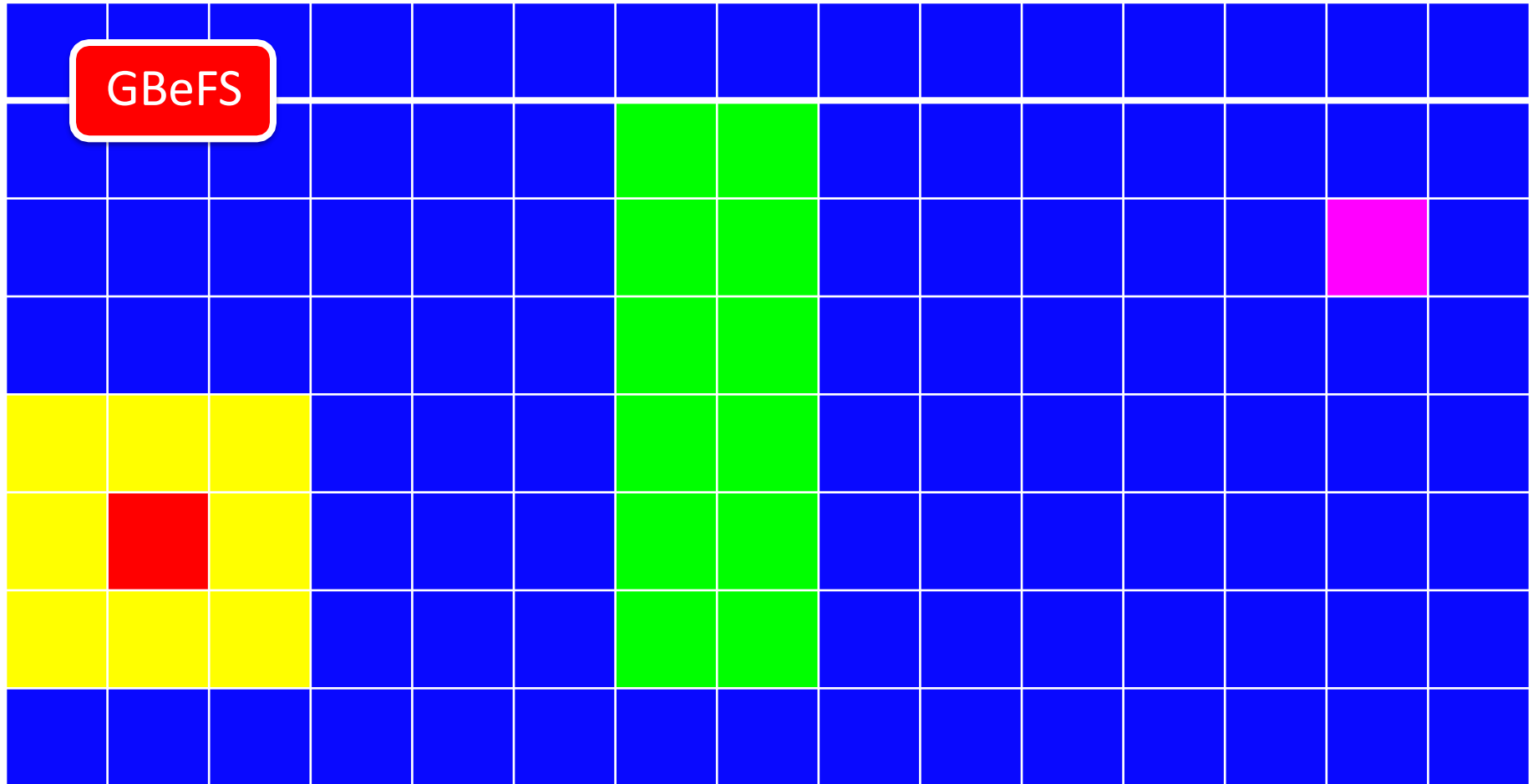


Cu galben nodurile din OPEN LIST
Cu roșu nodurile din CLOSED LIST

$$h(n) = d_{\text{euclid}}(n, n_{\text{goal}})$$

Căutare Greedy = Greedy Best First Search

Grid cu mișcare în 8 direcții.

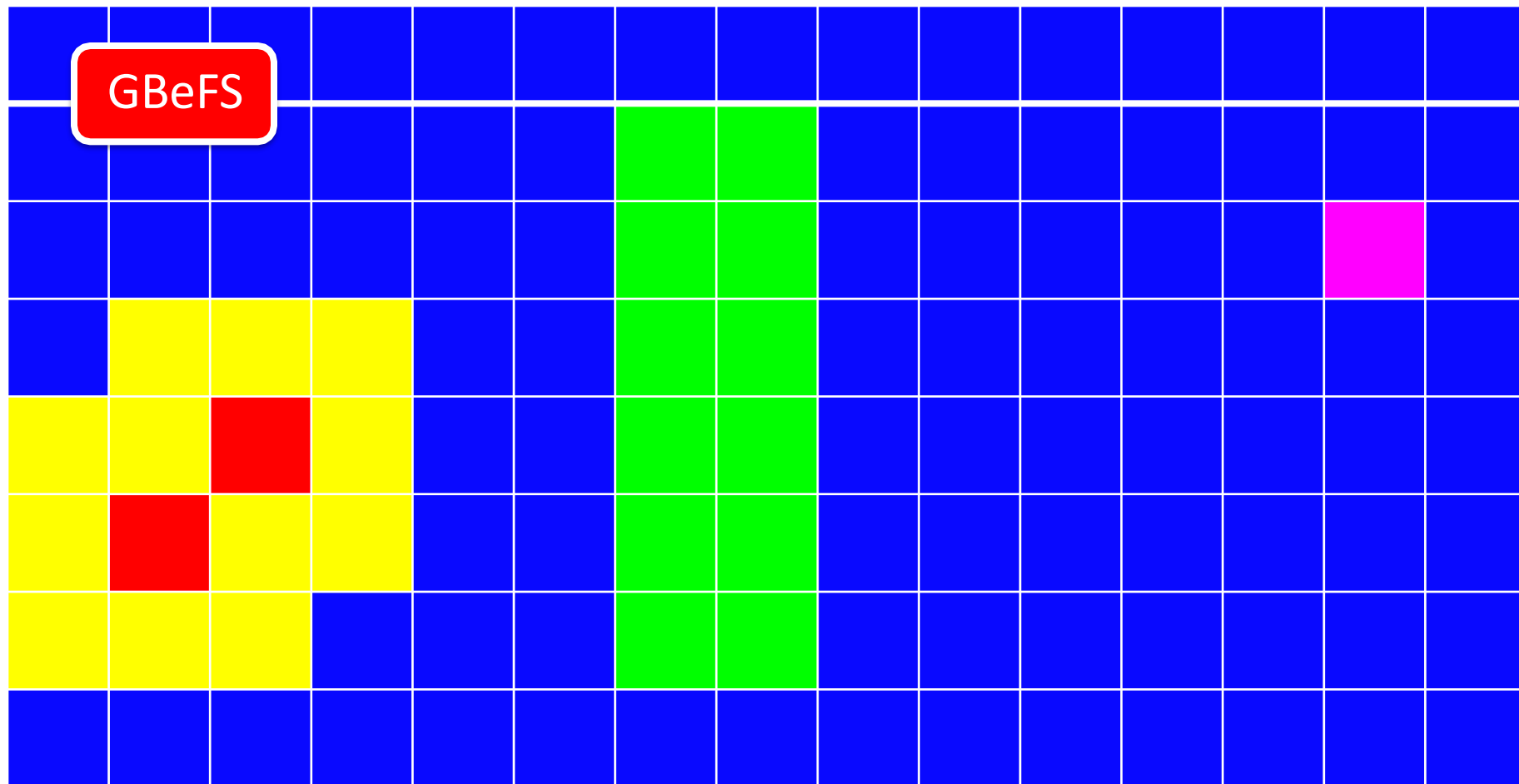


Cu galben nodurile din OPEN LIST
Cu roșu nodurile din CLOSED LIST

$$h(n) = d_{\text{euclid}}(n, n_{\text{goal}})$$

Căutare Greedy = Greedy Best First Search

Grid cu mișcare în 8 direcții.

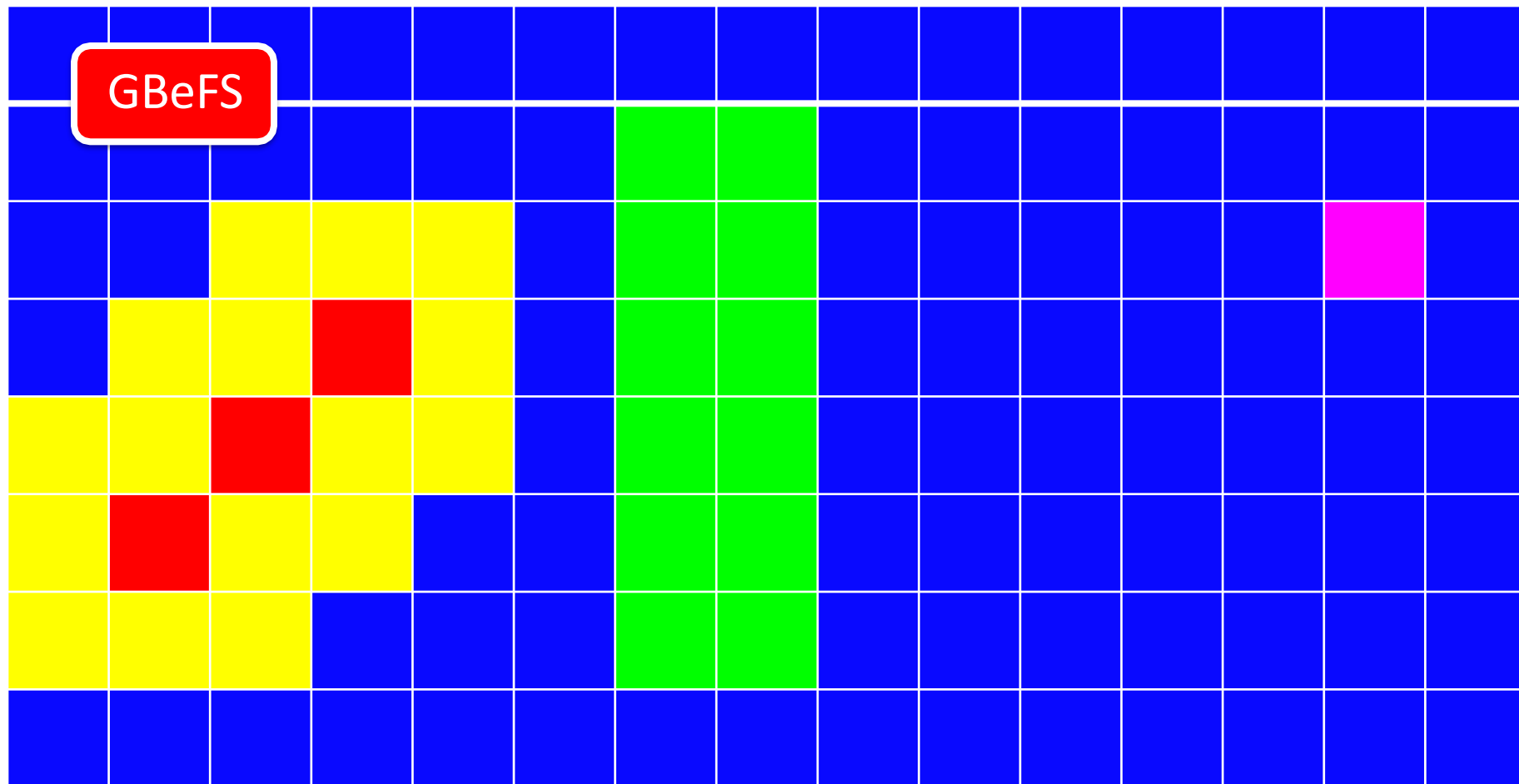


Cu galben nodurile din OPEN LIST
Cu roșu nodurile din CLOSED LIST

$$h(n) = d_{\text{euclid}}(n, n_{\text{goal}})$$

Căutare Greedy = Greedy Best First Search

Grid cu mișcare în 8 direcții.

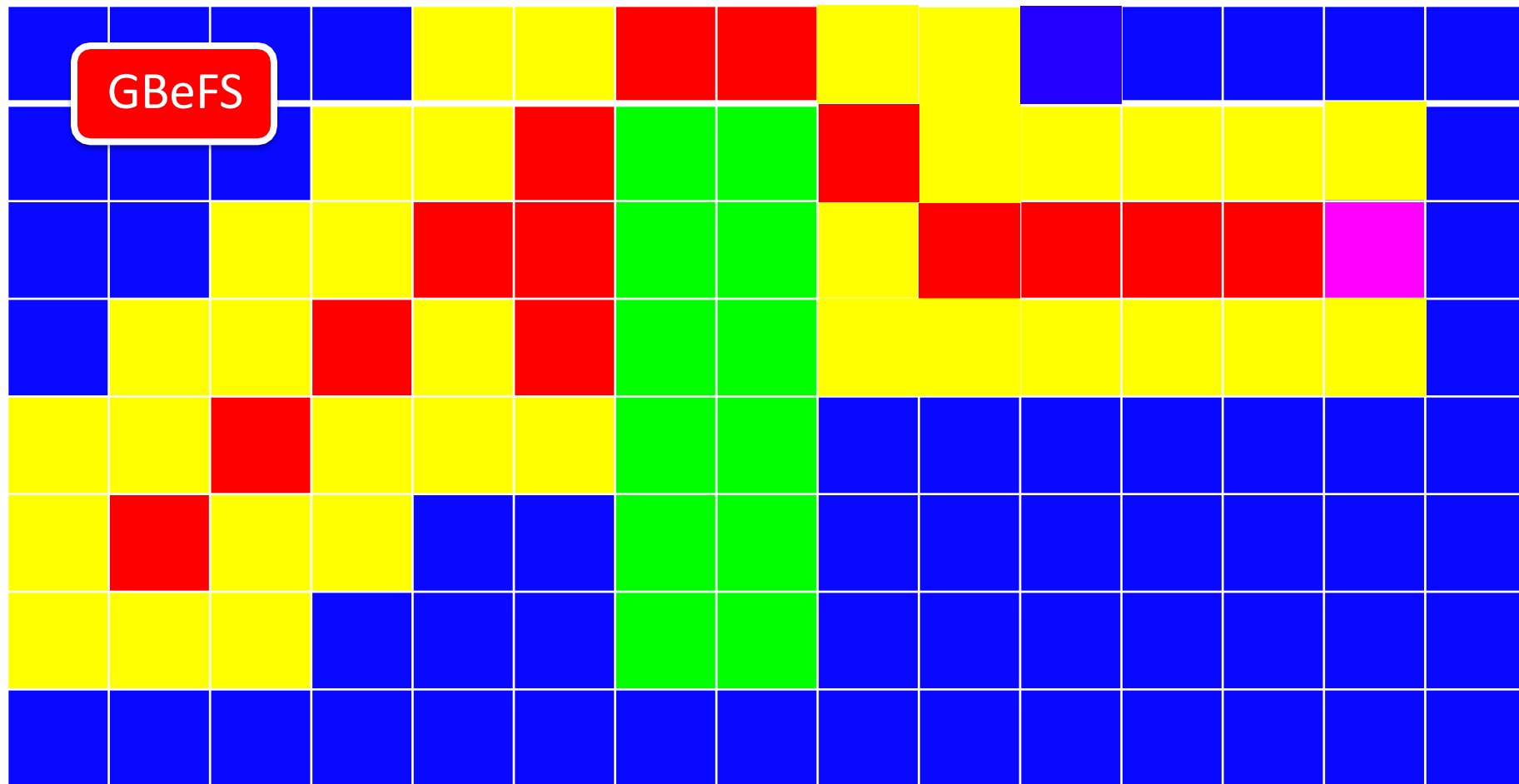


Cu galben nodurile din OPEN LIST
Cu roșu nodurile din CLOSED LIST

$$h(n) = d_{\text{euclid}}(n, n_{\text{goal}})$$

Căutare Greedy = Greedy Best First Search

Grid cu mișcare în 8 direcții.

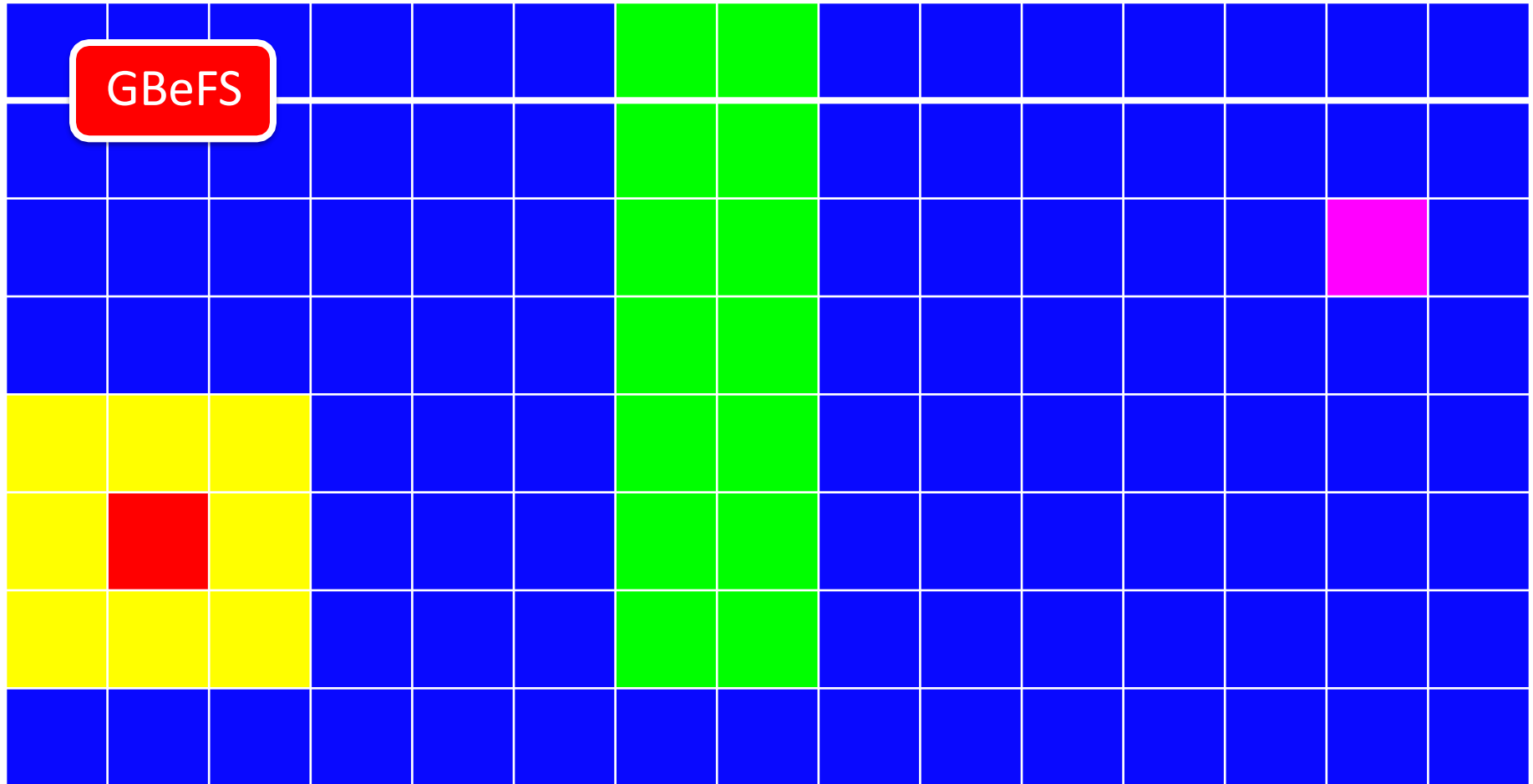


Cu galben nodurile din OPEN LIST
Cu roșu nodurile din CLOSED LIST

$$h(n) = d_{\text{euclid}}(n, n_{\text{goal}})$$

Căutare Greedy = Greedy Best First Search

Grid cu mișcare în 8 direcții.

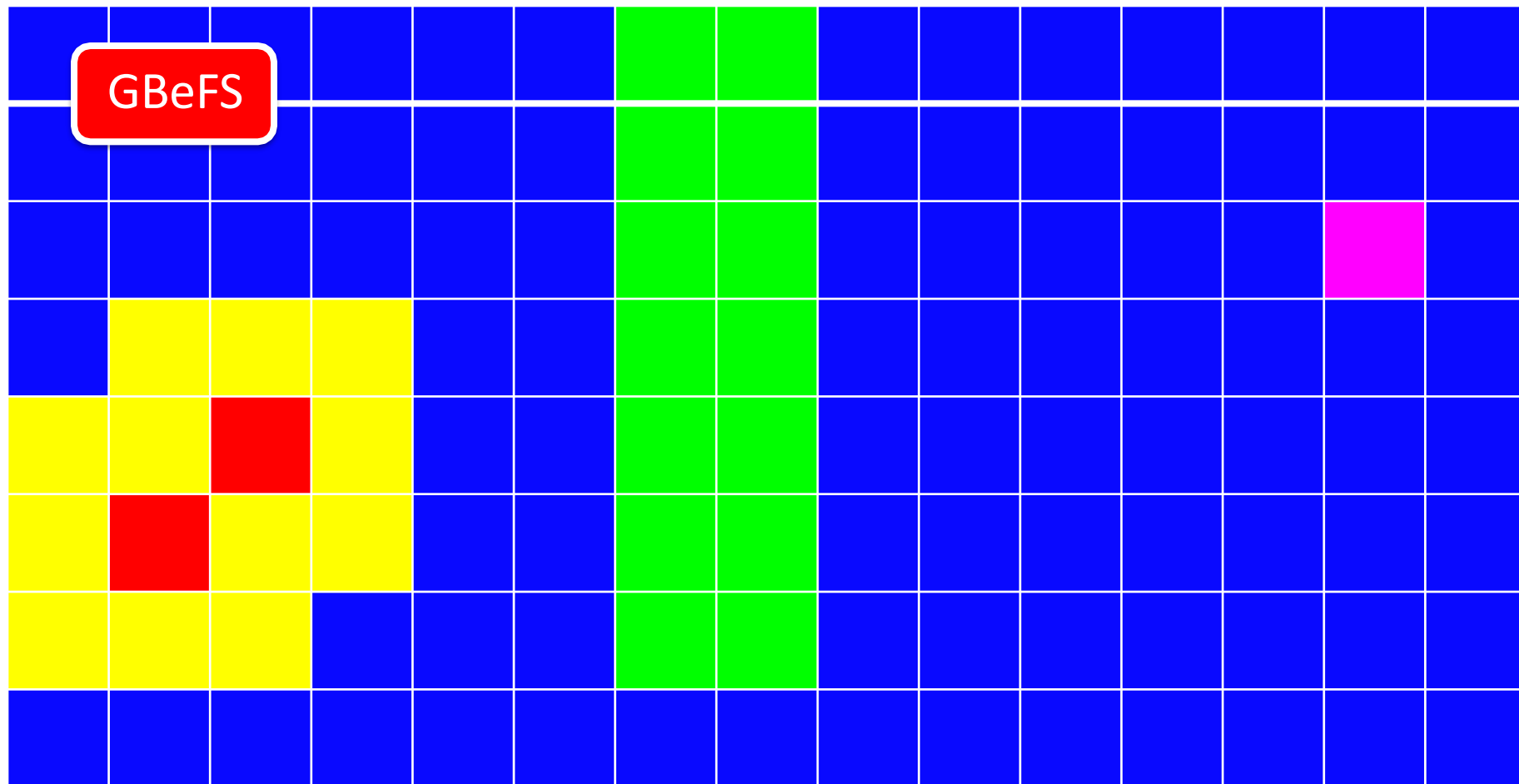


Cu galben nodurile din OPEN LIST
Cu roșu nodurile din CLOSED LIST

$$h(n) = d_{\text{euclid}}(n, n_{\text{goal}})$$

Căutare Greedy = Greedy Best First Search

Grid cu mișcare în 8 direcții.

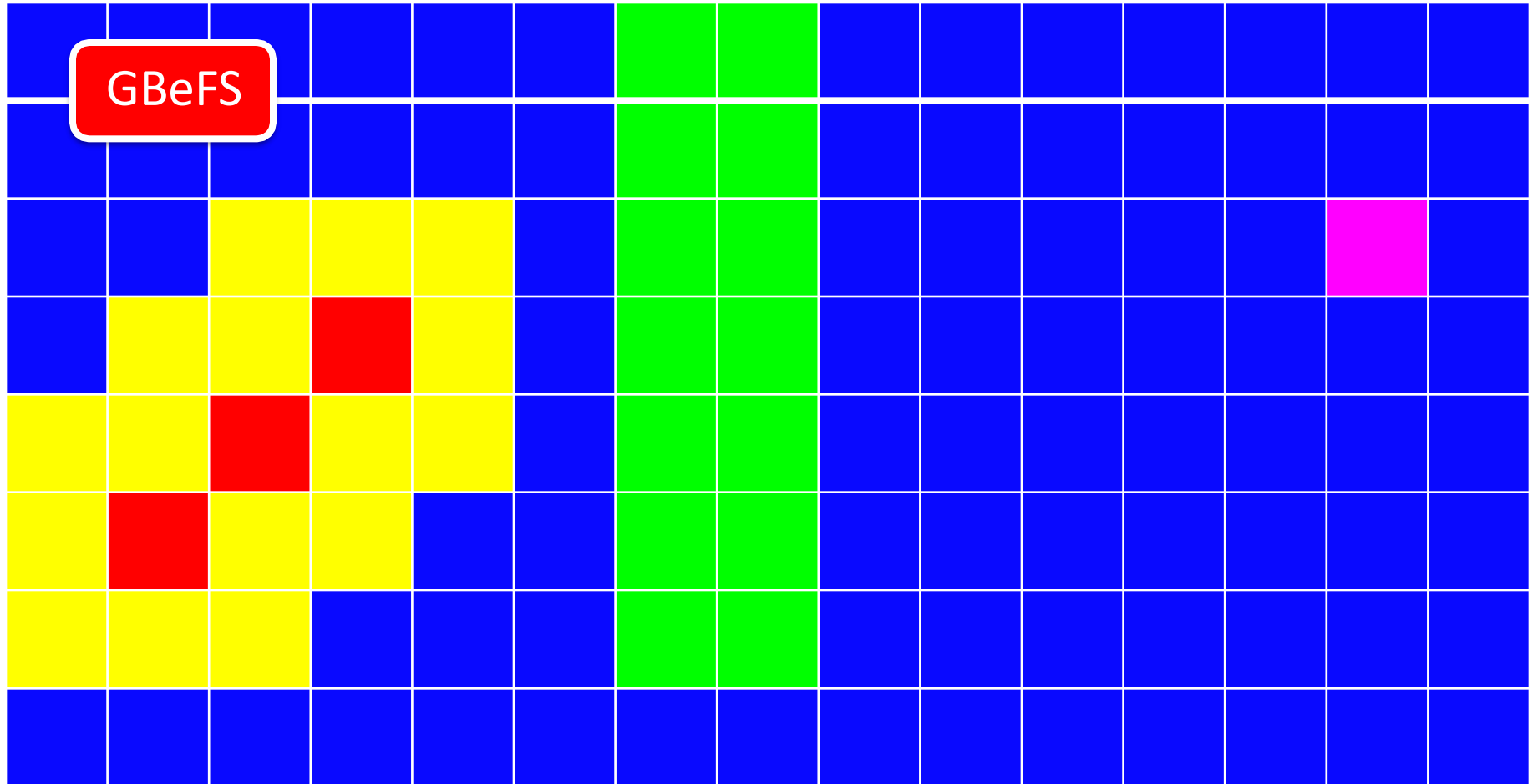


Cu galben nodurile din OPEN LIST
Cu roșu nodurile din CLOSED LIST

$$h(n) = d_{\text{euclid}}(n, n_{\text{goal}})$$

Căutare Greedy = Greedy Best First Search

Grid cu mișcare în 8 direcții.

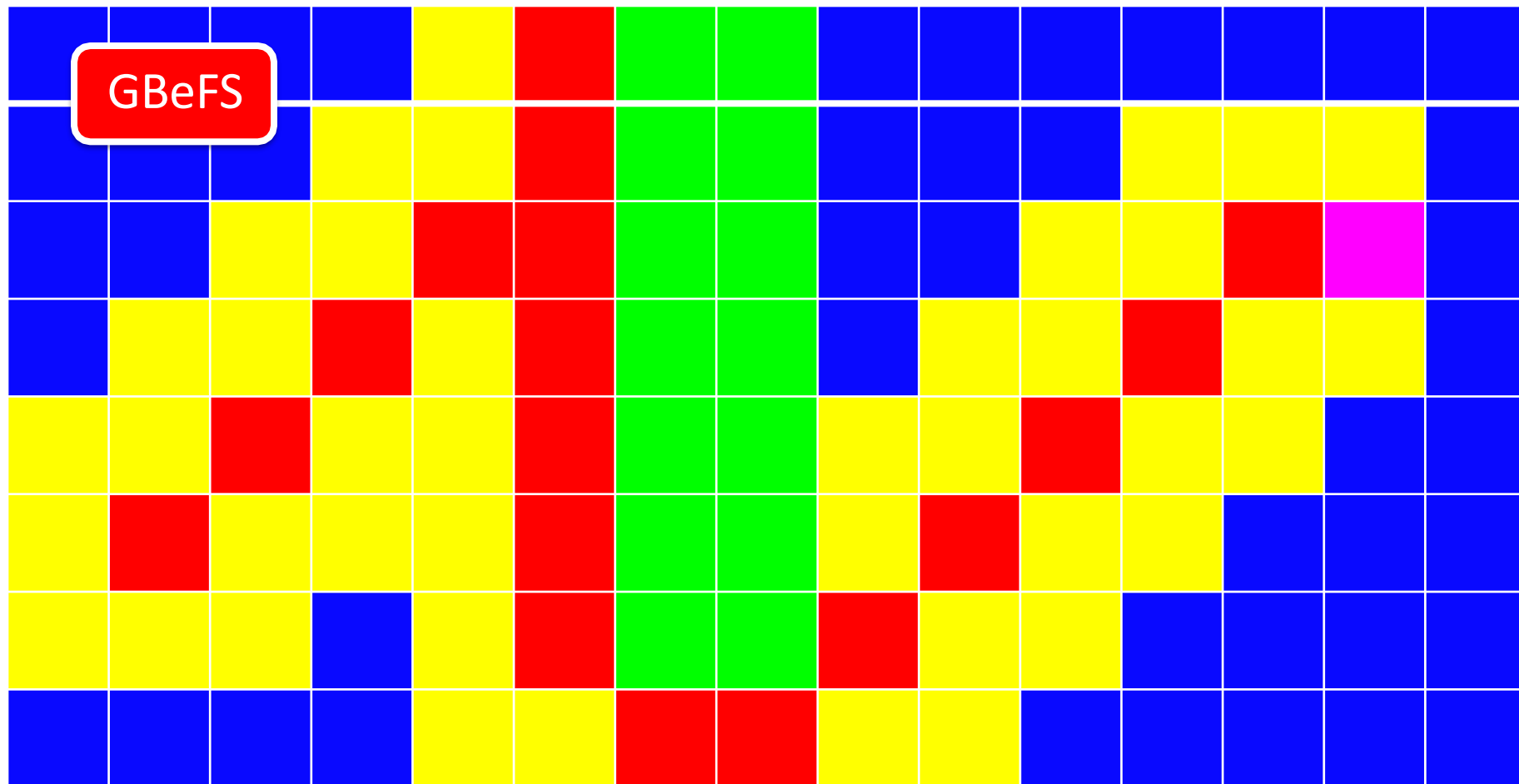


Cu galben nodurile din OPEN LIST
Cu roșu nodurile din CLOSED LIST

$$h(n) = d_{\text{euclid}}(n, n_{\text{goal}})$$

Căutare Greedy = Greedy Best First Search

Grid cu mișcare în 8 direcții.



Cu galben nodurile din OPEN LIST
Cu roșu nodurile din CLOSED LIST

$$h(n) = d_{\text{euclid}}(n, n_{\text{goal}})$$

Cuprinsul cursului de azi

1. Căutare informată

- Căutare Greedy (Greedy Best-First Search)
- Euristici admisibile
- Căutare A^* (A star Best-First Search)

Euristică admisibilă

- O euristică h este *admisibilă* (optimistă) dacă:

$$0 \leq h(n) \leq h^*(n)$$

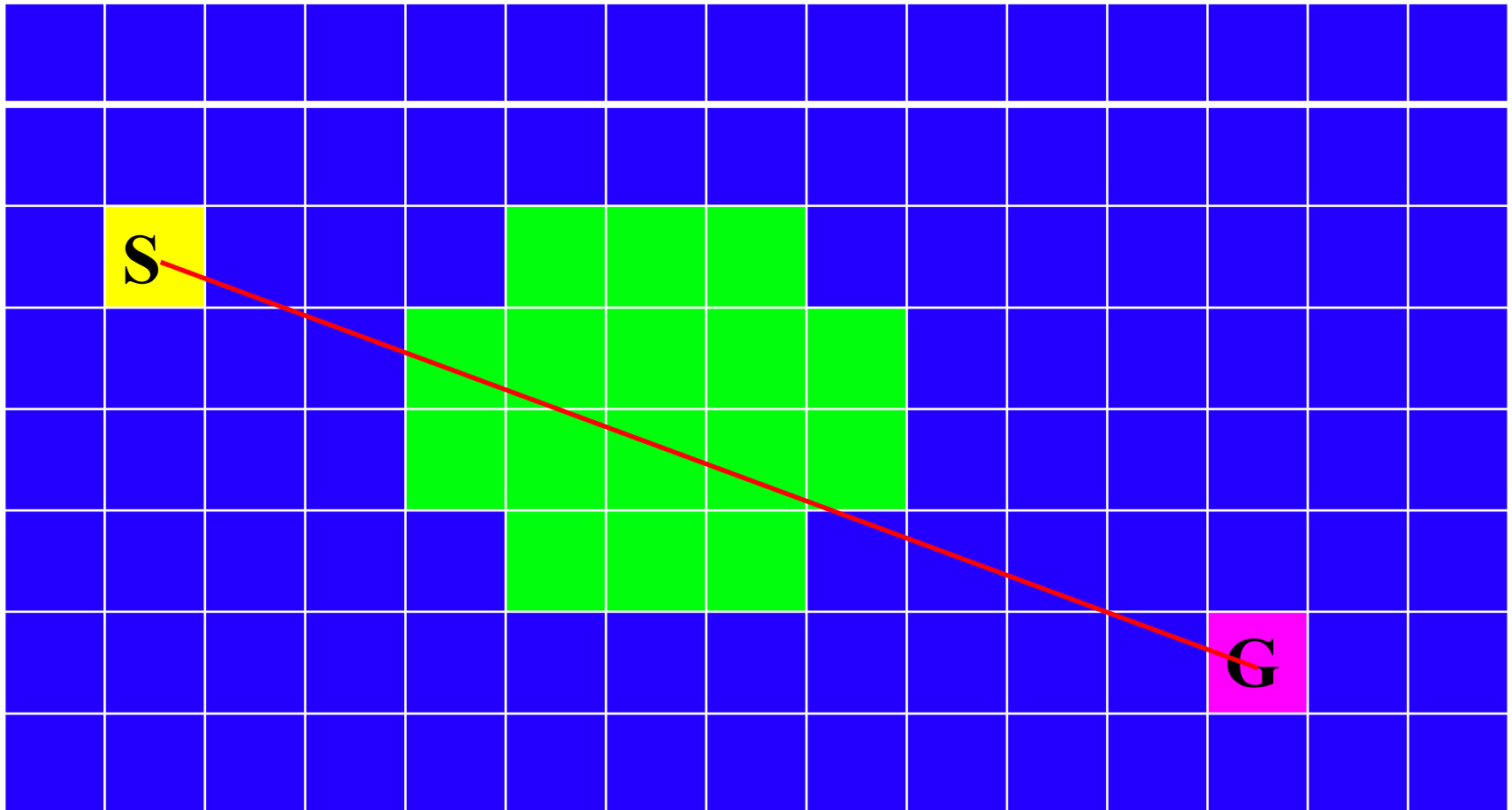
unde $h^*(n)$ este costul real către cel mai apropiat nod scop

- **O euristică admisibilă nu supraestimează niciodată distanța până la scop**
- **$f(n) = g(n) + h(n)$ nu supraestimează costul real al unui drum care trece prin nodul n , atunci când $h(n)$ este admisibilă**

Euristică admisibilă

$h(n)$ = distanța euclidiană între nodul curent n și nodul scop n_{goal}

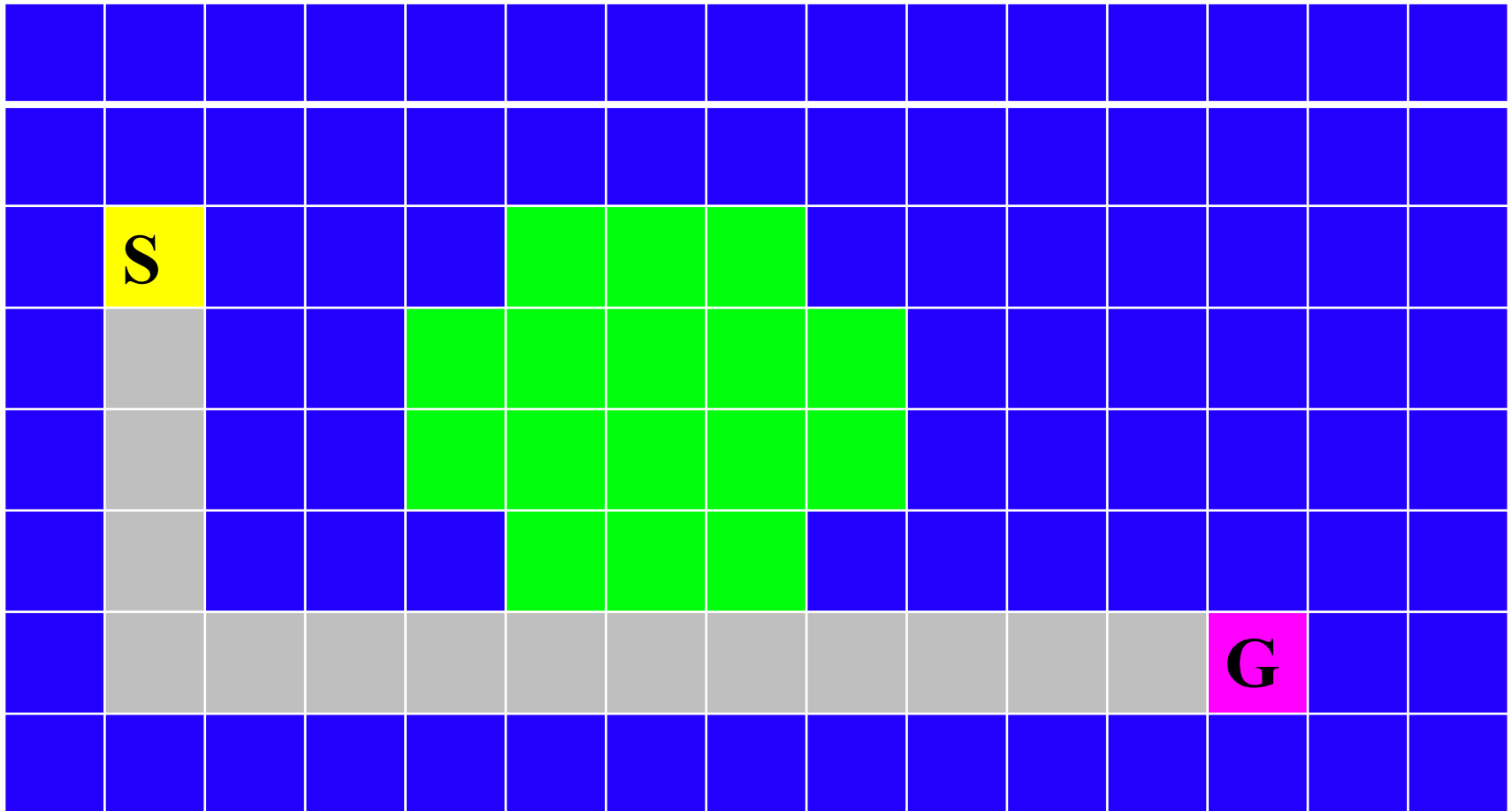
Este admisibilă? **DA**



Euristică admisibilă

$h(n)$ = distanța Manhattan între nodul curent n și nodul scop n_{goal}

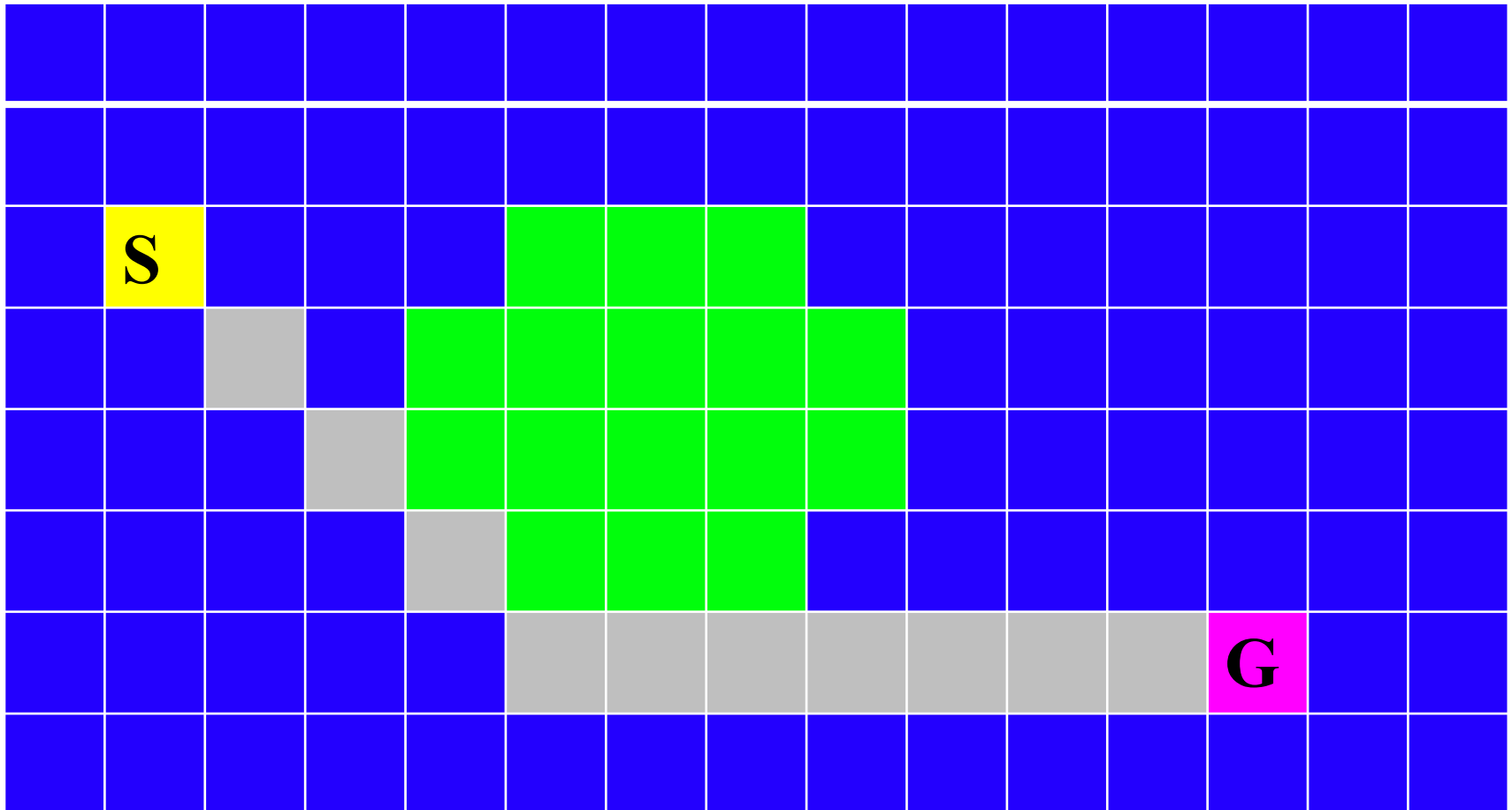
Este admisibilă? **DA – dacă mă pot mișca N, S, V, E (4 direcții)**



Euristică admisibilă

$$h(n) = \text{distanța Manhattan între nodul curent } n \text{ și nodul scop } n_{goal}$$

Este admisibilă? **NU – dacă mă pot mișca în 8 direcții**



Algoritmul de căutare A*

- Cel mai cunoscut algoritm de tip **Best-First Search**
- Evaluează nodurile n folosind:
 - $g(n)$ – costul exact de la starea inițială la n
 - $h(n)$ – estimarea costului de la n la scop
- Funcția de evaluare: $f(n) = g(n) + h(n)$
- $f(n)$ = estimarea costului total al unui drum optim care trece prin n
- Selectează nodul din **OPEN LIST** cu valoarea minimă a lui $f(n)$
- Performanța depinde de calitatea funcției euristice h

Căutare A* folosind TREE-SEARCH

funcția Astar-TREE-SEARCH (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**

ciclează

dacă frontiera este vidă **atunci returnează** eșec

nod = **pop_min_f(Open List)**

dacă nodul conține o **stare scop**

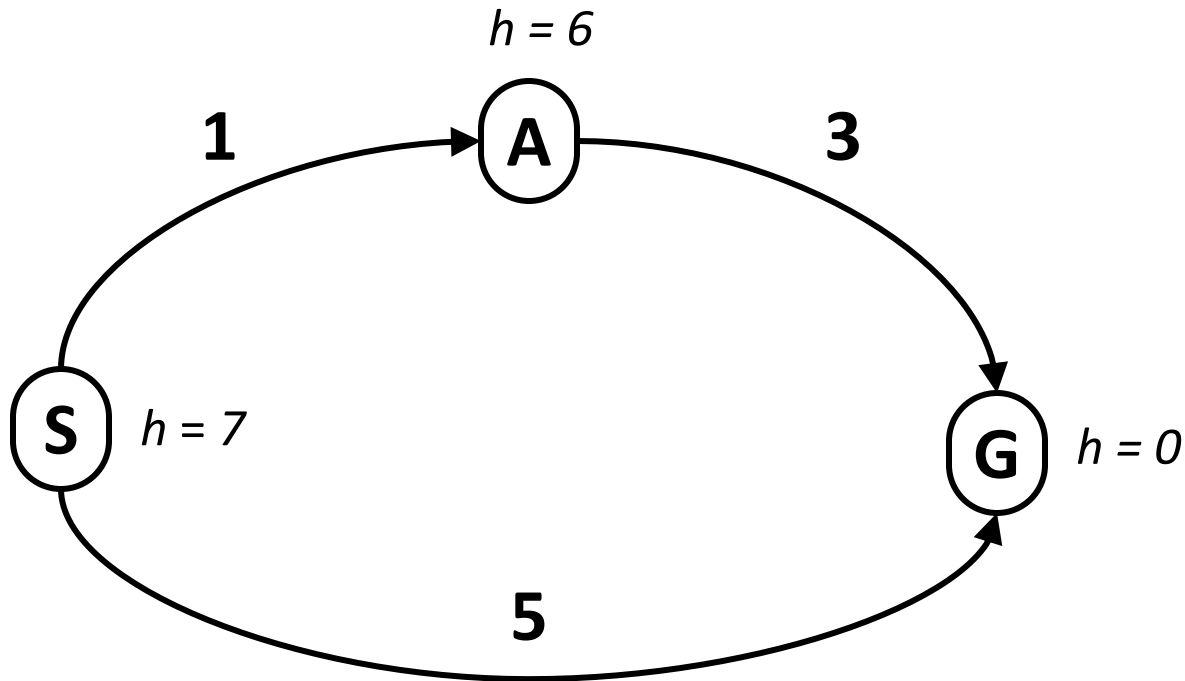
atunci returnează soluția corespunzătoare

expandează nodul ales, adăugând nodurile succesori generate în **frontieră**

Frontiera (OPEN LIST) pentru A*:

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = g(n) + h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a lui $f(n)$**

Este algoritmul de căutare A^* optimal?



- Găsește soluția S-G de cost 5 deși soluția optimă este S-A-G de cost 4.
- Avem nevoie de estimări (date de euristici) care să fie mai mici decât costurile reale (euristici optimiste)
- Optimalitate (în cazul Tree-Search) condiționată de euristici cu o asemenea caracteristică: A^* este optim dacă avem euristici optimiste

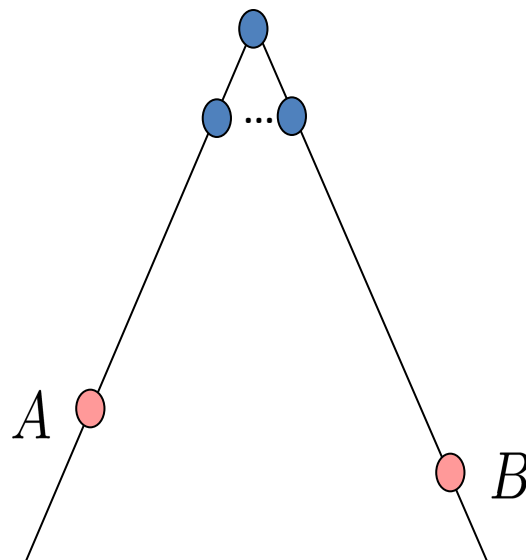
Optimalitatea algoritmului de căutare A^* folosind Tree-Search

Presupunem:

- A este un nod-scop optim global (nu există alt nod-scop cu cost mai mic)
- B este un nod-scop suboptimal (are cost mai mare decât soluția optimă)
- h este euristică admisibilă

Afirmație:

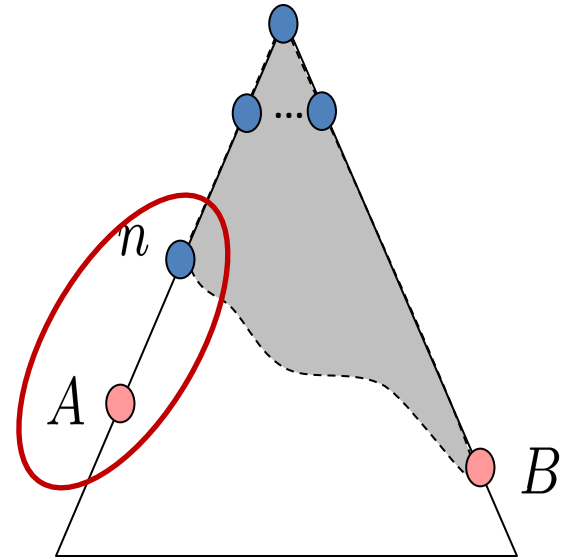
- A va fi explorat de algoritmul de căutare A^* înaintea lui B



Optimalitatea algoritmului de căutare A* folosind Tree-Search

Demonstrație:

- presupunem că B este pe frontieră
- un nod ascendent n al lui A (strămoș) este de asemenea pe frontieră (poate chiar nodul A!)
- afirmație: n va fi explorat înaintea nodului B
 1. $f(n) \leq f(A)$



$$f(n) = g(n) + h(n)$$

definiția costului f

$$f(n) \leq g(A) \quad \text{h subestimează costul până la A}$$

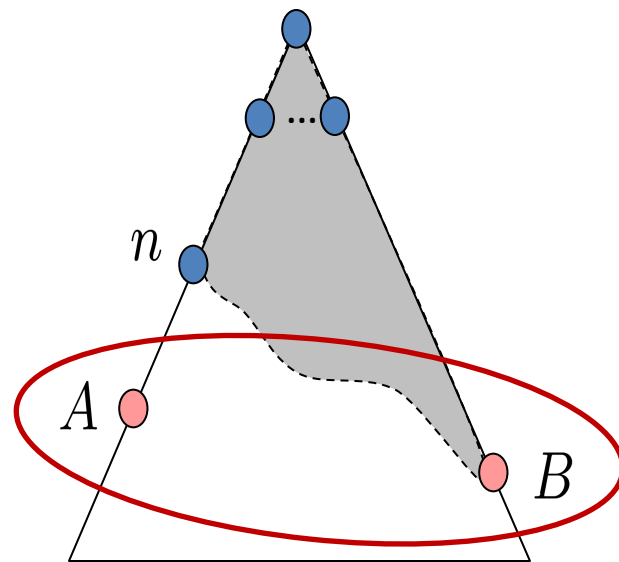
$$g(A) = f(A)$$

$h(A) = 0$, A e nod-scop

Optimalitatea algoritmului de căutare A* folosind Tree-Search

Demonstrație:

- presupunem că B este pe frontieră
- un nod ascendent n al lui A (strămoș) este de asemenea pe frontieră (poate chiar nodul A!)
- afirmație: n va fi explorat înaintea nodului B



$$g(A) < g(B)$$

B este nod suboptimal

$$f(A) < f(B)$$

$$\mathbf{h(A) = 0, h(B) = 0}$$

h e euristică

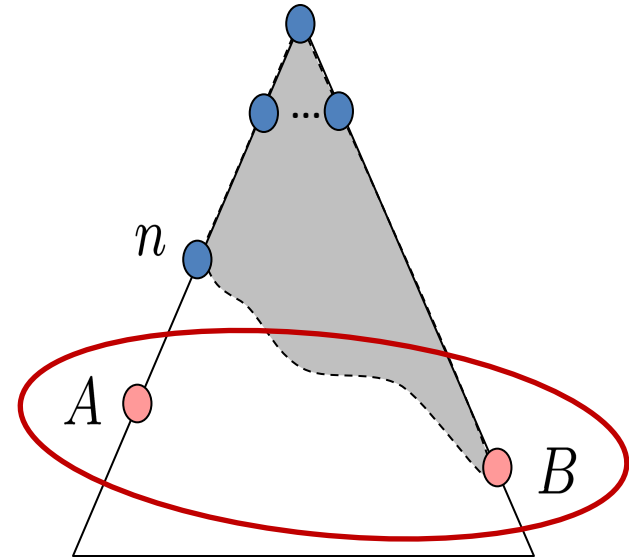
$$f(A) = g(A) + h(A) = g(A)$$

$$f(B) = g(B) + h(B) = g(B)$$

Optimalitatea algoritmului de căutare A* folosind Tree-Search

Demonstrație:

- presupunem că B este pe frontieră
- un nod ascendent n al lui A (strămoș) este de asemenea pe frontieră (poate chiar nodul A!)
- afirmație: n va fi explorat înaintea nodului B
 1. $f(n) \leq f(A)$
 2. $f(A) < f(B)$
 3. n este explorat înaintea lui B

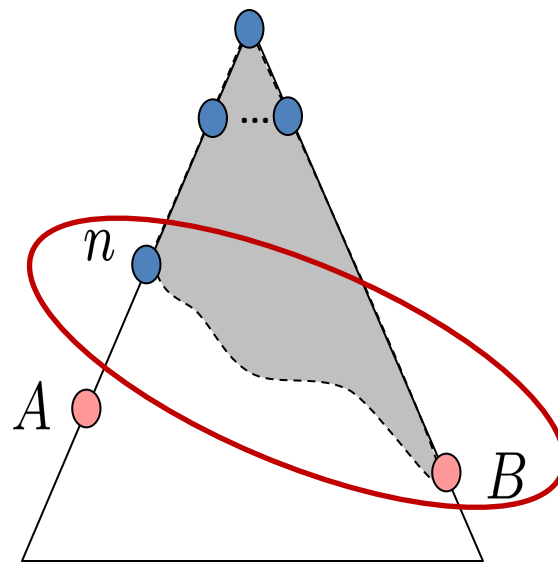


$$f(n) \leq f(A) < f(B)$$

Optimalitatea algoritmului de căutare A* folosind Tree-Search

Demonstrație:

- presupunem că B este pe frontieră
- un nod ascendent n al lui A (strămoș) este de asemenea pe frontieră (poate chiar nodul A!)
- afirmație: n va fi explorat înaintea nodului B
 1. $f(n) \leq f(A)$
 2. $f(A) < f(B)$
 3. n este explorat înaintea lui B
- toți ascendenții lui A sunt explorați înaintea lui B
- A este explorat înaintea lui B
- algoritmul A* este optimal



Căutare A* folosind TREE-SEARCH

funcția Astar-TREE-SEARCH (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**

ciclează

dacă frontiera este vidă **atunci returnează** eșec

nod = **pop_min_f(Open List)**

dacă nodul conține o **stare scop**

atunci returnează soluția corespunzătoare

expandează nodul ales, adăugând nodurile succesori generate în **frontieră**

Frontiera (OPEN LIST) pentru A*:

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = g(n) + h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a lui $f(n)$**

**Optimal dacă
h este admisibilă**

Căutare A* folosind GRAPH-SEARCH

funcția Astar--GRAPH-SEARCH (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**
inițializează mulțimea (Closed List) de stări explorate (vizitate) cu mulțimea vidă
ciclează

dacă frontiera este vidă **atunci returnează** eșec

nod = pop_min_f(Open List)

dacă nodul conține o **stare scop**

atunci returnează soluția corespunzătoare

dacă starea nodului curent este în Closed List

atunci continuă

adaugă starea nodului curent la Closed List

expandează nodul ales, adăugând nodurile succesori generate în frontieră

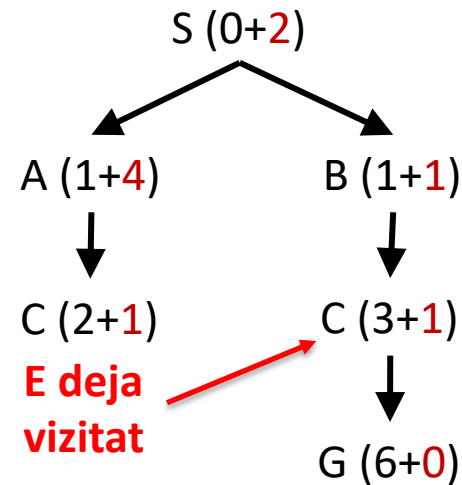
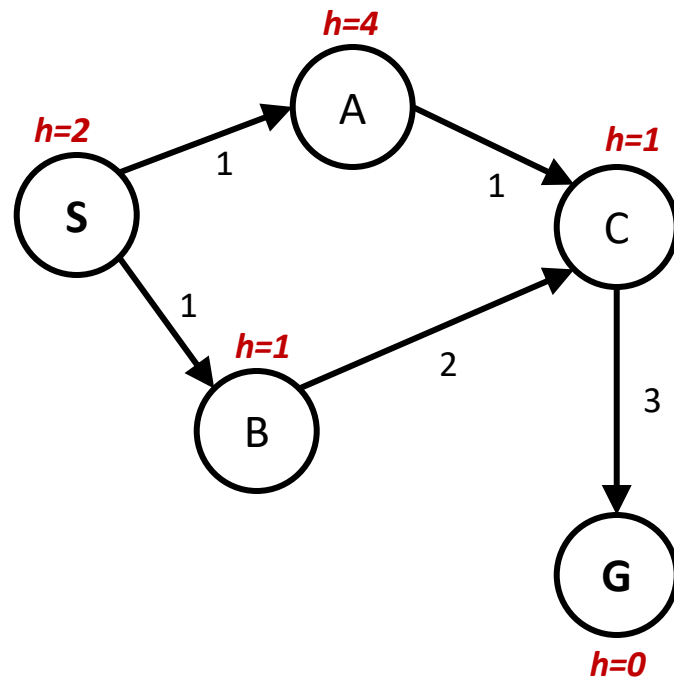
Frontiera (OPEN LIST) pentru A*:

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = g(n) + h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a lui $f(n)$**

Algoritmul A* folosind Graph-Search

- Poate returna soluții suboptimale chiar dacă folosim o euristică admisibilă
- A* folosind Graph-Search poate elimina un drum optim către o stare deja vizitată dacă acesta nu este primul generat (*un drum ulterior poate fi mai bun*)
- Problema apare deoarece nu reexpandează stările deja explorate
- Putem evita acest comportament impunând o condiție asupra euristicii (să fie consistentă)

Exemplu cu algoritmul A* folosind Graph-Search



Nu obțin soluția optimă
Obțin soluția S – B – C – G cu costul 6
Dar soluția S – A – C – G are costul 5

Euristică consistentă

- O euristică este consistentă (monotonă) dacă:

$$| h(a) - h(b) | \leq c(a,b)$$

unde:

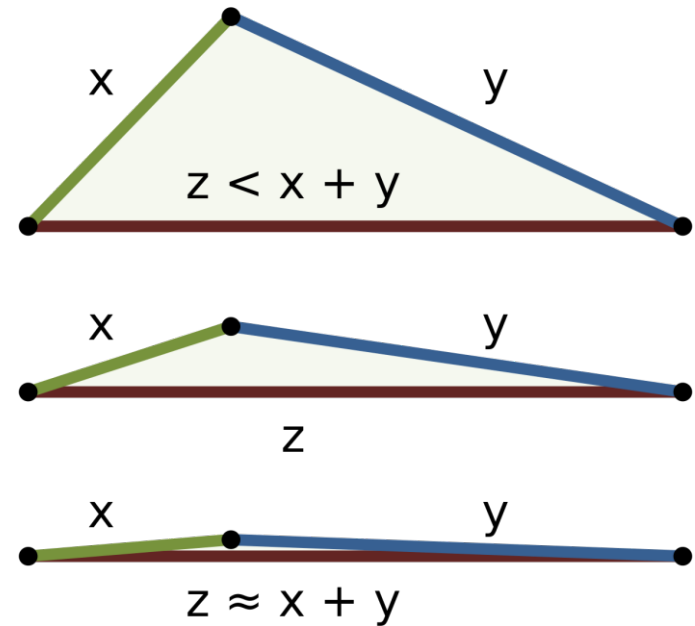
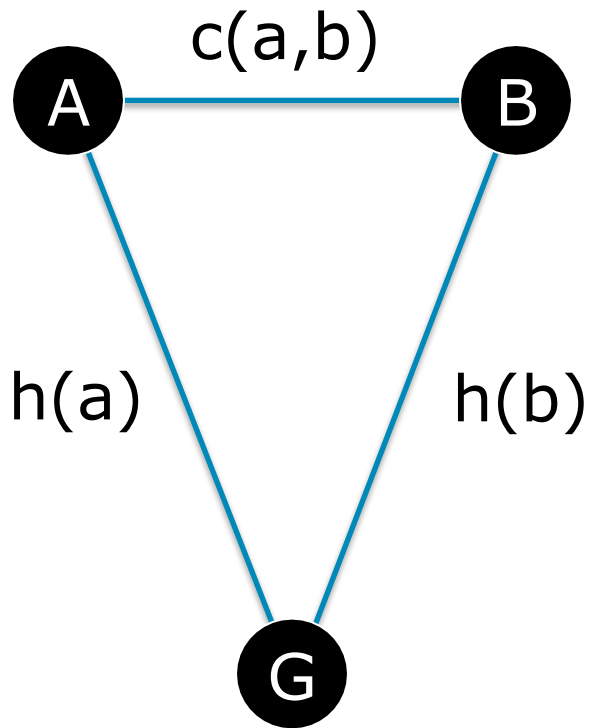
- $h(n)$ = estimarea costului de la nodul n până la scop
- $c(a,b)$ = costul real al tranziției de la a la b

Intuiție

- Diferența dintre valorile euristice a două stări vecine nu poate depăși costul real dintre ele
- Euristica h „nu sare” prea mult între stări

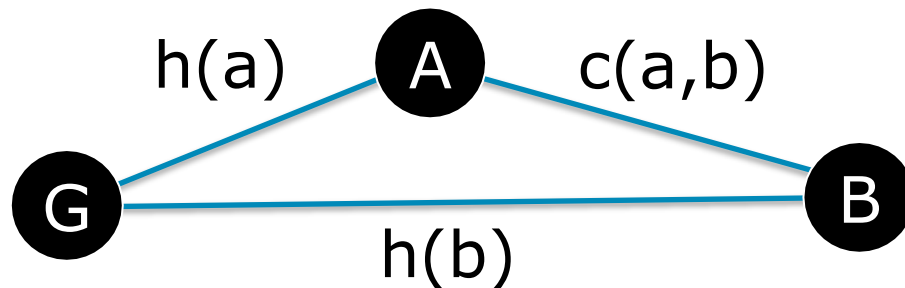
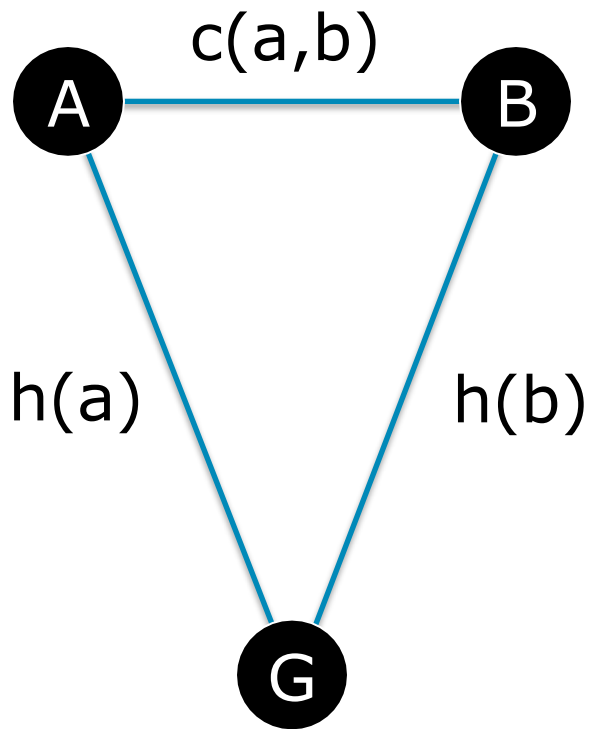
Euristică consistentă

$$| h(a) - h(b) | \leq c(a,b)$$



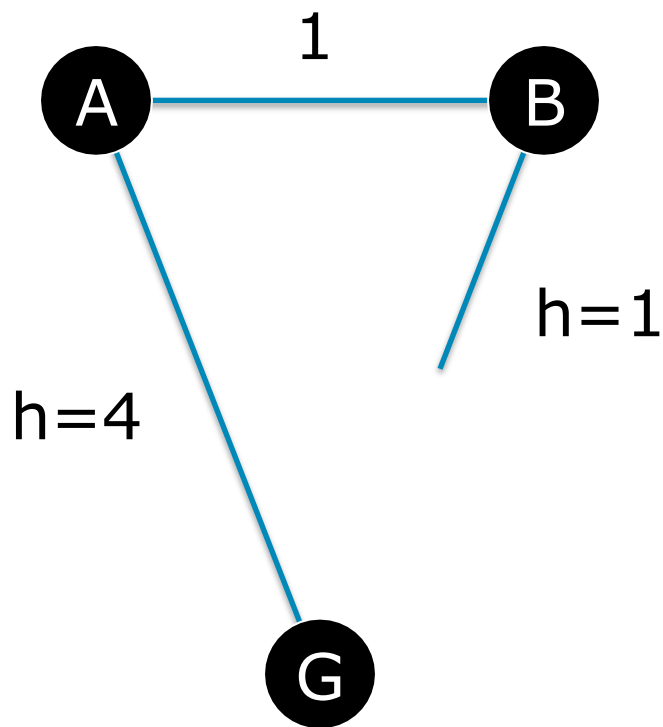
Euristică consistentă

$$| h(a) - h(b) | \leq c(a,b)$$



Euristică consistentă

$$| h(a) - h(b) | \leq c(a,b)$$



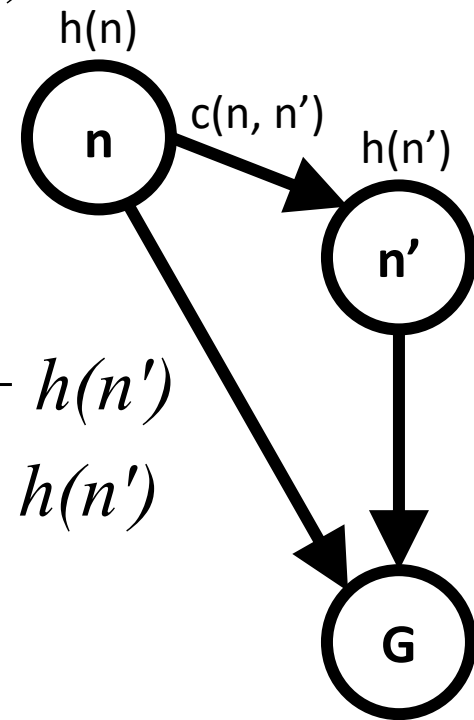
Inconsistentă

Proprietăți ale euristicii consistente

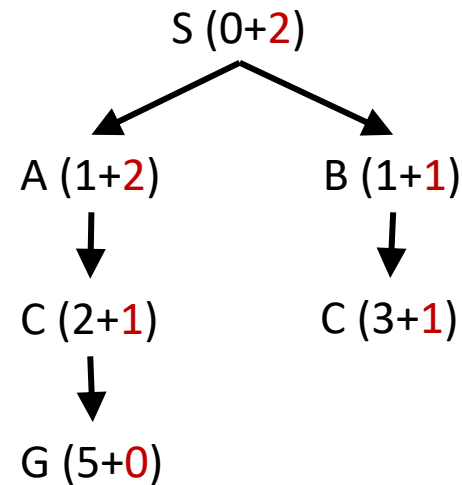
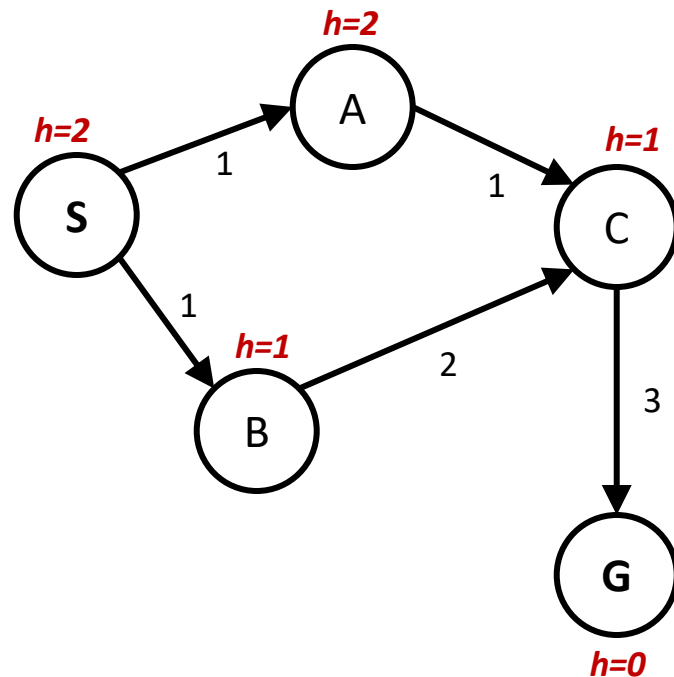
- Orice euristică consistentă este și admisibilă
- În practică, este dificil să construim o euristică admisibilă care să nu fie și consistentă
- Dacă $h(n)$ este consistentă, atunci valorile $f(n) = g(n) + h(n)$ sunt monotone nedescrescătoare de-a lungul oricărui drum
- Nodurile sunt expandate de către căutarea A^* folosind Graph-Search în ordine crescătoare a lui $f(n)$
- Primul nod scop selectat pentru expandare este optimal (pentru că celelalte noduri au cost cel puțin la fel de mare)

Proprietăți ale euristicii consistente

- Dacă $h(n)$ este consistentă, atunci valorile $f(n) = g(n) + h(n)$ sunt monotone nedescrescătoare de-a lungul oricărui drum
- Condiția de consistență: $h(n) \leq c(n, n') + h(n')$
- Pe un drum $n \rightarrow n' \rightarrow G$ avem:
 - $f(n) = g(n) + h(n)$
 - $f(n') = g(n') + h(n') = g(n) + c(n, n') + h(n')$
 - din consistență avem $h(n) \leq c(n, n') + h(n')$
 $\Rightarrow g(n) + h(n) \leq g(n) + c(n, n') + h(n')$
 $\Rightarrow f(n) \leq f(n')$



Exemplu cu algoritmul A* folosind Graph-Search



Obțin soluția optimă
S – A – C – G are costul 5

Căutare A* folosind GRAPH-SEARCH

funcția Astar--GRAPH-SEARCH (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**
inițializează mulțimea (Closed List) de stări explorate (vizitate) cu mulțimea vidă
ciclează

dacă frontiera este vidă **atunci returnează** eșec

nod = pop_min_f(Open List)

dacă nodul conține o **stare scop**

atunci returnează soluția corespunzătoare

dacă starea nodului curent este în Closed List

atunci continuă

adaugă starea nodului curent la Closed List

expandează nodul ales, adăugând nodurile succesori generate în frontieră

Frontiera (OPEN LIST) pentru A*:

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = g(n) + h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a lui $f(n)$**

**Optimal dacă
h este consistentă**

Implementare A* folosind GRAPH-SEARCH

```
1. Function AStar(problem, h)
2.   closed = {}
3.   open = PQ(f=g+h, Node(problem.init_state))
4.   while (true)
5.     if (open.empty) return fail
6.     node = pop_min_f(open)
7.     if (node.state is goal) return solution
8.     if (node.state in closed) continue
9.     closed.add(node.state)
10.    for c in Expand(node, problem)
11.      if (c.state in closed) continue

    open.add(c)
```

Dacă o stare a unui nod
succesor a fost deja
explorată (se află în
CLOSED), nu va mai fi
procesată din nou.

Reduce memoria
utilizată de OPEN List

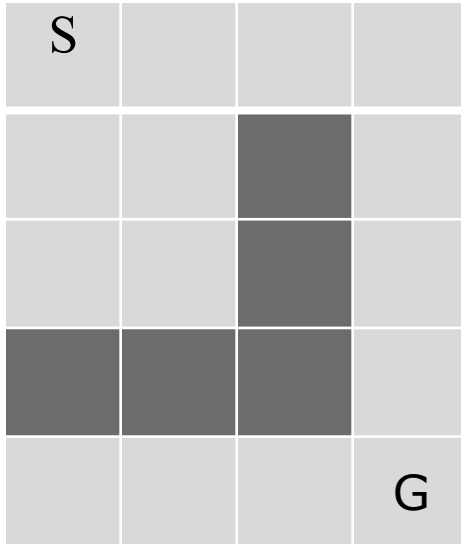
Implementare A* folosind GRAPH-SEARCH

```
1. Function AStar(problem, h)
2.   closed = {}
3.   open = PQ(f=g+h, Node(problem.init_state))
4.   while (true)
5.     if (open.empty) return fail
6.     node = pop_min_f(open)
7.     if (node.state is goal) return solution
8.     if (node.state in closed) continue
9.     closed.add(node.state)
10.    for c in Expand(node, problem)
11.      if (c.state in closed) continue
12.      if (node n in open with c.state and n.g <= c.g) continue
      open.add(c)
```

Dacă există deja un drum mai bun către această stare în OPEN, ignorăm nodul.

Poate fi foarte lent în practică

Exemplu A* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with <= g: continue
9.         else open.add(c)

```

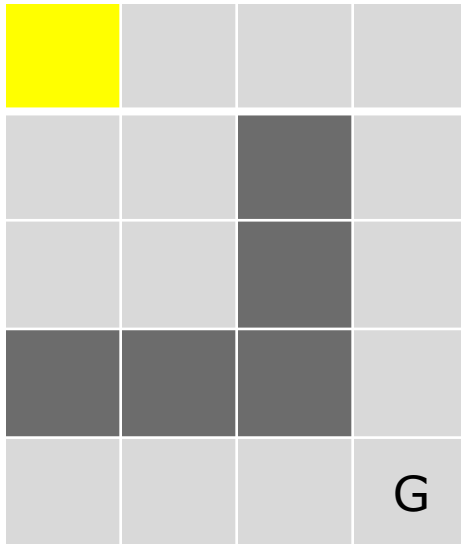
Open List (Noduri)

Closed

[illegible]

Cost acțiuni
N, S, V, E – 100
Diagonale - 141

Exemplu A* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

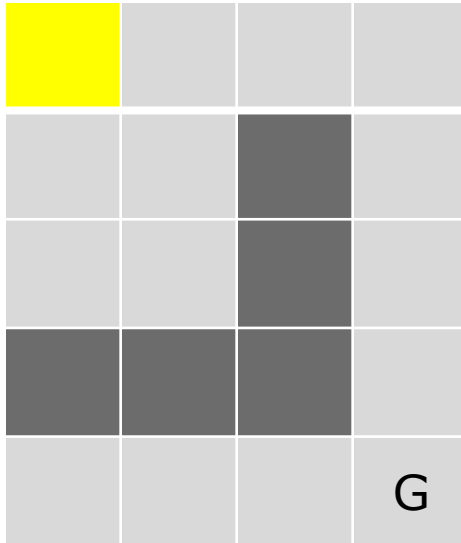
```

Open List (Noduri)

Closed

[illegible]

Exemplu A* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

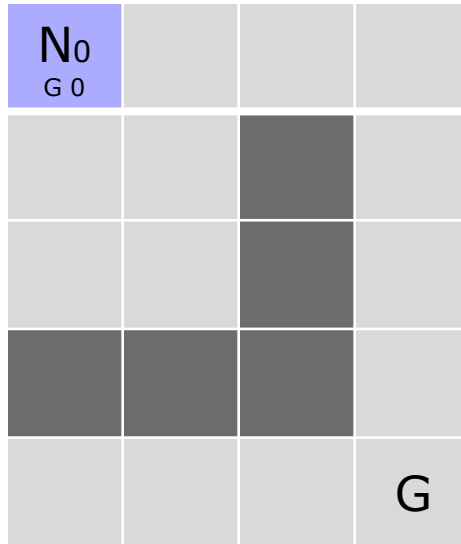
```

Open List (Noduri)

Closed

[illegible]

Exemplu A* folosind GRAPH-SEARCH



Open List (Noduri)

Closed

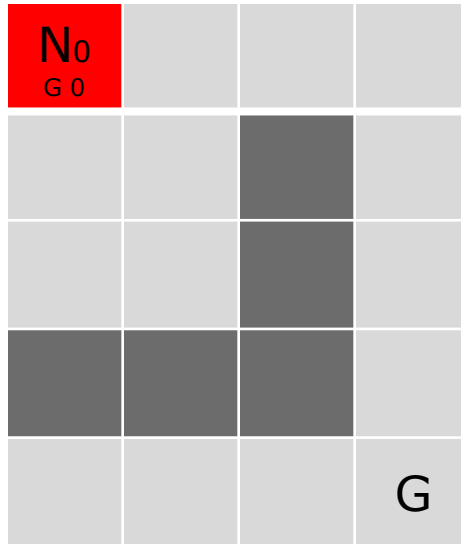
[illegible]

- ```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

Open List (Noduri)

| ID | S | G | H | F | A | P |
|----|---|---|---|---|---|---|
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |

Closed

Stare

0, 0

# Exemplu A\* folosind GRAPH-SEARCH

|                       |            |  |   |
|-----------------------|------------|--|---|
| N <sub>0</sub><br>G 0 | C<br>G 100 |  |   |
|                       |            |  |   |
|                       |            |  |   |
|                       |            |  |   |
|                       |            |  |   |
|                       |            |  | G |

```

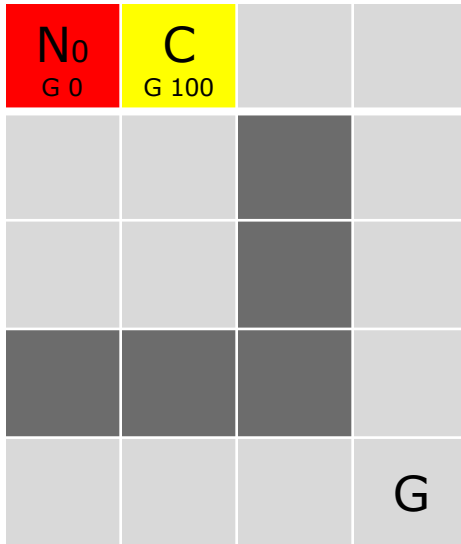
1. while (!open.empty)
2. node ← remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

[illegible]



# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

## Open List (Noduri)

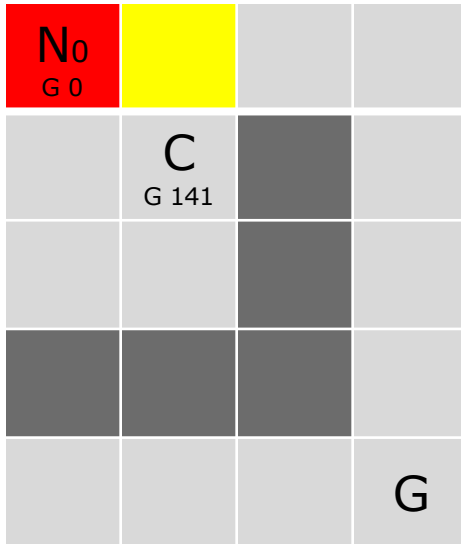
| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0 | 0 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

### Stare

0, 0

# Exemplu A\* folosind GRAPH-SEARCH



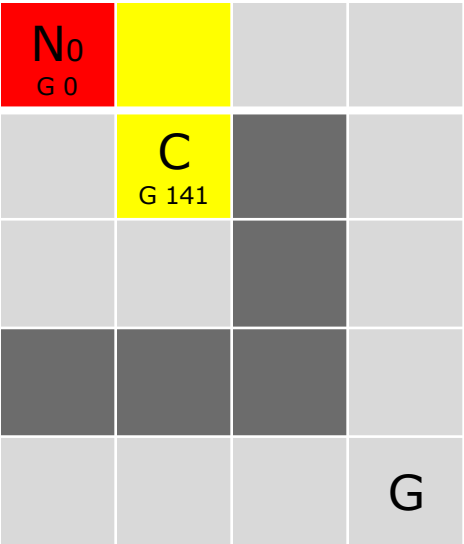
```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

[illegible]

# Exemplu A\* folosind GRAPH-SEARCH



## Open List (Noduri)

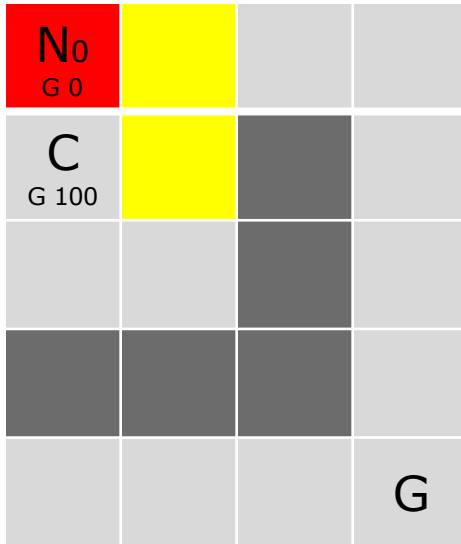
| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0 | 0 |
| 2  | 1,1 | 141 | 382 | 523 | 1,1 | 0 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
|       |
|       |
|       |
|       |
|       |
|       |
|       |
|       |
|       |

```
1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)
```

# Exemplu A\* folosind GRAPH-SEARCH



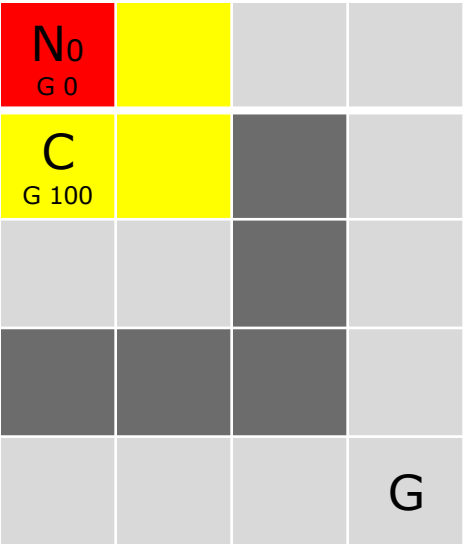
```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

```

[illegible]

# Exemplu A\* folosind GRAPH-SEARCH



## Open List (Noduri)

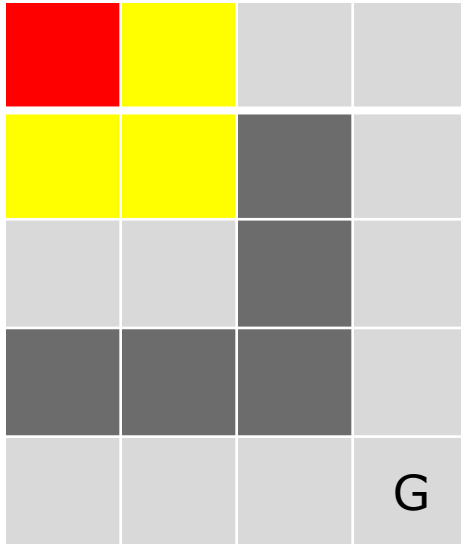
| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0 | 0 |
| 2  | 1,1 | 141 | 382 | 523 | 1,1 | 0 |
| 3  | 0,1 | 100 | 423 | 523 | 0,1 | 0 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
|       |
|       |
|       |
|       |
|       |
|       |
|       |
|       |
|       |

```
1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)
```

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

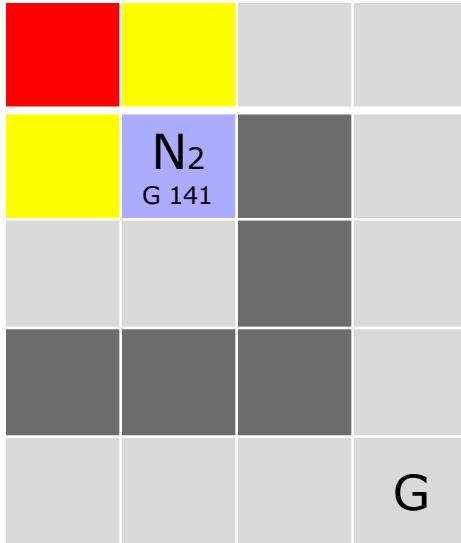
```

## Open List (Noduri)

[illegible]

Closed

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

## Open List (Noduri)

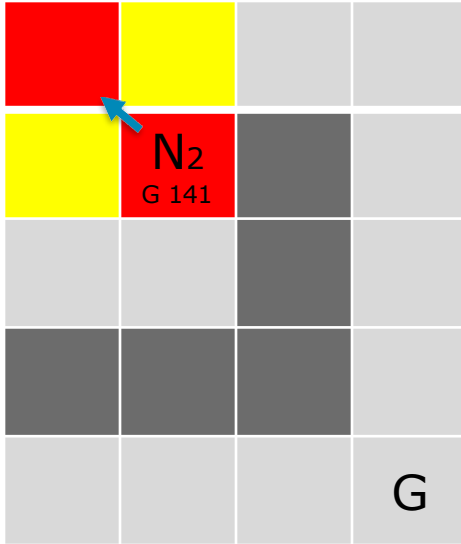
| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0 | 0 |
| 2  | 1,1 | 141 | 382 | 523 | 1,1 | 0 |
| 3  | 0,1 | 100 | 423 | 523 | 0,1 | 0 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

### Stare

0, 0

# Exemplu A\* folosind GRAPH-SEARCH



```

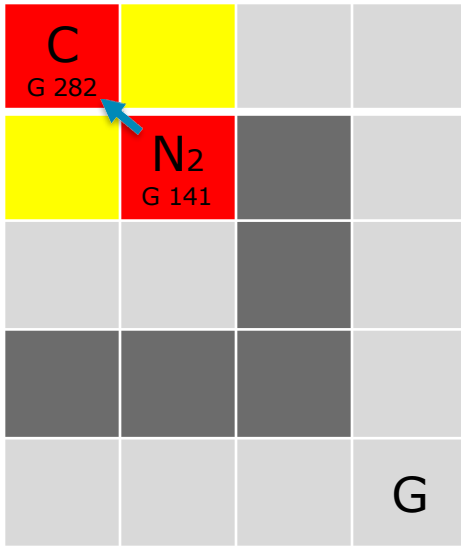
1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

```

[illegible]



# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

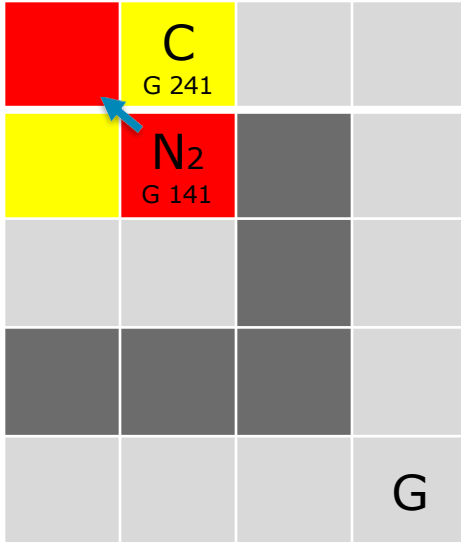
```

## Open List (Noduri)

[illegible]

Closed

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

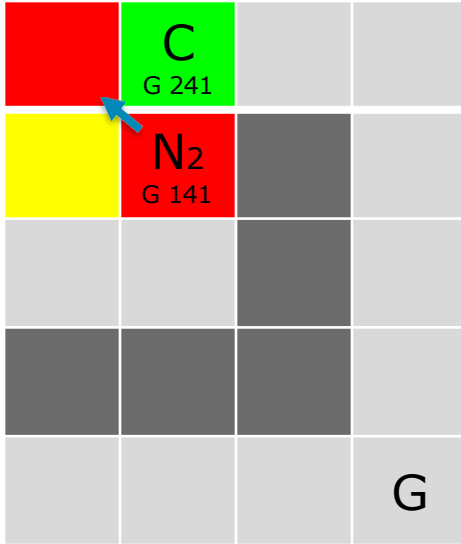
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P | Stare |
|----|-----|-----|-----|-----|-----|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0 | 0 | 0, 0  |
| 3  | 0,1 | 100 | 423 | 523 | 0,1 | 0 | 1, 1  |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

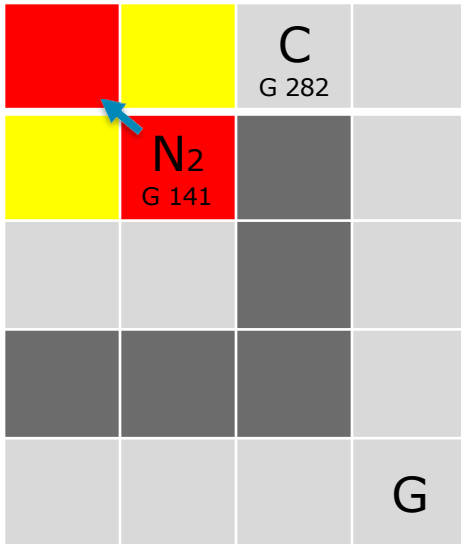
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P | Stare |
|----|-----|-----|-----|-----|-----|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0 | 0 | 0, 0  |
| 3  | 0,1 | 100 | 423 | 523 | 0,1 | 0 | 1, 1  |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

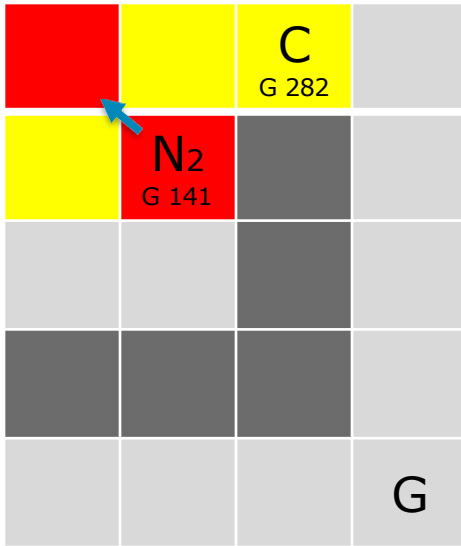
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P | Stare |
|----|-----|-----|-----|-----|-----|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0 | 0 | 0, 0  |
| 3  | 0,1 | 100 | 423 | 523 | 0,1 | 0 | 1, 1  |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



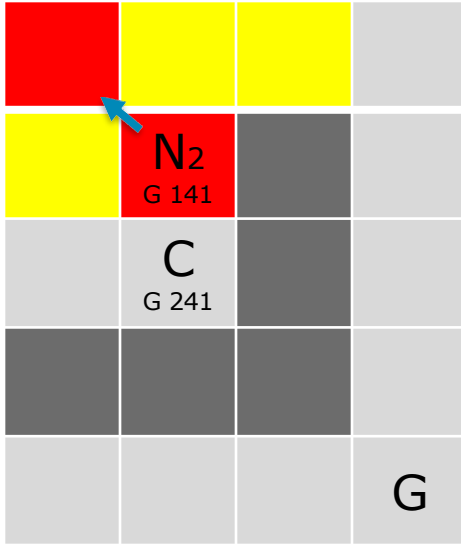
```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

[illegible]

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

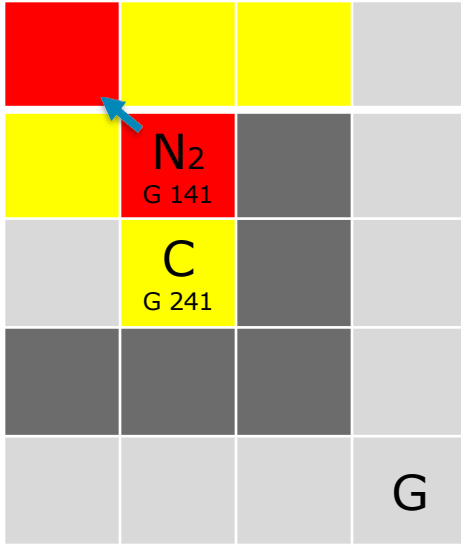
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 3  | 0,1 | 100 | 423 | 523 | 0,1  | 0 | 1, 1  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



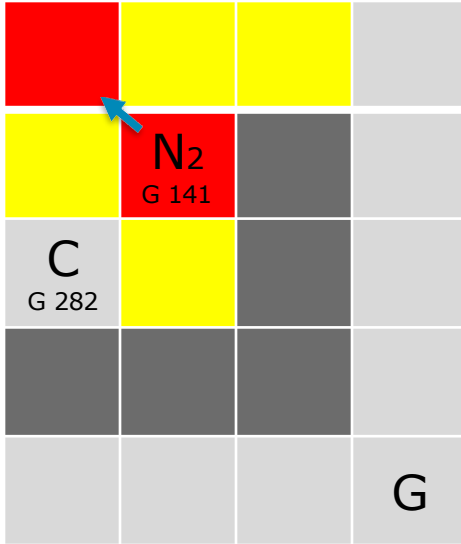
1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 3  | 0,1 | 100 | 423 | 523 | 0,1  | 0 | 1, 1  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |       |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

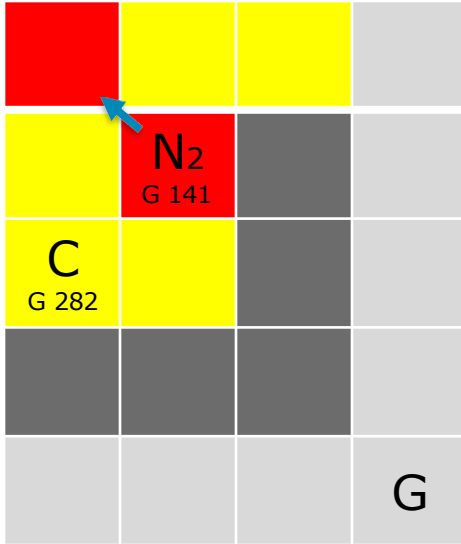
## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 3  | 0,1 | 100 | 423 | 523 | 0,1  | 0 | 1, 1  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |       |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed



# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node ← remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

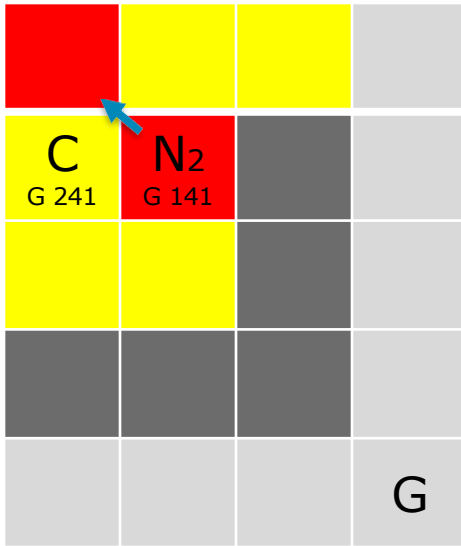
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 3  | 0,1 | 100 | 423 | 523 | 0,1  | 0 | 1, 1  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |       |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 |       |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

Closed

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node ← remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

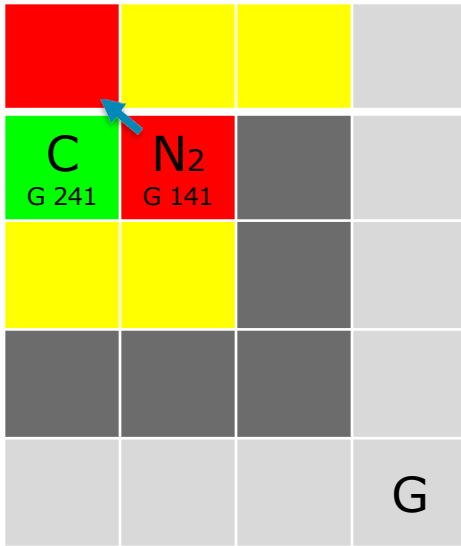
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 3  | 0,1 | 100 | 423 | 523 | 0,1  | 0 | 1, 1  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |       |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 |       |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

Closed

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node ← remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

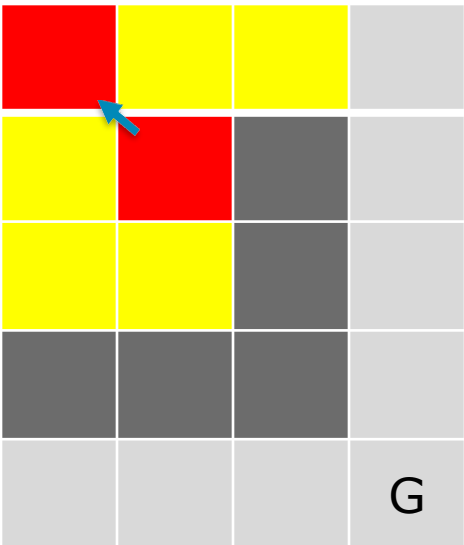
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 3  | 0,1 | 100 | 423 | 523 | 0,1  | 0 | 1, 1  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |       |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 |       |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

Closed

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P |
|----|-----|-----|-----|-----|------|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 |
| 3  | 0,1 | 100 | 423 | 523 | 0,1  | 0 |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |

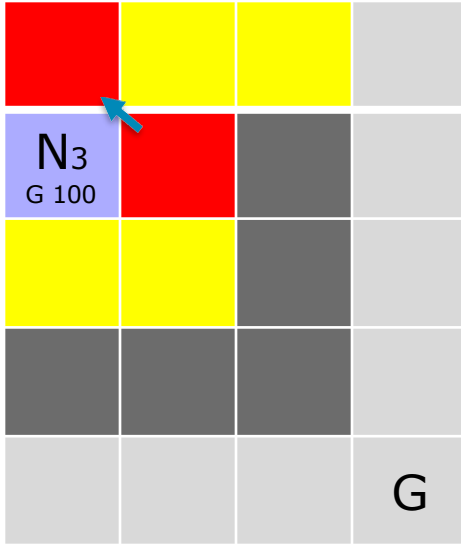
## Closed

### Stare

0, 0

1, 1

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P |
|----|-----|-----|-----|-----|------|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 |
| 3  | 0,1 | 100 | 423 | 523 | 0,1  | 0 |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |

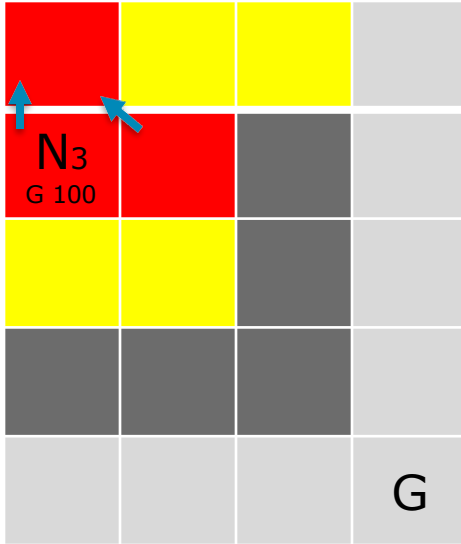
## Closed

### Stare

0, 0

1, 1

# Exemplu A\* folosind GRAPH-SEARCH



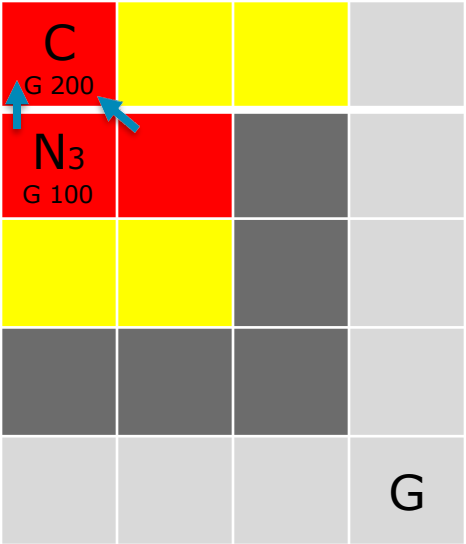
```
1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 1, 1  |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 | 0, 1  |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



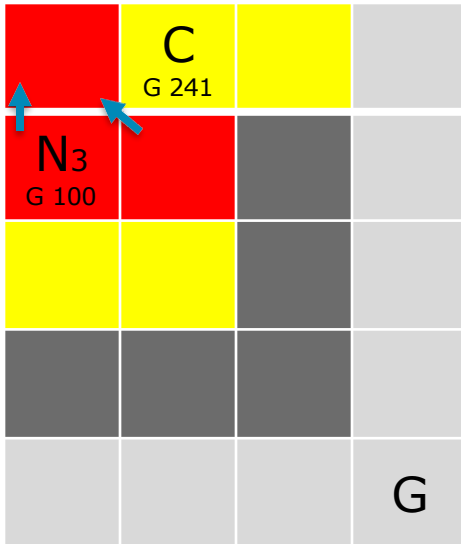
1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 1, 1  |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 | 0, 1  |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

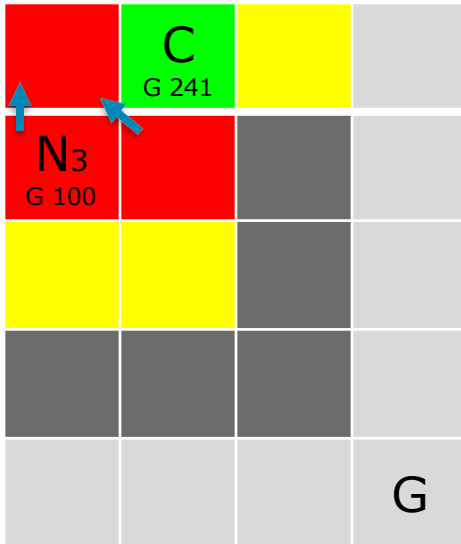
## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 1, 1  |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 | 0, 1  |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed



# Exemplu A\* folosind GRAPH-SEARCH



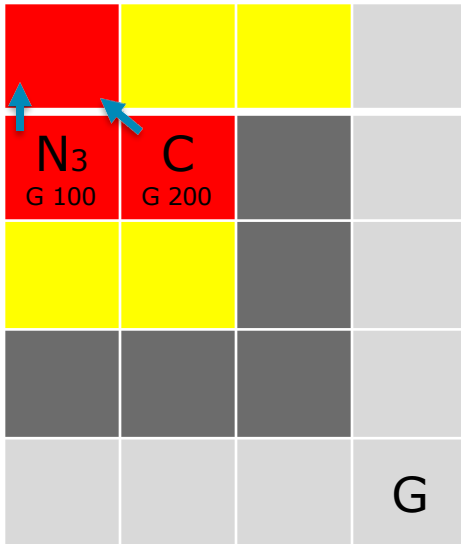
1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 1, 1  |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 | 0, 1  |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



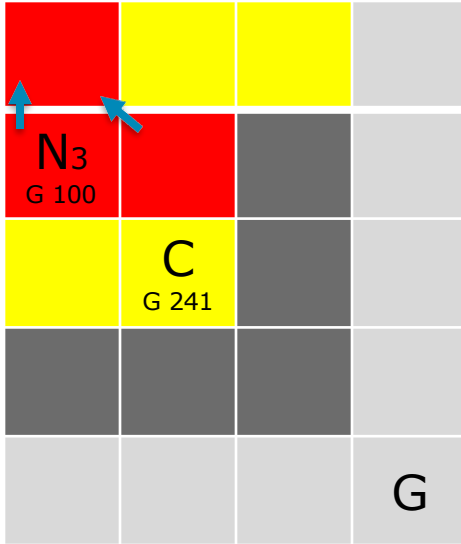
1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 1, 1  |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 | 0, 1  |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



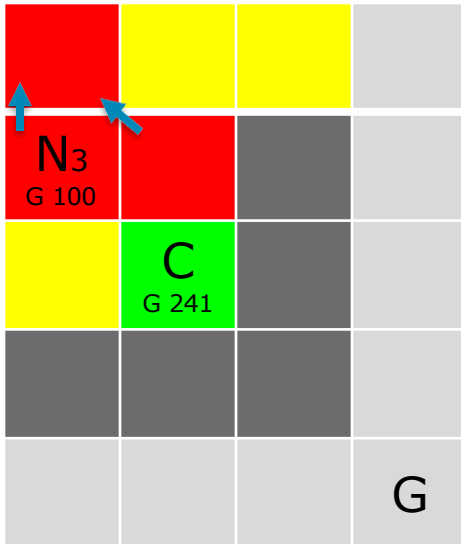
1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 1, 1  |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 | 0, 1  |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



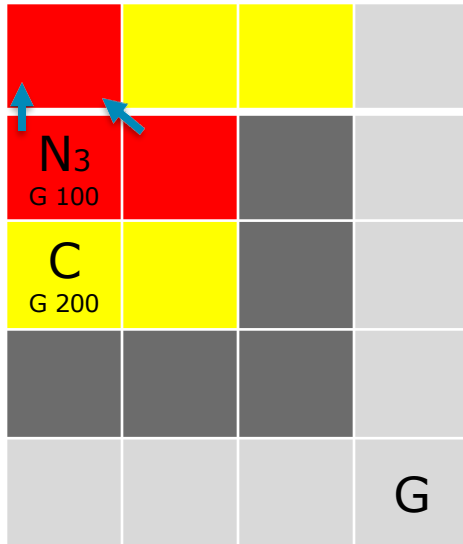
## Open List (Noduri)

## Closed

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 1, 1  |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 | 0, 1  |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

# Exemplu A\* folosind GRAPH-SEARCH



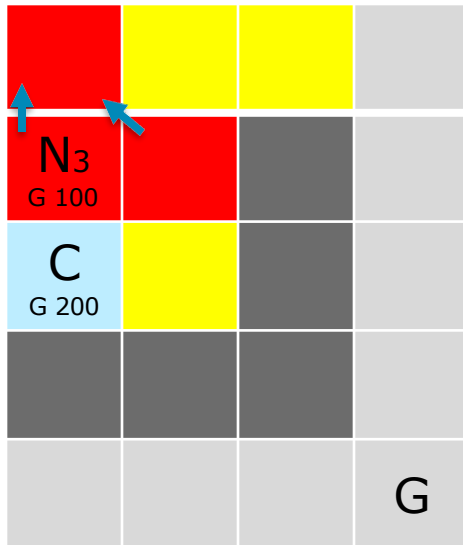
1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 1, 1  |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 | 0, 1  |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |       |
| 7  | 0,2 | 200 | 382 | 582 | 0,1  | 3 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

## Closed

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P |
|----|-----|-----|-----|-----|------|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |
| 7  | 0,2 | 200 | 382 | 582 | 0,1  | 3 |

## Closed

### Stare

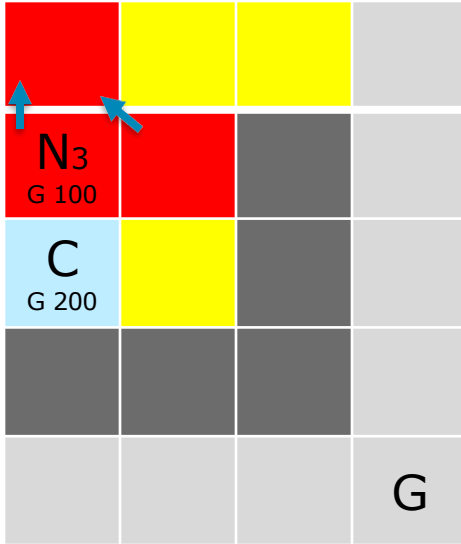
0, 0

1, 1

0, 1

**Important:**  
Nodul 6 nu va fi expandat  
Șterge (implementare eficientă)

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P |
|----|-----|-----|-----|-----|------|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 |
| 6  | 0,2 | 282 | 382 | 664 | -1,1 | 2 |
| 7  | 0,2 | 200 | 382 | 582 | 0,1  | 3 |

## Closed

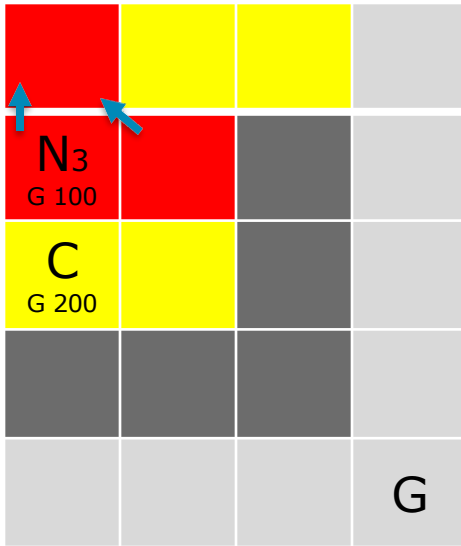
### Stare

0, 0

1, 1

0, 1

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

```

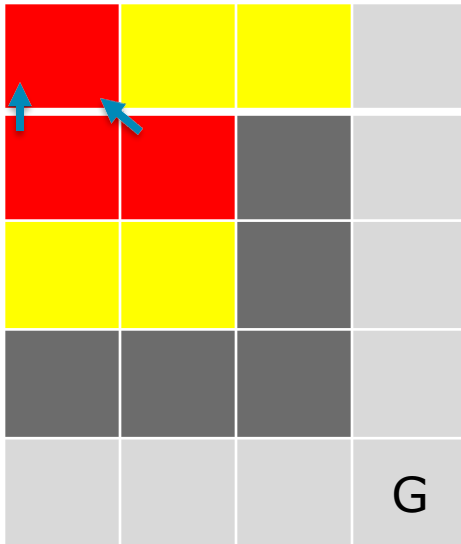
## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 | 0, 0  |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 1, 1  |
| 5  | 1,2 | 241 | 282 | 523 | 0,1  | 2 | 0, 1  |
| 7  | 0,2 | 200 | 382 | 582 | 0,1  | 3 |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

Closed



# Exemplu A\* folosind GRAPH-SEARCH



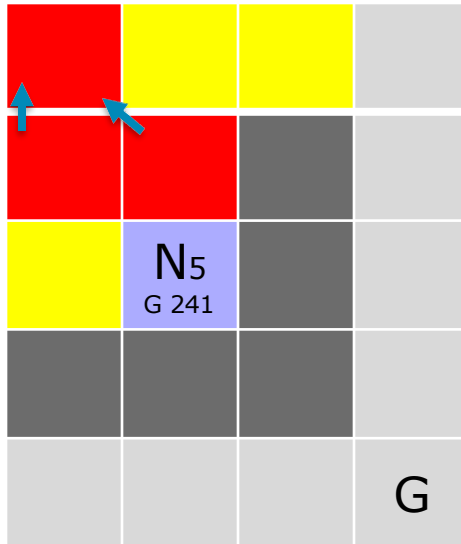
```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

```

[illegible]

## Exemplu A\* folosind GRAPH-SEARCH



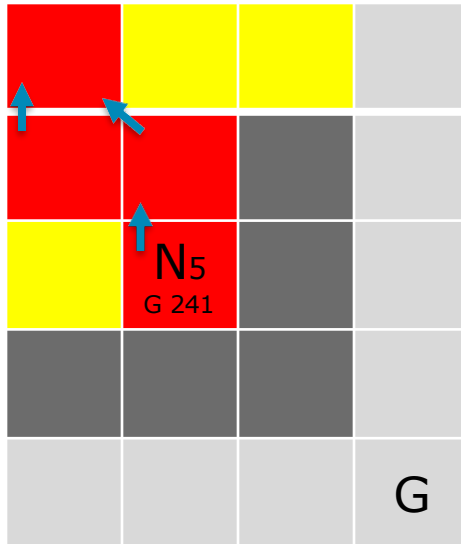
```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

```

[illegible]

# Exemplu A\* folosind GRAPH-SEARCH



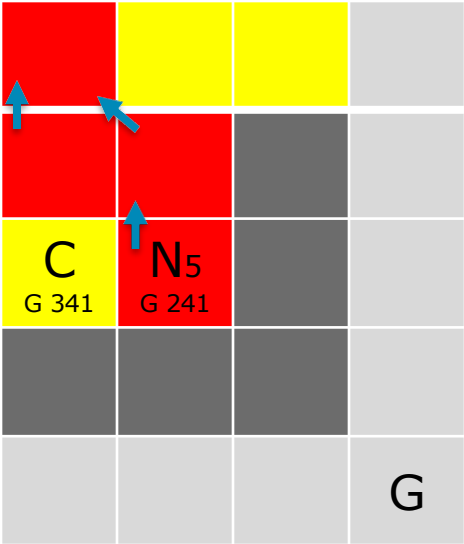
```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

```

[illegible]

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

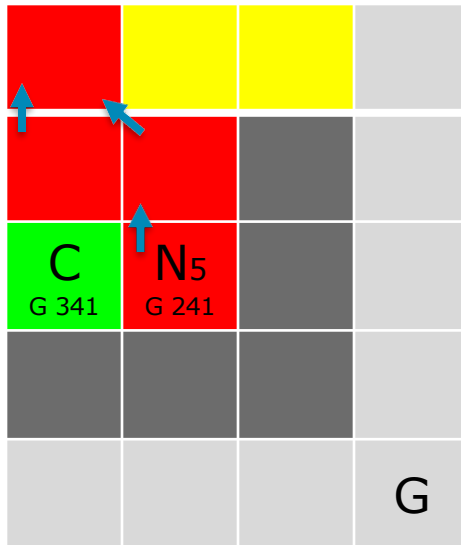
## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P |
|----|-----|-----|-----|-----|------|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |
| 7  | 0,2 | 200 | 382 | 582 | 0,1  | 3 |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
|       |
|       |
|       |
|       |
|       |
|       |

## Exemplu A\* folosind GRAPH-SEARCH



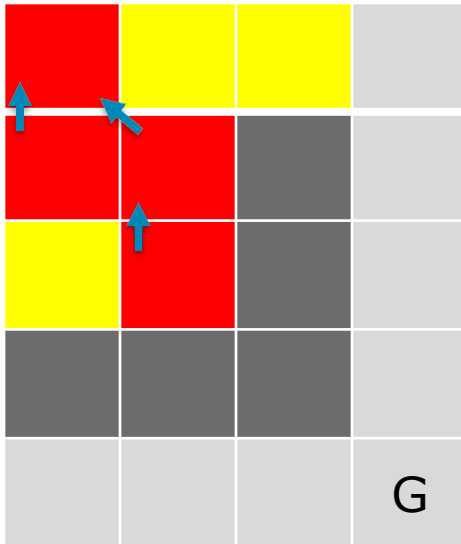
```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

[illegible]

## Exemplu A\* folosind GRAPH-SEARCH



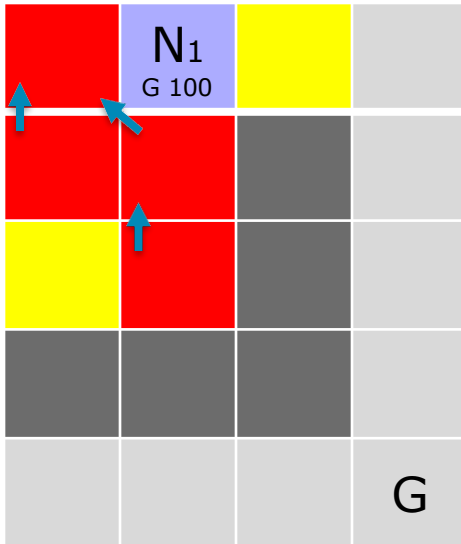
```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

[illegible]

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.      $node = \text{remove min } f \text{ from open}$
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

## Open List (Noduri)

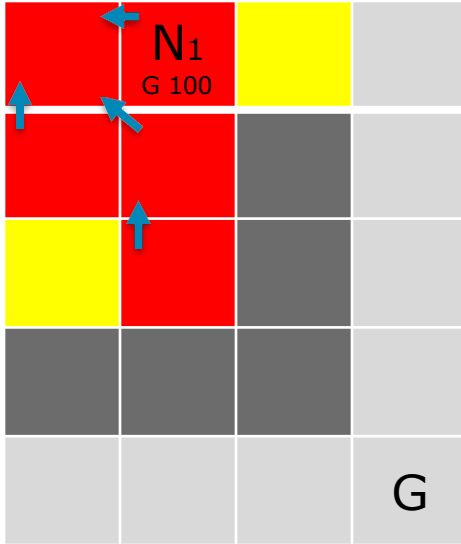
| ID | S   | G   | H   | F   | A    | P |
|----|-----|-----|-----|-----|------|---|
| 1  | 1,0 | 100 | 482 | 582 | 1,0  | 0 |
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |
| 7  | 0,2 | 200 | 382 | 582 | 0,1  | 3 |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |

## Closed

### Stare

0, 0  
1, 1  
0, 1  
1, 2

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

```

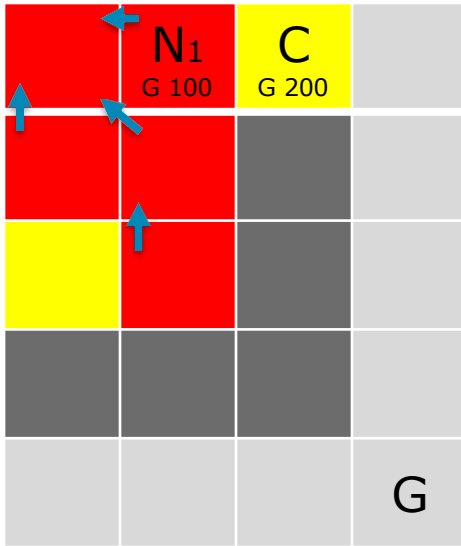
## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P | Stare |
|----|-----|-----|-----|-----|------|---|-------|
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 0, 0  |
| 7  | 0,2 | 200 | 382 | 582 | 0,1  | 3 | 1, 1  |
|    |     |     |     |     |      |   | 0, 1  |
|    |     |     |     |     |      |   | 1, 2  |
|    |     |     |     |     |      |   | 1, 0  |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |
|    |     |     |     |     |      |   |       |

Closed



# Exemplu A\* folosind GRAPH-SEARCH



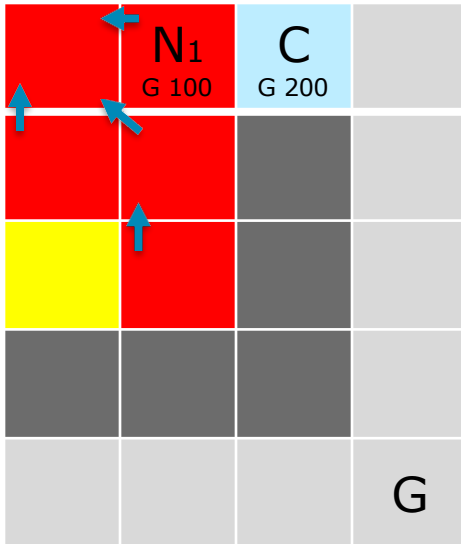
```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

| Open List (Noduri) |     |     |     |     |      |   | Closed |
|--------------------|-----|-----|-----|-----|------|---|--------|
| ID                 | S   | G   | H   | F   | A    | P | Stare  |
| 4                  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 | 0, 0   |
| 7                  | 0,2 | 200 | 382 | 582 | 0,1  | 3 | 1, 1   |
|                    |     |     |     |     |      |   | 0, 1   |
|                    |     |     |     |     |      |   | 1, 2   |
|                    |     |     |     |     |      |   | 1, 0   |
|                    |     |     |     |     |      |   |        |
|                    |     |     |     |     |      |   |        |
|                    |     |     |     |     |      |   |        |
|                    |     |     |     |     |      |   |        |
|                    |     |     |     |     |      |   |        |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

## Open List (Noduri)

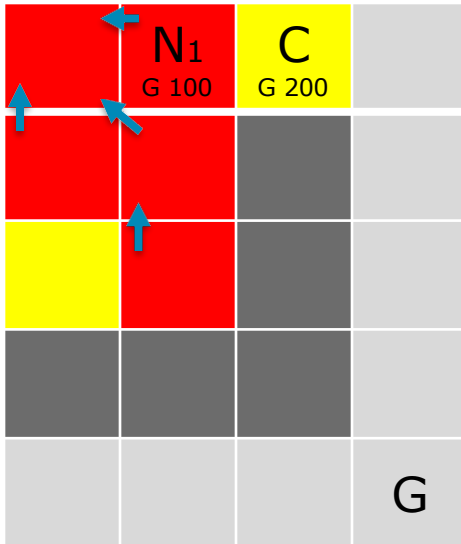
| ID | S   | G   | H   | F   | A    | P |
|----|-----|-----|-----|-----|------|---|
| 4  | 2,0 | 282 | 441 | 723 | 1,-1 | 2 |
| 7  | 0,2 | 200 | 382 | 582 | 0,1  | 3 |
| 8  | 2,0 | 200 | 441 | 641 | 1,0  | 1 |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |
|    |     |     |     |     |      |   |

## Closed

### Stare

0, 0  
1, 1  
0, 1  
1, 2  
1, 0

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

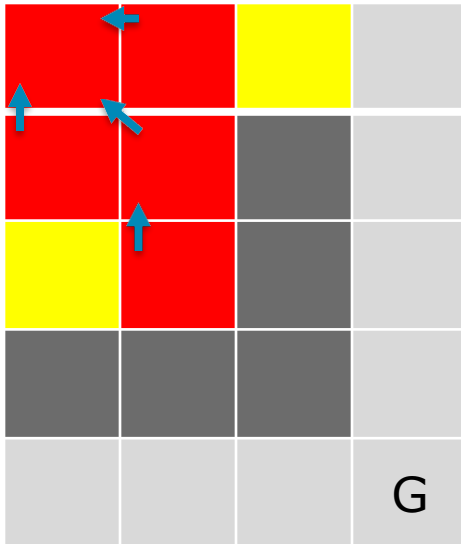
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 8  | 2,0 | 200 | 441 | 641 | 1,0 | 1 |
| 7  | 0,2 | 200 | 382 | 582 | 0,1 | 3 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
|       |
|       |
|       |
|       |
|       |

## Exemplu A\* folosind GRAPH-SEARCH



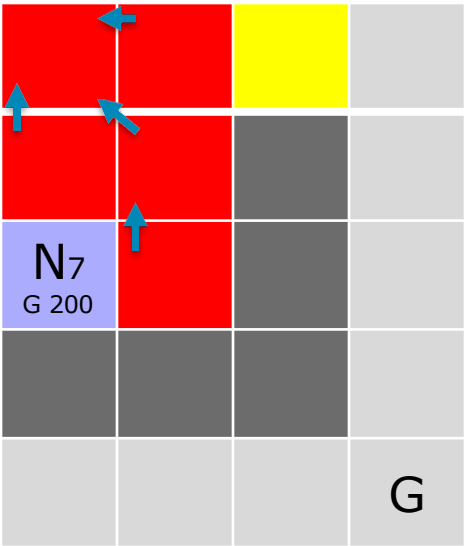
```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

```

| Open List (Noduri) |     |     |     |     |     |   | Closed |
|--------------------|-----|-----|-----|-----|-----|---|--------|
| ID                 | S   | G   | H   | F   | A   | P | Stare  |
| 8                  | 2,0 | 200 | 441 | 641 | 1,0 | 1 | 0, 0   |
| 7                  | 0,2 | 200 | 382 | 582 | 0,1 | 3 | 1, 1   |
|                    |     |     |     |     |     |   | 0, 1   |
|                    |     |     |     |     |     |   | 1, 2   |
|                    |     |     |     |     |     |   | 1, 0   |
|                    |     |     |     |     |     |   |        |
|                    |     |     |     |     |     |   |        |
|                    |     |     |     |     |     |   |        |
|                    |     |     |     |     |     |   |        |
|                    |     |     |     |     |     |   |        |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with <= g: continue
9.     else open.add(c)

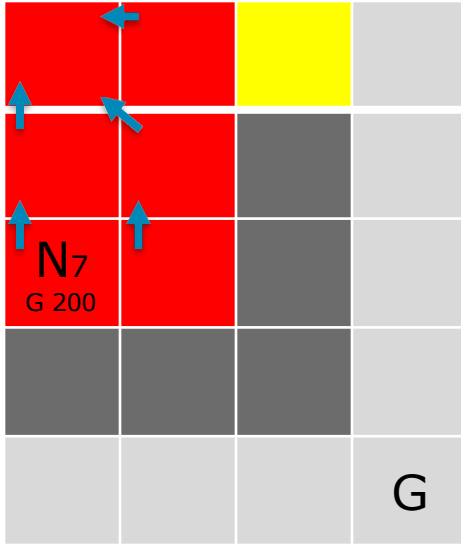
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 8  | 2,0 | 200 | 441 | 641 | 1,0 | 1 |
| 7  | 0,2 | 200 | 382 | 582 | 0,1 | 3 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
|       |
|       |
|       |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

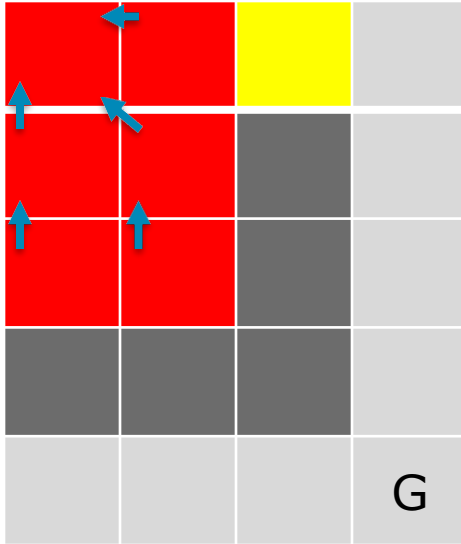
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 8  | 2,0 | 200 | 441 | 641 | 1,0 | 1 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
|       |
|       |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

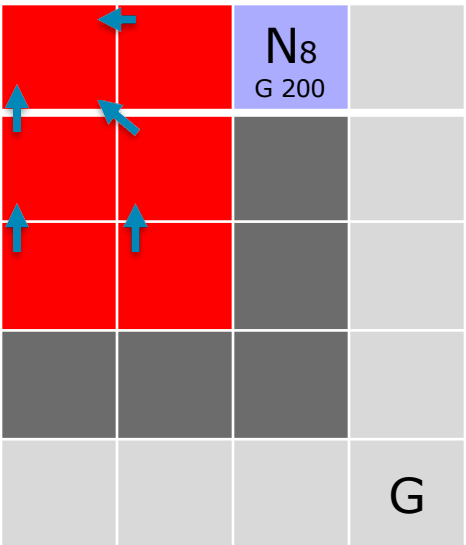
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 8  | 2,0 | 200 | 441 | 641 | 1,0 | 1 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
|       |
|       |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with <= g: continue
9.     else open.add(c)

## Open List (Noduri)

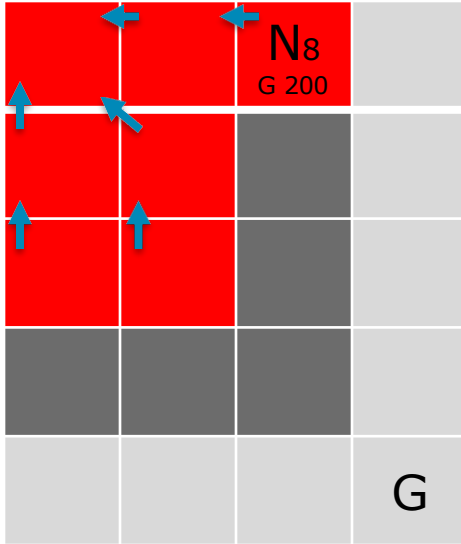
| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 8  | 2,0 | 200 | 441 | 641 | 1,0 | 1 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
|       |
|       |
|       |
|       |



# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

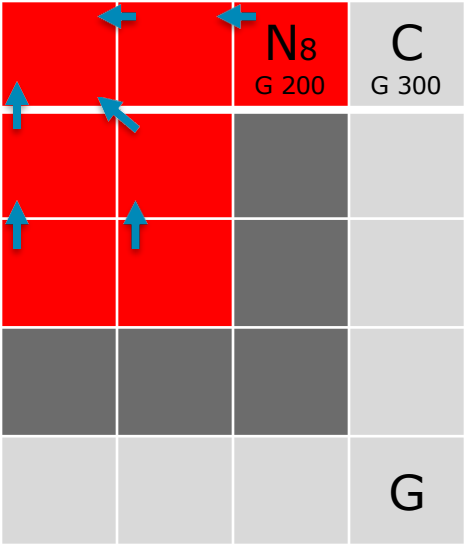
Open List (Noduri)

| ID | S | G | H | F | A | P |
|----|---|---|---|---|---|---|
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |

Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

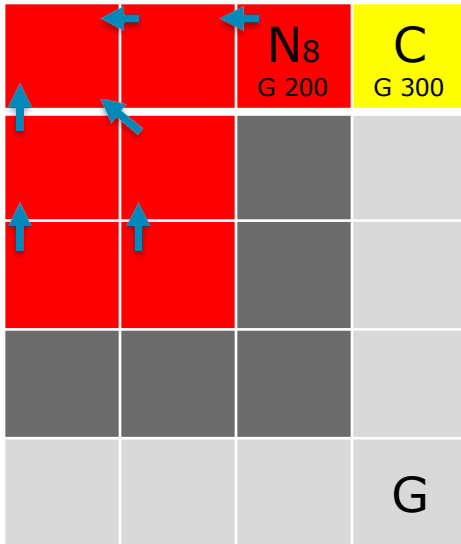
Open List (Noduri)

| ID | S | G | H | F | A | P |
|----|---|---|---|---|---|---|
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |
|    |   |   |   |   |   |   |

Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

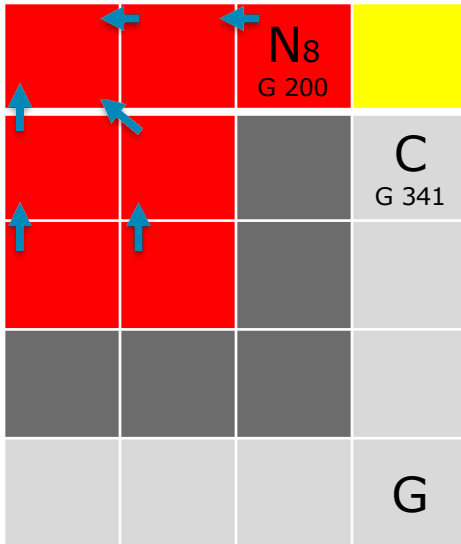
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty())
2.     node = remove min f from open
3.     if (node.state is goal) return path
4.     if (node.state in closed) continue
5.     add node to closed
6.     for c in expand(node)
7.         if c.state in closed: continue
8.         if c in open with  $\leq g$ : continue
9.         else open.add(c)

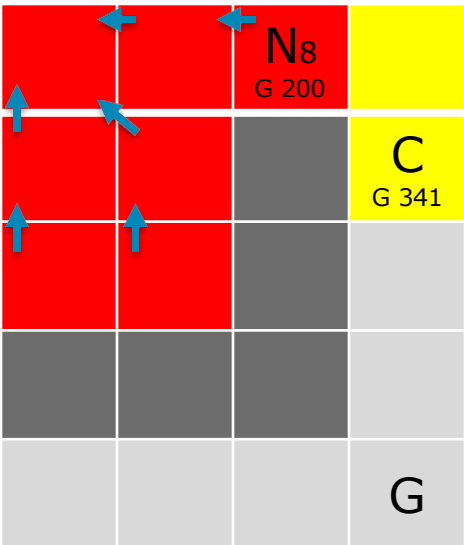
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



- 1. while (!open.empty)
- 2.   node = remove min f from open
- 3.   if (node.state is goal) return path
- 4.   if (node.state in closed) continue
- 5.   add node to closed
- 6.   for c in expand(node)
- 7.     if c.state in closed: continue
- 8.     if c in open with <= g: continue
- 9.     else open.add(c)

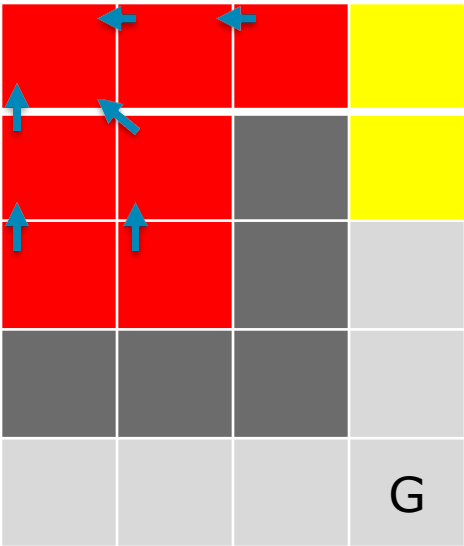
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
| 10 | 3,1 | 341 | 300 | 641 | 1,1 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
|       |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

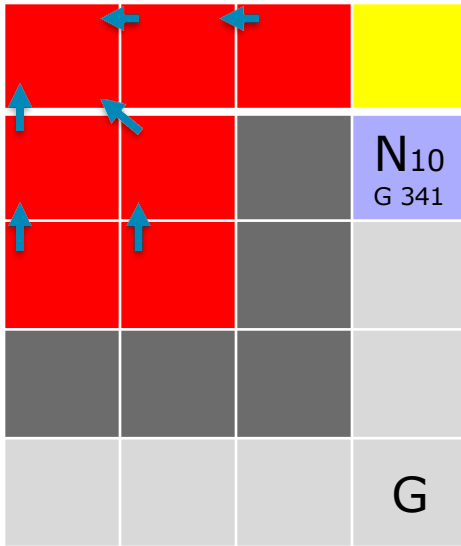
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
| 10 | 3,1 | 341 | 300 | 641 | 1,1 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
|       |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

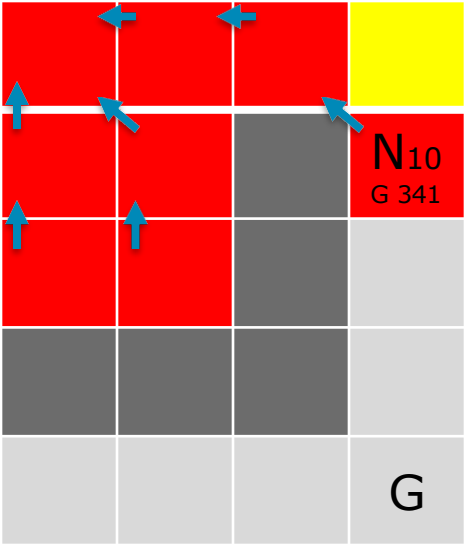
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P | Stare |
|----|-----|-----|-----|-----|-----|---|-------|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 | 0, 0  |
| 10 | 3,1 | 341 | 300 | 641 | 1,1 | 8 | 1, 1  |
|    |     |     |     |     |     |   | 0, 1  |
|    |     |     |     |     |     |   | 1, 2  |
|    |     |     |     |     |     |   | 1, 0  |
|    |     |     |     |     |     |   | 0, 2  |
|    |     |     |     |     |     |   | 2, 0  |
|    |     |     |     |     |     |   |       |
|    |     |     |     |     |     |   |       |

Closed

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

## Open List (Noduri)

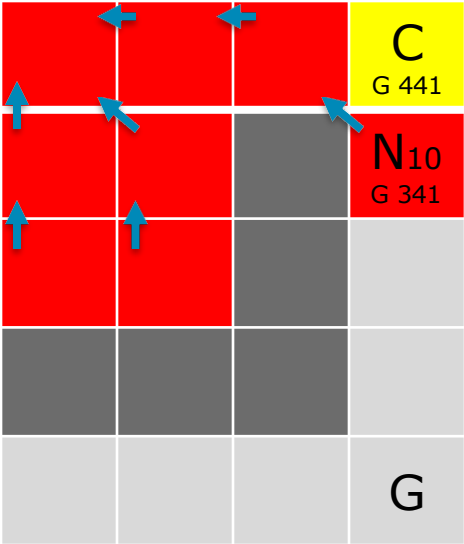
| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
|       |
|       |



# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

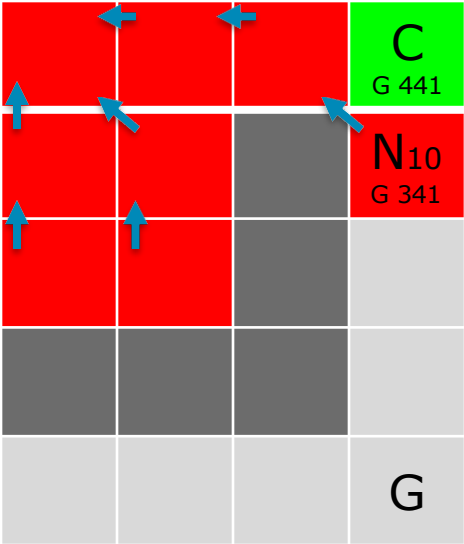
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

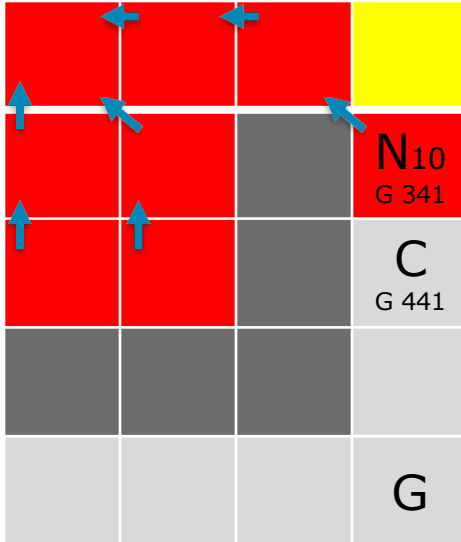
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

## Open List (Noduri)

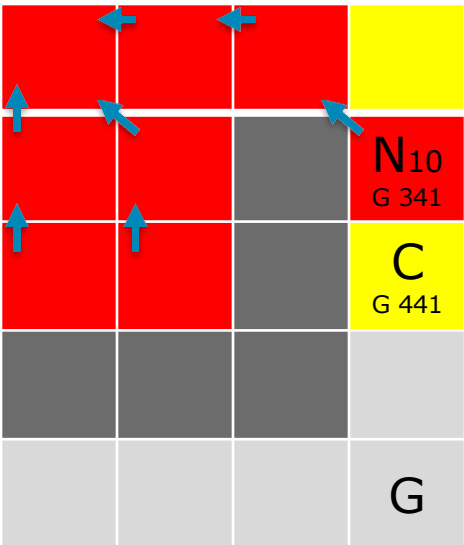
| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

### Stare

0, 0  
1, 1  
0, 1  
1, 2  
1, 0  
0, 2  
2, 0  
3, 1

# Exemplu A\* folosind GRAPH-SEARCH



```
1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)
```

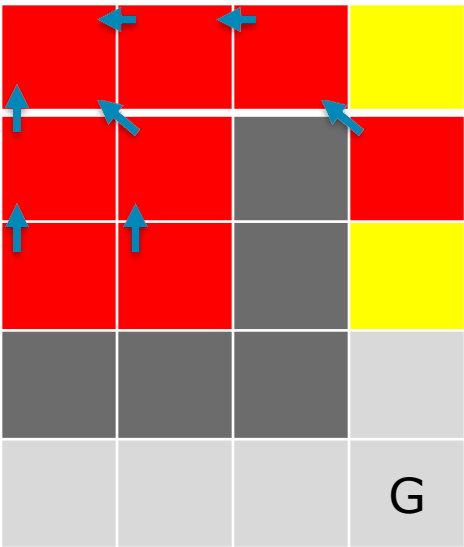
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P  |
|----|-----|-----|-----|-----|-----|----|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8  |
| 11 | 3,2 | 441 | 200 | 641 | 0,1 | 10 |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



```
1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)
```

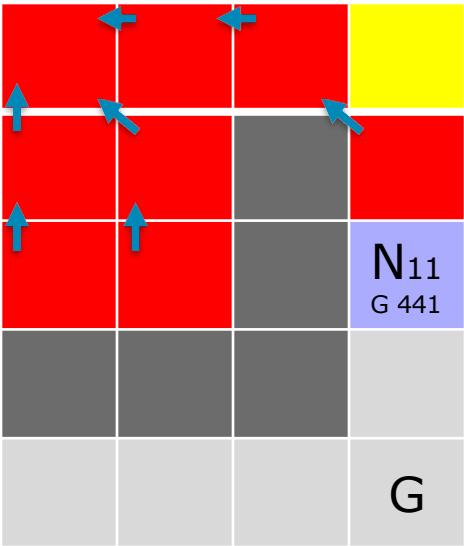
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P  |
|----|-----|-----|-----|-----|-----|----|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8  |
| 11 | 3,2 | 441 | 200 | 641 | 0,1 | 10 |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

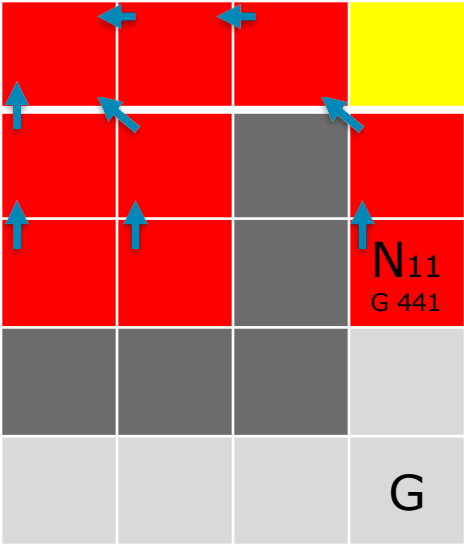
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P  |
|----|-----|-----|-----|-----|-----|----|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8  |
| 11 | 3,2 | 441 | 200 | 641 | 0,1 | 10 |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
|       |
|       |

# Exemplu A\* folosind GRAPH-SEARCH



- 1. while (!open.empty)
- 2.   node = remove min f from open
- 3.   if (node.state is goal) return path
- 4.   if (node.state in closed) continue
- 5.   add node to closed
- 6.   for c in expand(node)
- 7.     if c.state in closed: continue
- 8.     if c in open with <= g: continue
- 9.     else open.add(c)

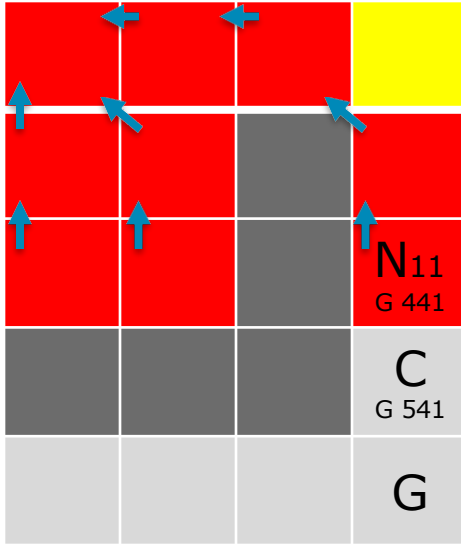
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
| 3, 2  |

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with $\leq g$: continue
9. else open.add(c)

```

## Open List (Noduri)

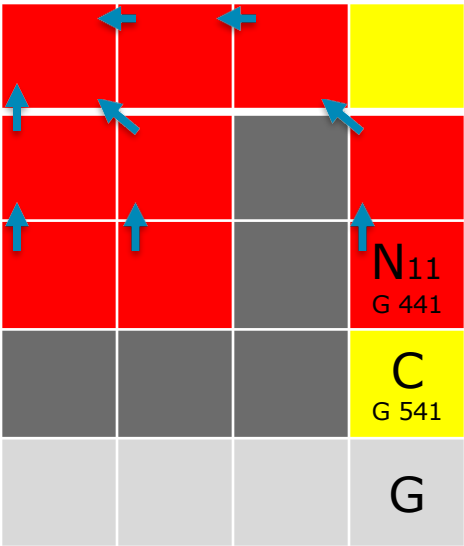
[illegible]

Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
| 3, 2  |



# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with <= g: continue
9.     else open.add(c)

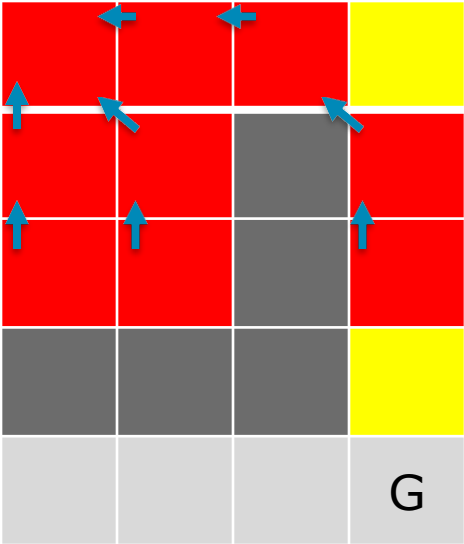
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P  |
|----|-----|-----|-----|-----|-----|----|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8  |
| 12 | 3,3 | 541 | 100 | 641 | 0,1 | 11 |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
| 3, 2  |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq g$ : continue
9.     else open.add(c)

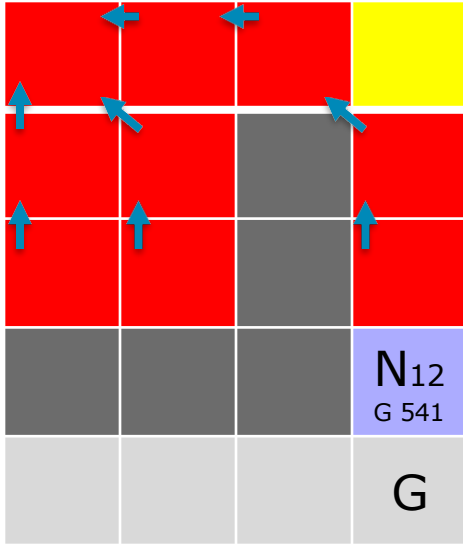
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P  |
|----|-----|-----|-----|-----|-----|----|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8  |
| 12 | 3,3 | 541 | 100 | 641 | 0,1 | 11 |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
| 3, 2  |

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

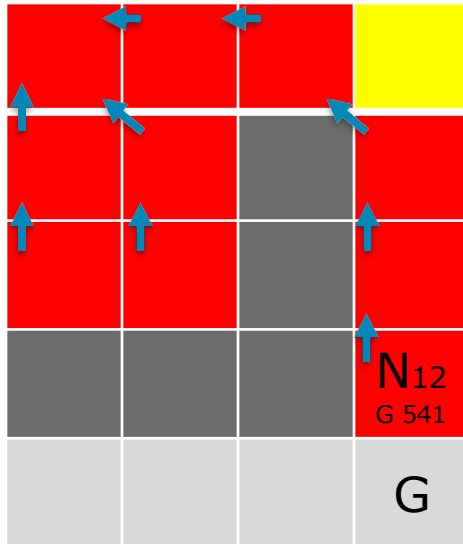
```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P  | Stare |
|----|-----|-----|-----|-----|-----|----|-------|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8  | 0, 0  |
| 12 | 3,3 | 541 | 100 | 641 | 0,1 | 11 | 1, 1  |
|    |     |     |     |     |     |    | 0, 1  |
|    |     |     |     |     |     |    | 1, 2  |
|    |     |     |     |     |     |    | 1, 0  |
|    |     |     |     |     |     |    | 0, 2  |
|    |     |     |     |     |     |    | 2, 0  |
|    |     |     |     |     |     |    | 3, 1  |
|    |     |     |     |     |     |    | 3, 2  |

# Closed

# Exemplu A\* folosind GRAPH-SEARCH



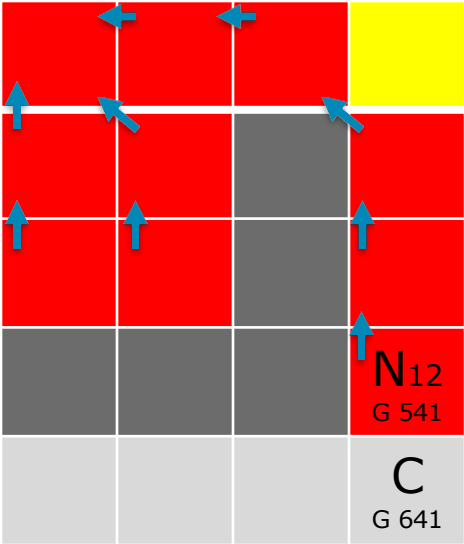
```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

| Open List (Noduri) |     |     |     |     |     |   | Closed |
|--------------------|-----|-----|-----|-----|-----|---|--------|
| ID                 | S   | G   | H   | F   | A   | P | Stare  |
| 9                  | 3,0 | 300 | 400 | 700 | 1,0 | 8 | 0, 0   |
|                    |     |     |     |     |     |   | 1, 1   |
|                    |     |     |     |     |     |   | 0, 1   |
|                    |     |     |     |     |     |   | 1, 2   |
|                    |     |     |     |     |     |   | 1, 0   |
|                    |     |     |     |     |     |   | 0, 2   |
|                    |     |     |     |     |     |   | 2, 0   |
|                    |     |     |     |     |     |   | 3, 1   |
|                    |     |     |     |     |     |   | 3, 2   |
|                    |     |     |     |     |     |   | 3, 3   |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq$  g: continue
9.     else open.add(c)

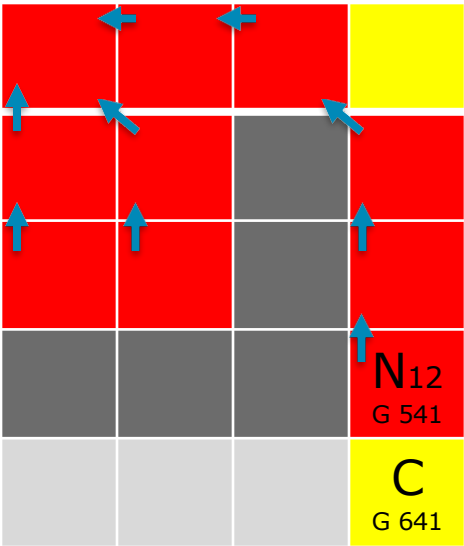
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P |
|----|-----|-----|-----|-----|-----|---|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8 |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |
|    |     |     |     |     |     |   |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
| 3, 2  |
| 3, 3  |

# Exemplu A\* folosind GRAPH-SEARCH



- 1. while (!open.empty)
- 2.   node = remove min f from open
- 3.   if (node.state is goal) return path
- 4.   if (node.state in closed) continue
- 5.   add node to closed
- 6.   for c in expand(node)
- 7.     if c.state in closed: continue
- 8.     if c in open with <= g: continue
- 9.     else open.add(c)

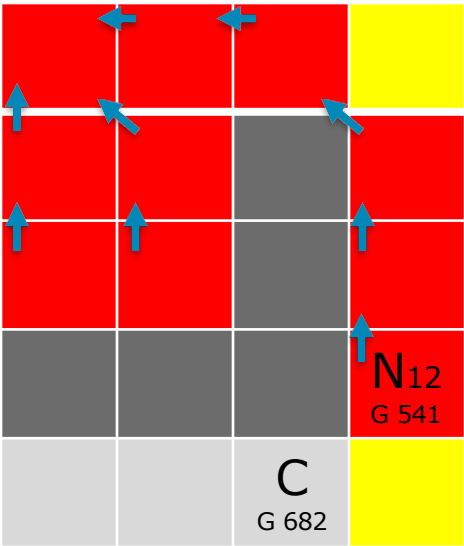
Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P  |
|----|-----|-----|-----|-----|-----|----|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8  |
| 13 | 3,4 | 641 | 0   | 641 | 0,1 | 12 |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |

Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
| 3, 2  |
| 3,3   |

# Exemplu A\* folosind GRAPH-SEARCH



1. while (!open.empty)
2.   node = remove min f from open
3.   if (node.state is goal) return path
4.   if (node.state in closed) continue
5.   add node to closed
6.   for c in expand(node)
7.     if c.state in closed: continue
8.     if c in open with  $\leq$  g: continue
9.     else open.add(c)

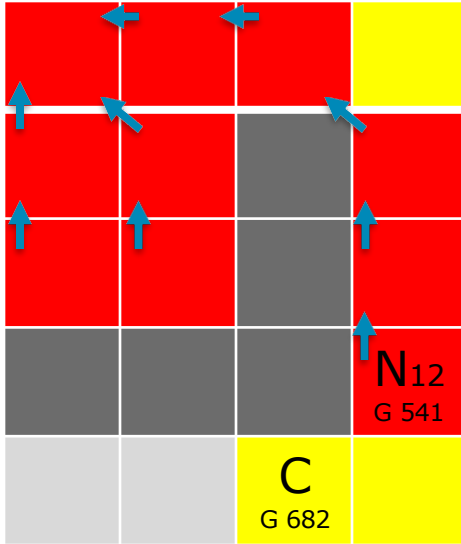
## Open List (Noduri)

| ID | S   | G   | H   | F   | A   | P  |
|----|-----|-----|-----|-----|-----|----|
| 9  | 3,0 | 300 | 400 | 700 | 1,0 | 8  |
| 13 | 3,4 | 641 | 0   | 641 | 0,1 | 12 |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |
|    |     |     |     |     |     |    |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
| 3, 2  |
| 3,3   |

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P  |
|----|-----|-----|-----|-----|------|----|
| 9  | 3,0 | 300 | 400 | 700 | 1,0  | 8  |
| 13 | 3,4 | 641 | 0   | 641 | 0,1  | 12 |
| 14 | 2,4 | 682 | 100 | 682 | -1,1 | 12 |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |

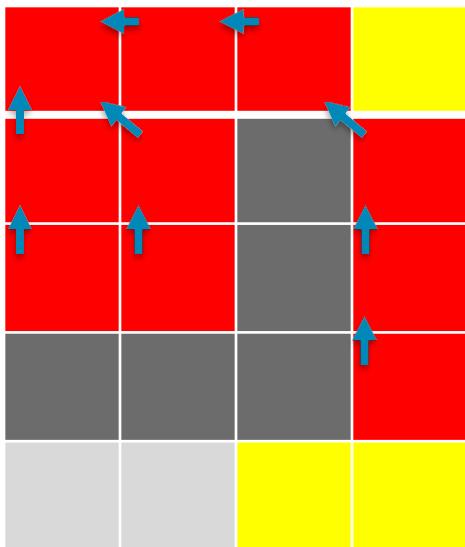
## Closed

### Stare

0, 0  
1, 1  
0, 1  
1, 2  
1, 0  
0, 2  
2, 0  
3, 1  
3, 2  
3, 3



# Exemplu A\* folosind GRAPH-SEARCH



- 1. while (!open.empty)
- 2.   node = remove min f from open
- 3.   if (node.state is goal) return path
- 4.   if (node.state in closed) continue
- 5.   add node to closed
- 6.   for c in expand(node)
- 7.     if c.state in closed: continue
- 8.     if c in open with  $\leq g$ : continue
- 9.     else open.add(c)

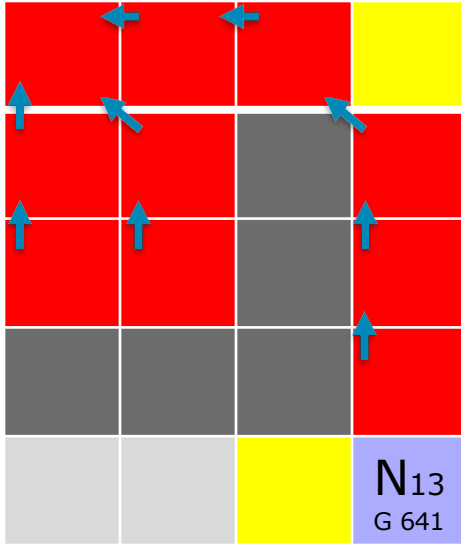
## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P  |
|----|-----|-----|-----|-----|------|----|
| 9  | 3,0 | 300 | 400 | 700 | 1,0  | 8  |
| 13 | 3,4 | 641 | 0   | 641 | 0,1  | 12 |
| 14 | 2,4 | 682 | 100 | 682 | -1,1 | 12 |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |
|    |     |     |     |     |      |    |

## Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
| 3, 2  |
| 3,3   |

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty())
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

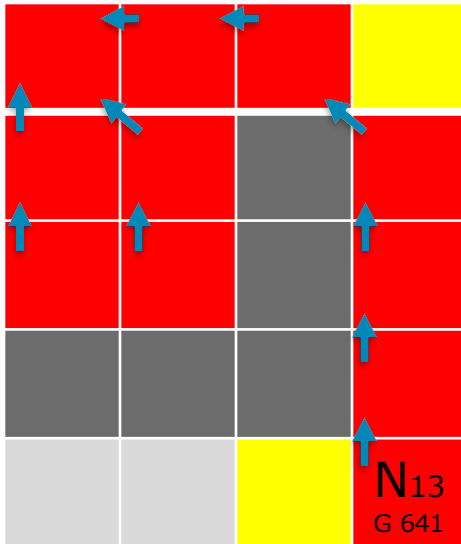
## Open List (Noduri)

[illegible]

# Closed

| Stare |
|-------|
| 0, 0  |
| 1, 1  |
| 0, 1  |
| 1, 2  |
| 1, 0  |
| 0, 2  |
| 2, 0  |
| 3, 1  |
| 3, 2  |
| 3, 3  |

# Exemplu A\* folosind GRAPH-SEARCH



```

1. while (!open.empty)
2. node = remove min f from open
3. if (node.state is goal) return path
4. if (node.state in closed) continue
5. add node to closed
6. for c in expand(node)
7. if c.state in closed: continue
8. if c in open with <= g: continue
9. else open.add(c)

```

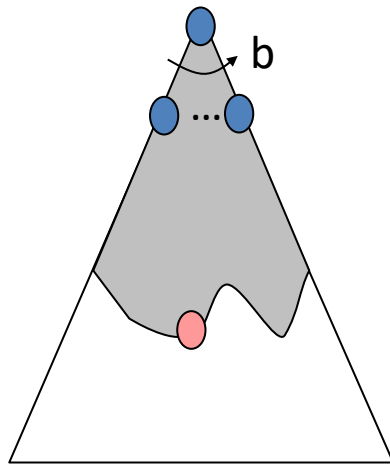
## Open List (Noduri)

| ID | S   | G   | H   | F   | A    | P  | Stare |
|----|-----|-----|-----|-----|------|----|-------|
| 9  | 3,0 | 300 | 400 | 700 | 1,0  | 8  | 0, 0  |
| 14 | 2,4 | 682 | 100 | 682 | -1,1 | 12 | 1, 1  |
|    |     |     |     |     |      |    | 0, 1  |
|    |     |     |     |     |      |    | 1, 2  |
|    |     |     |     |     |      |    | 1, 0  |
|    |     |     |     |     |      |    | 0, 2  |
|    |     |     |     |     |      |    | 2, 0  |
|    |     |     |     |     |      |    | 3, 1  |
|    |     |     |     |     |      |    | 3, 2  |
|    |     |     |     |     |      |    | 3,3   |

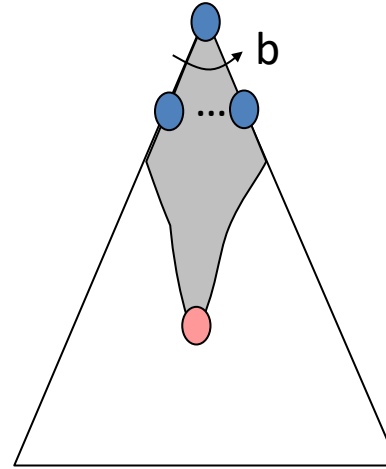
## Closed

Nod scop găsit  
Reconstruiește soluția

# Proprietățile algoritmului $A^*$



UCS

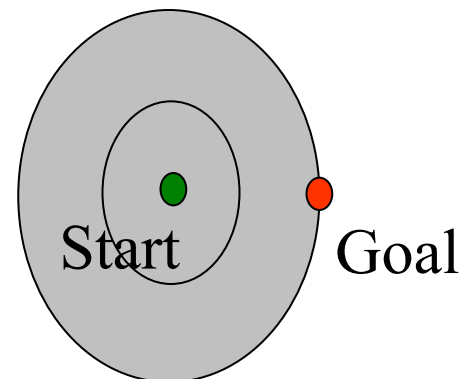


$A^*$

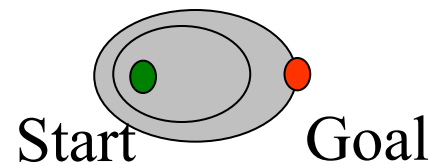
Căutare uniformă pe bază de cost explorează drumuri de cost minim, indiferent de cât de îndepărtate sunt de soluția-scop. Algoritmul  $A^*$  beneficiază de euristica  $h$  care restricționează căutarea.

# UCS vs $A^*$ - explorarea arborelui de căutare

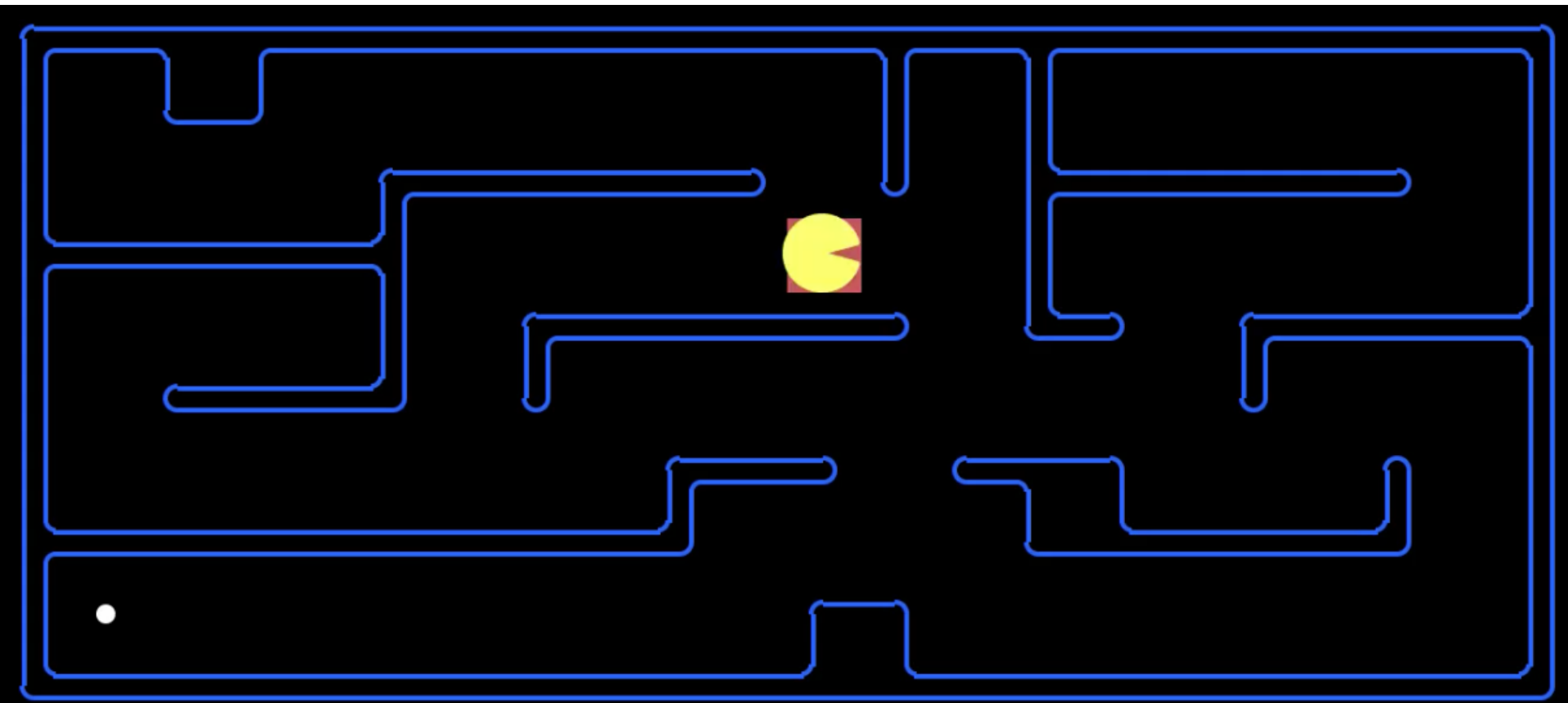
- Căutarea uniformă pe bază de cost (UCS) explorează soluții uniform în toate direcțiile.



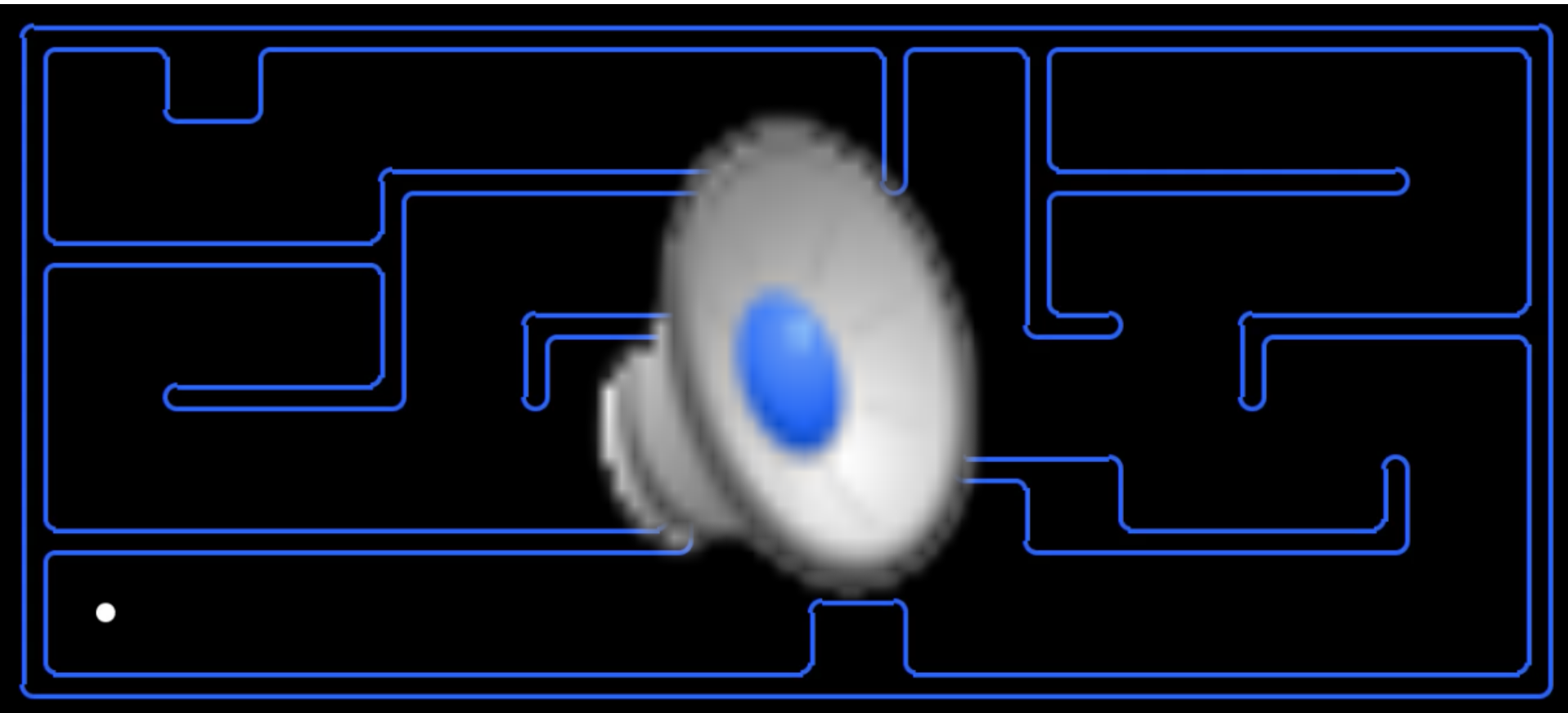
- Algoritmul  $A^*$  explorează soluții în mare parte în direcția soluției, totuși păstrează și soluții în alte direcții (pentru păstrarea optimalității).



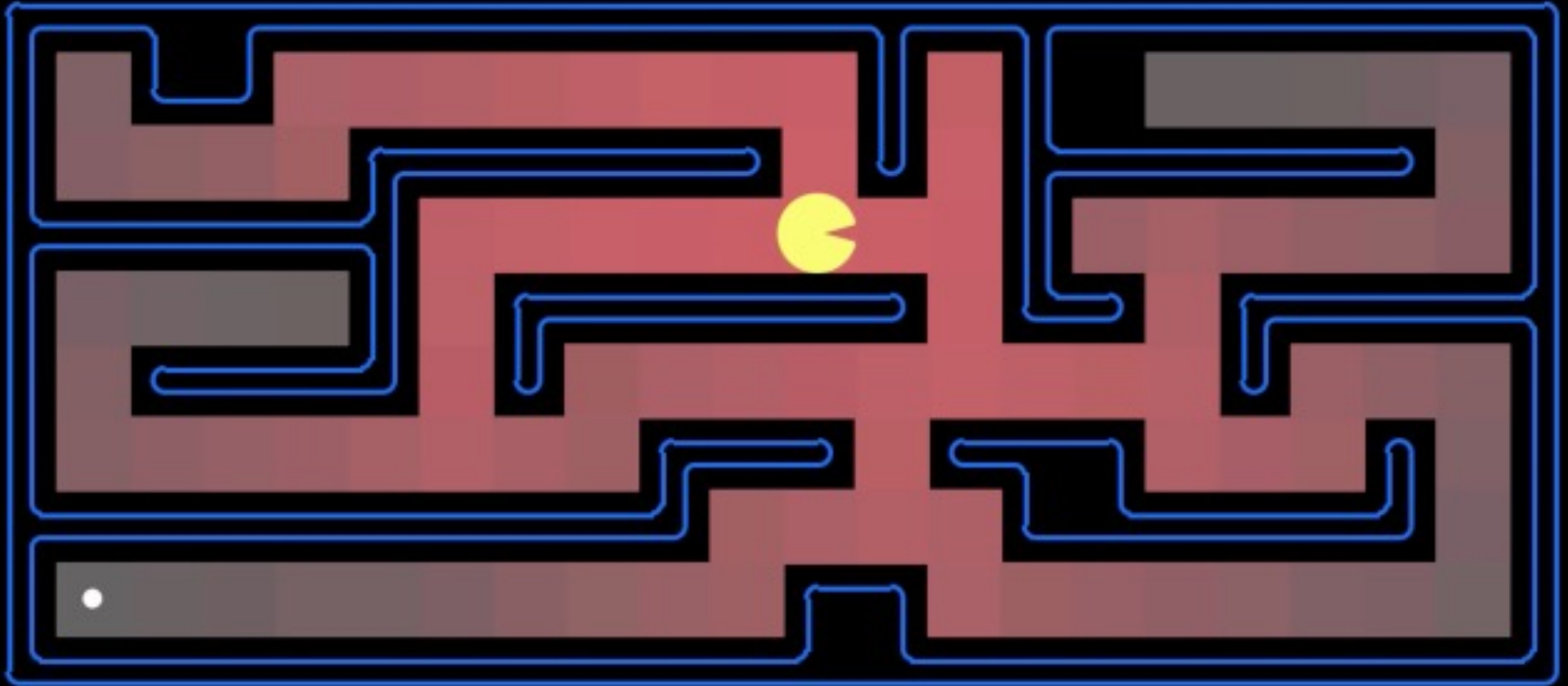
# PAC MAN - demo



# PAC MAN – demo UCS



# PAC MAN– demo UCS



**SCORE: 0**

Ordinea de explorare a nodurile este dată de culoare:

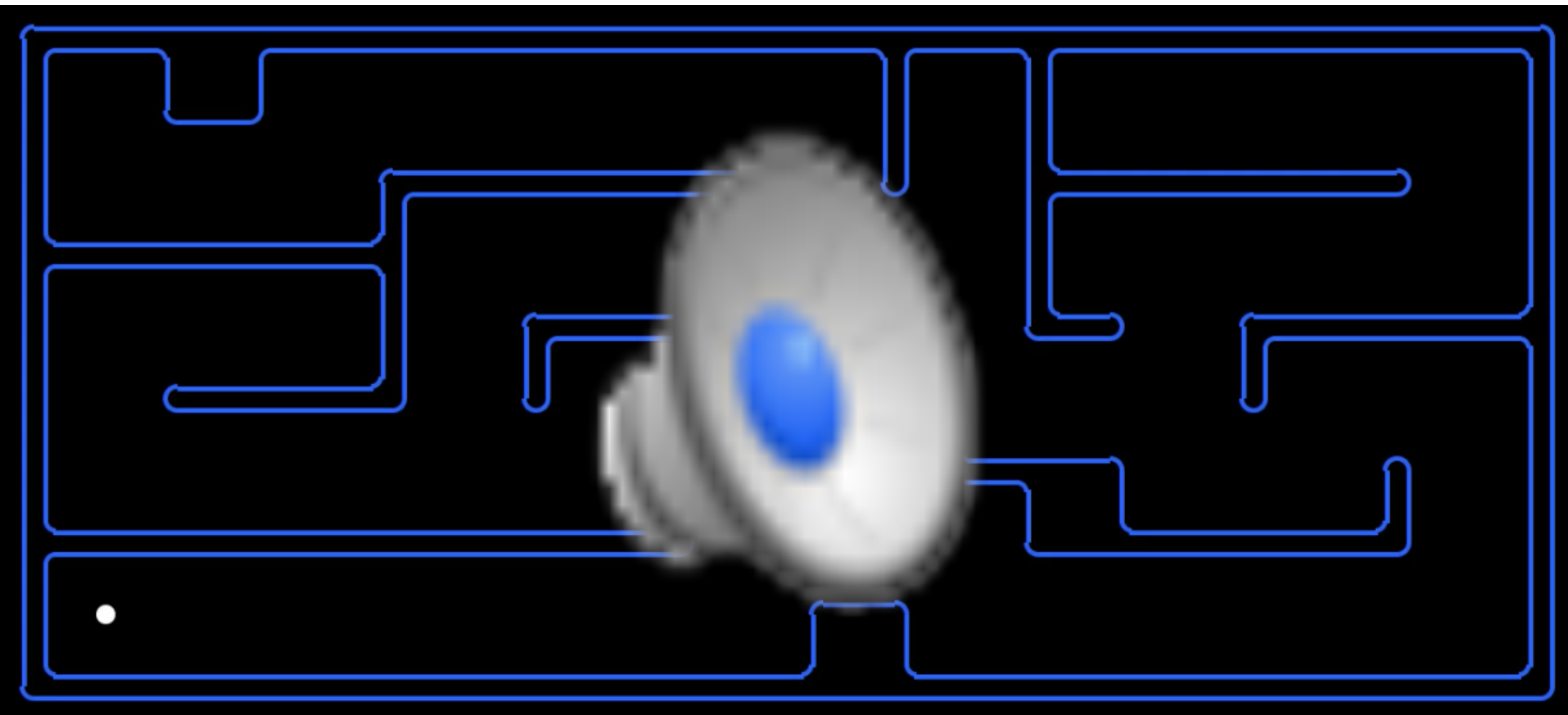
**Roșu intens – noduri explorate la început**

**Gri – noduri explorate la sfârșit**

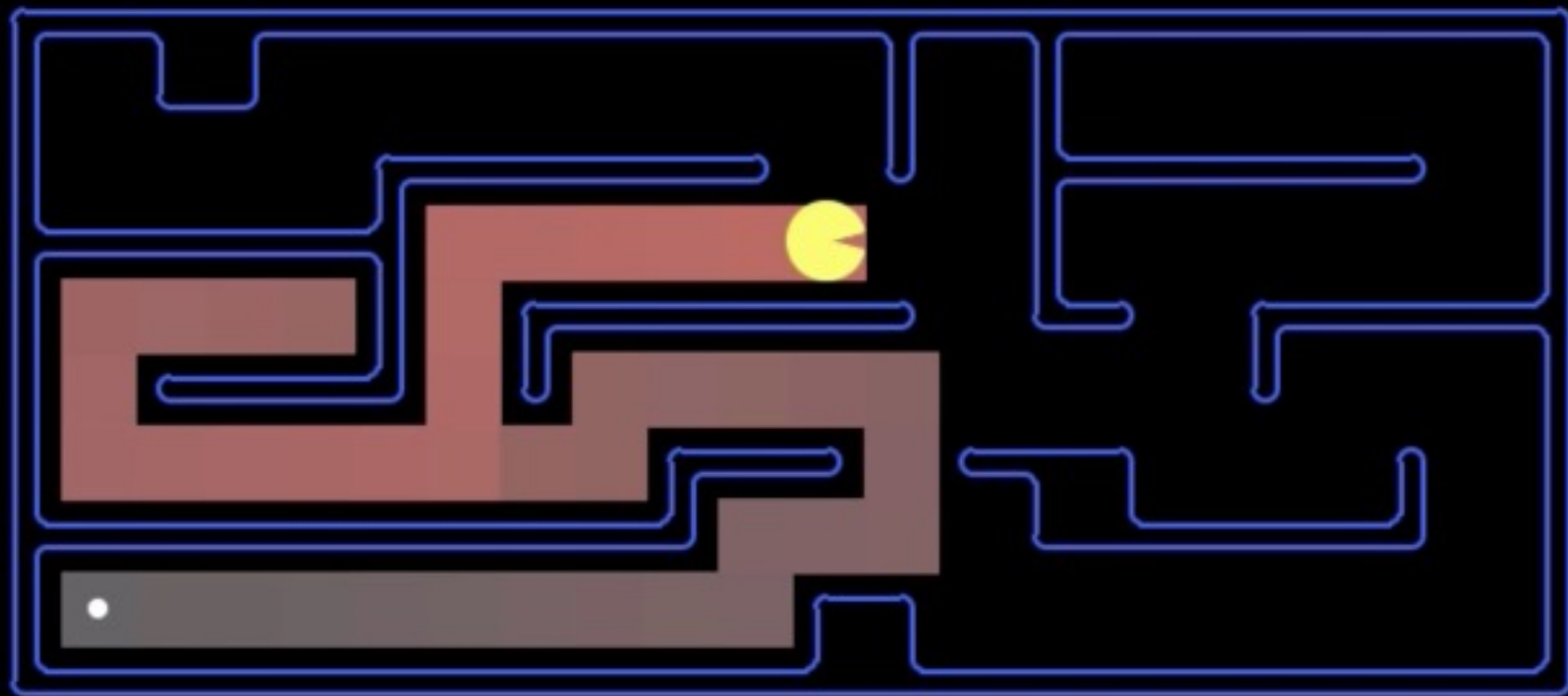
**Soluția UCS**



# PAC MAN – demo Greedy



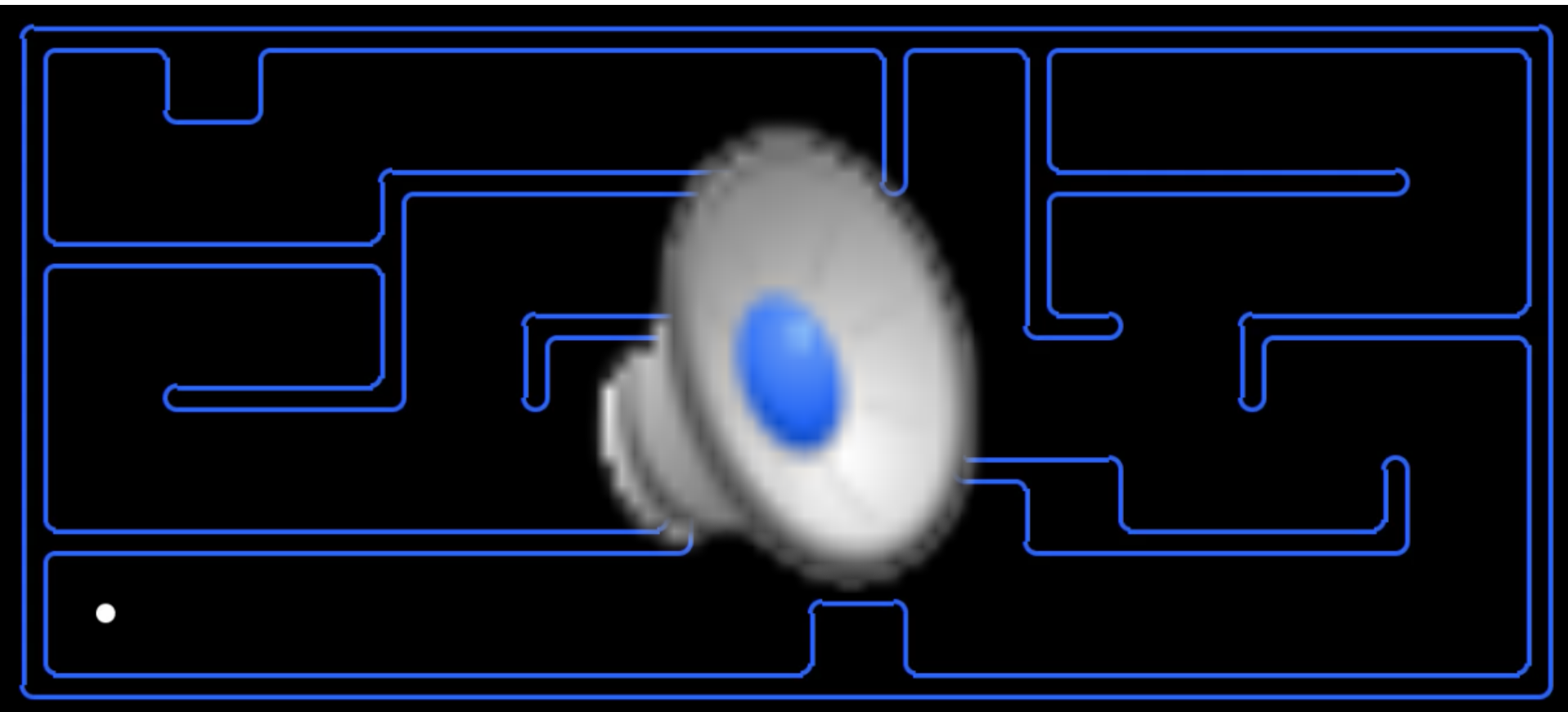
# PAC MAN – demo Greedy



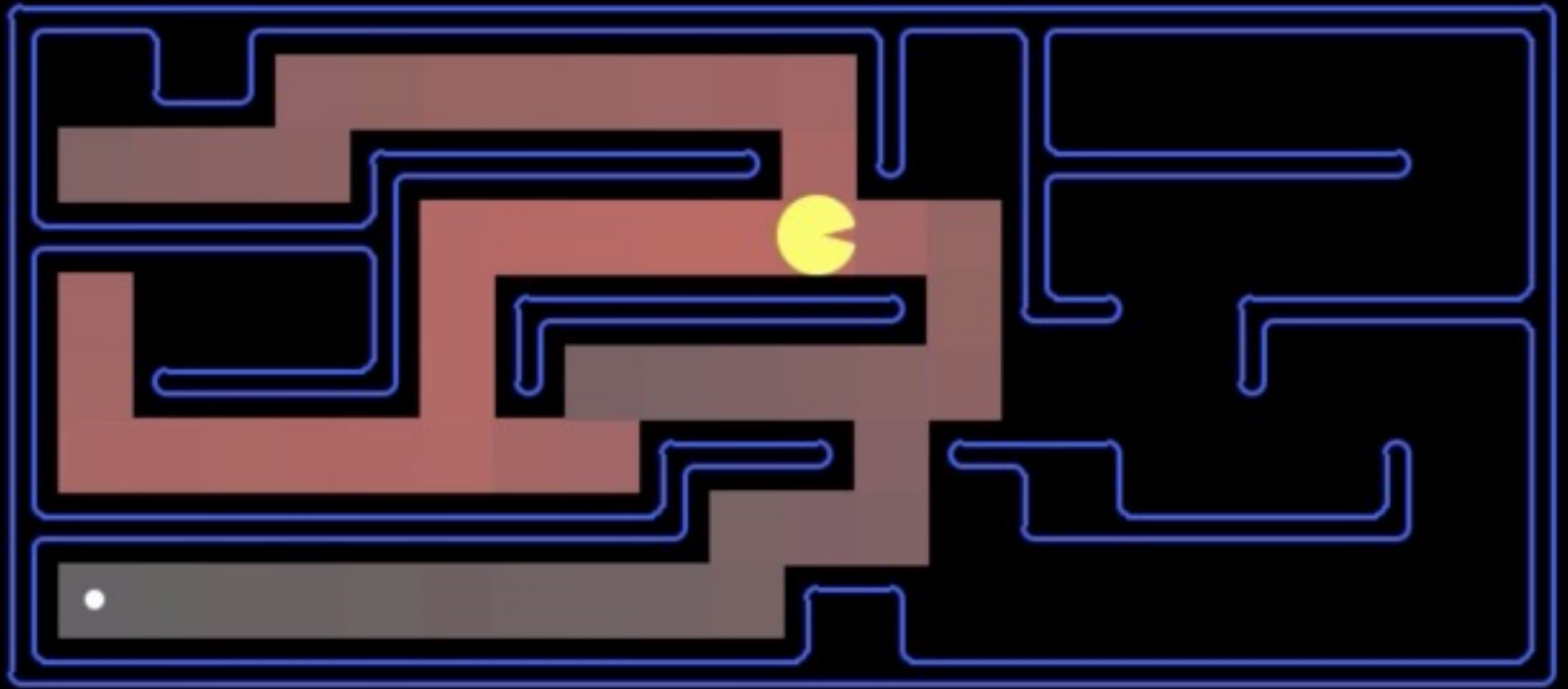
**SCORE: 0**

**Soluția Greedy**

# PAC MAN – demo A\*



# PAC MAN – demo A\*



**SCORE: 0**

**Soluția A\***

# Comparare PAC MAN



Greedy

**suboptimal**

**rapid**



UCS

**optimal**

**încet**



A\*

**optimal**

**rapid**

# Labirint - demo

STARE  
SCOP



STARE  
INIȚIALĂ



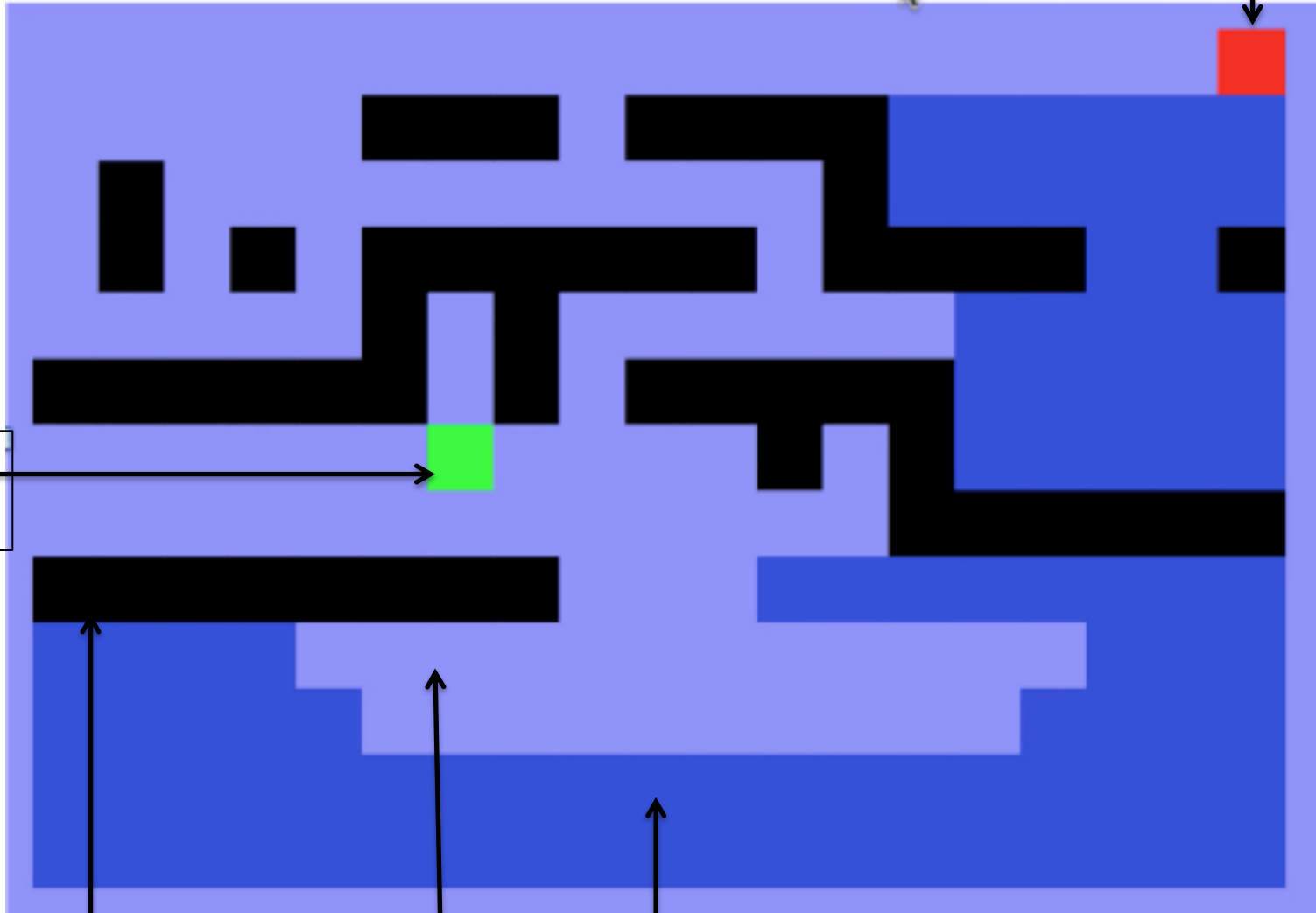
Zid



Cost 1



Cost 3

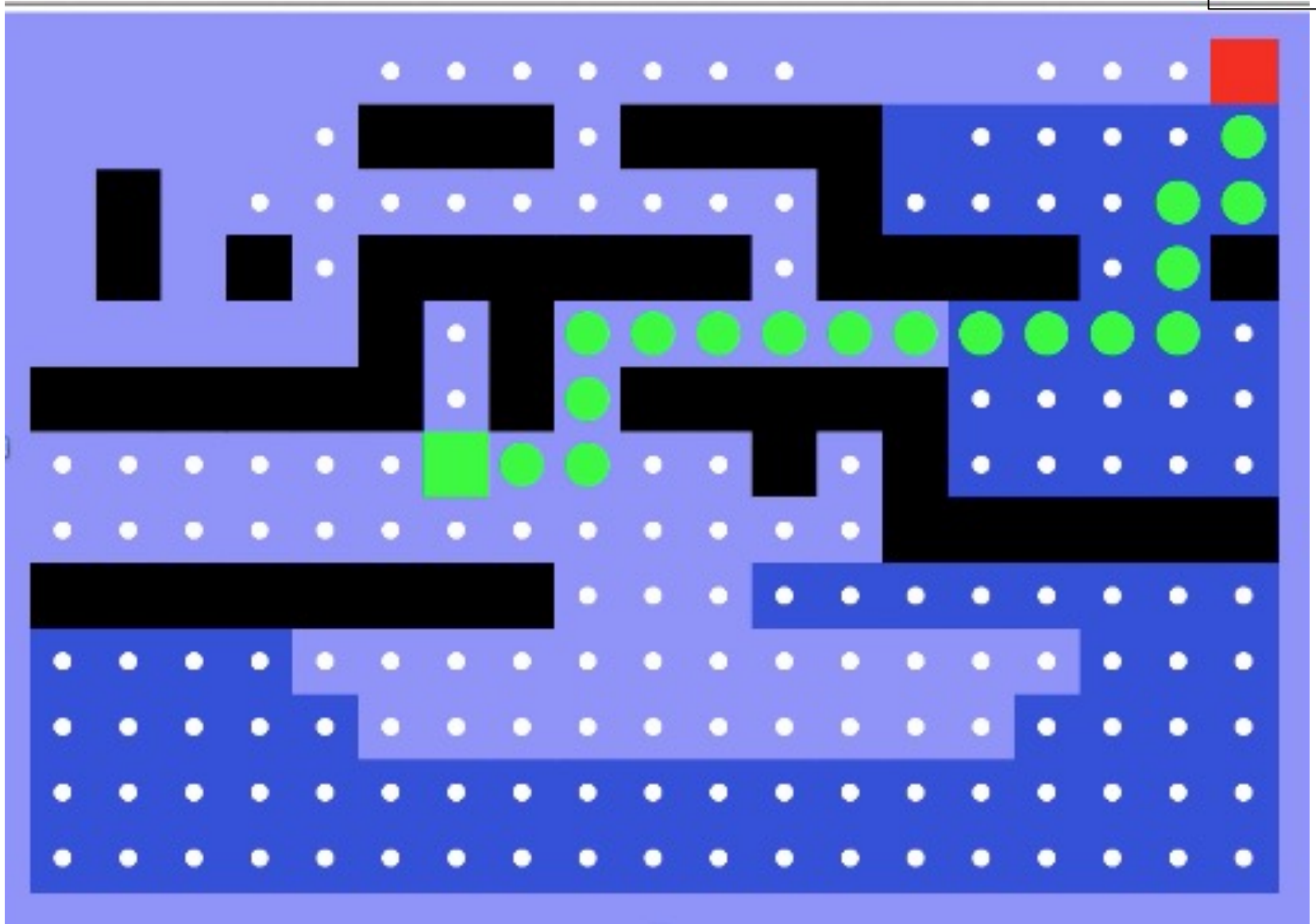


# Labirint – demo Bread First search



# Labirint – demo Bread First search

Cost 36



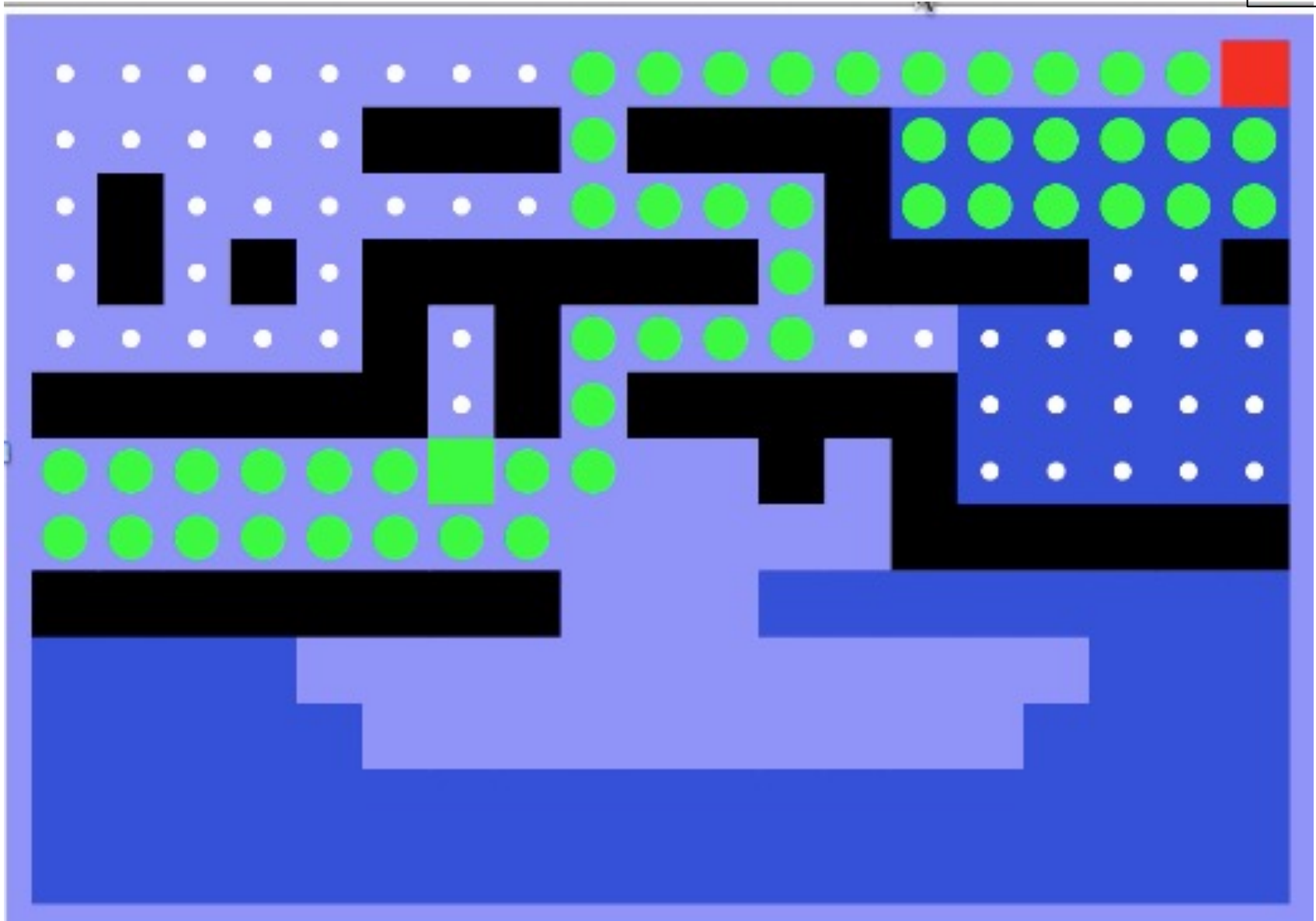


# Labirint – demo Depth First search



# Labirint – demo Depth First search

Cost 73

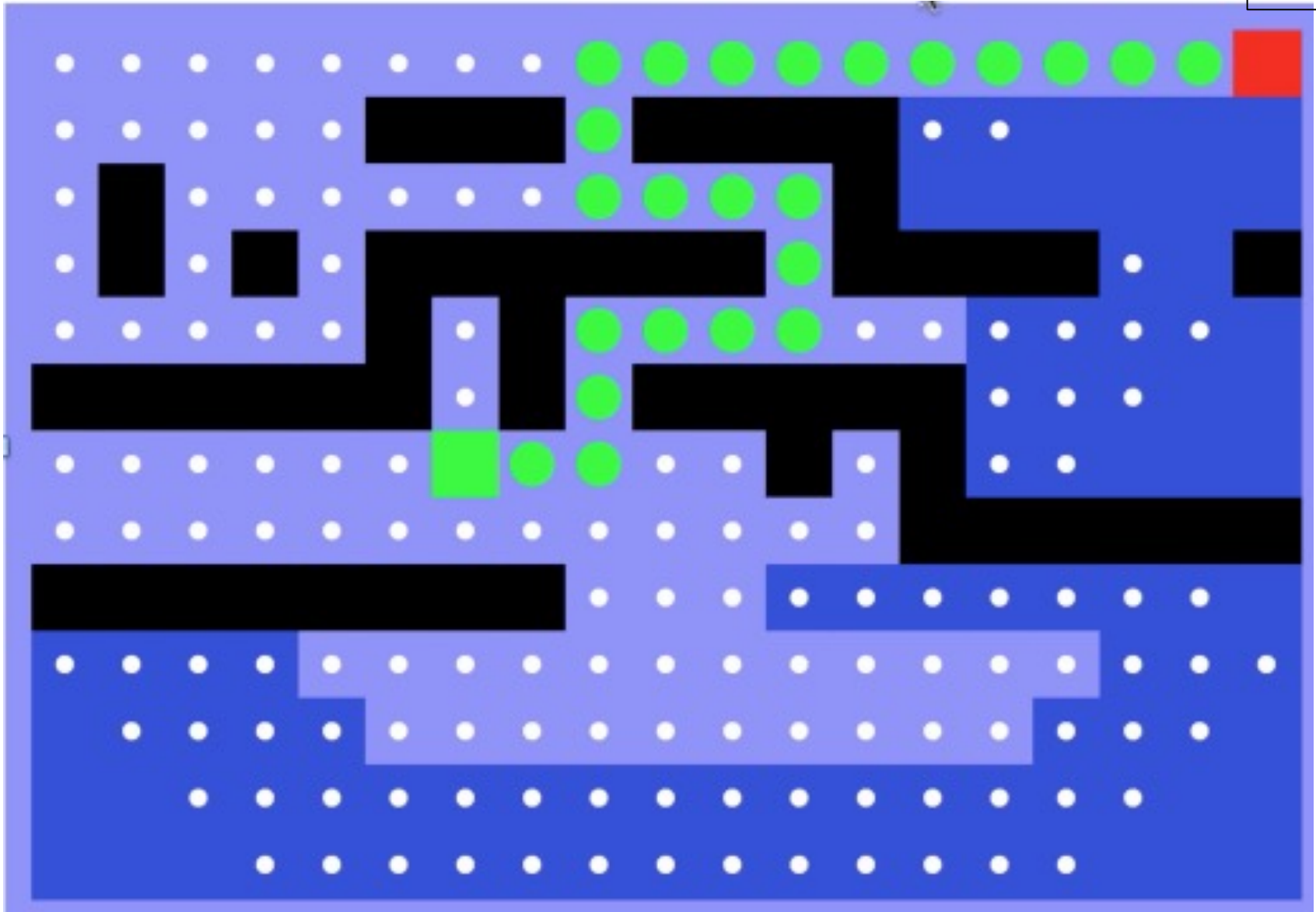


# Labirint – demo Uniform Cost Search



# Labirint – demo Uniform Cost Search

Cost 24

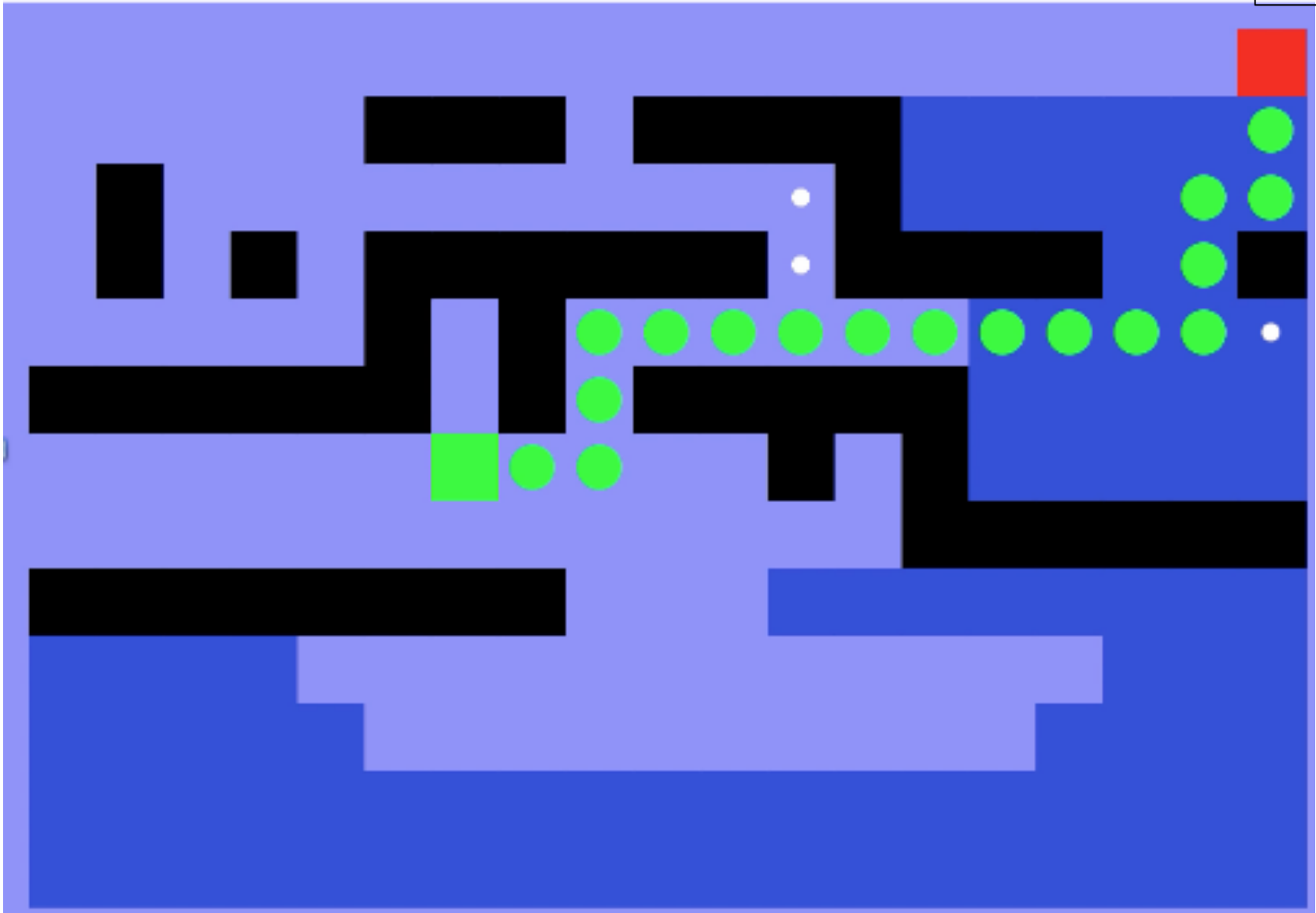


# Labirint – demo Greedy search



# Labirint – demo Greedy search

Cost 36

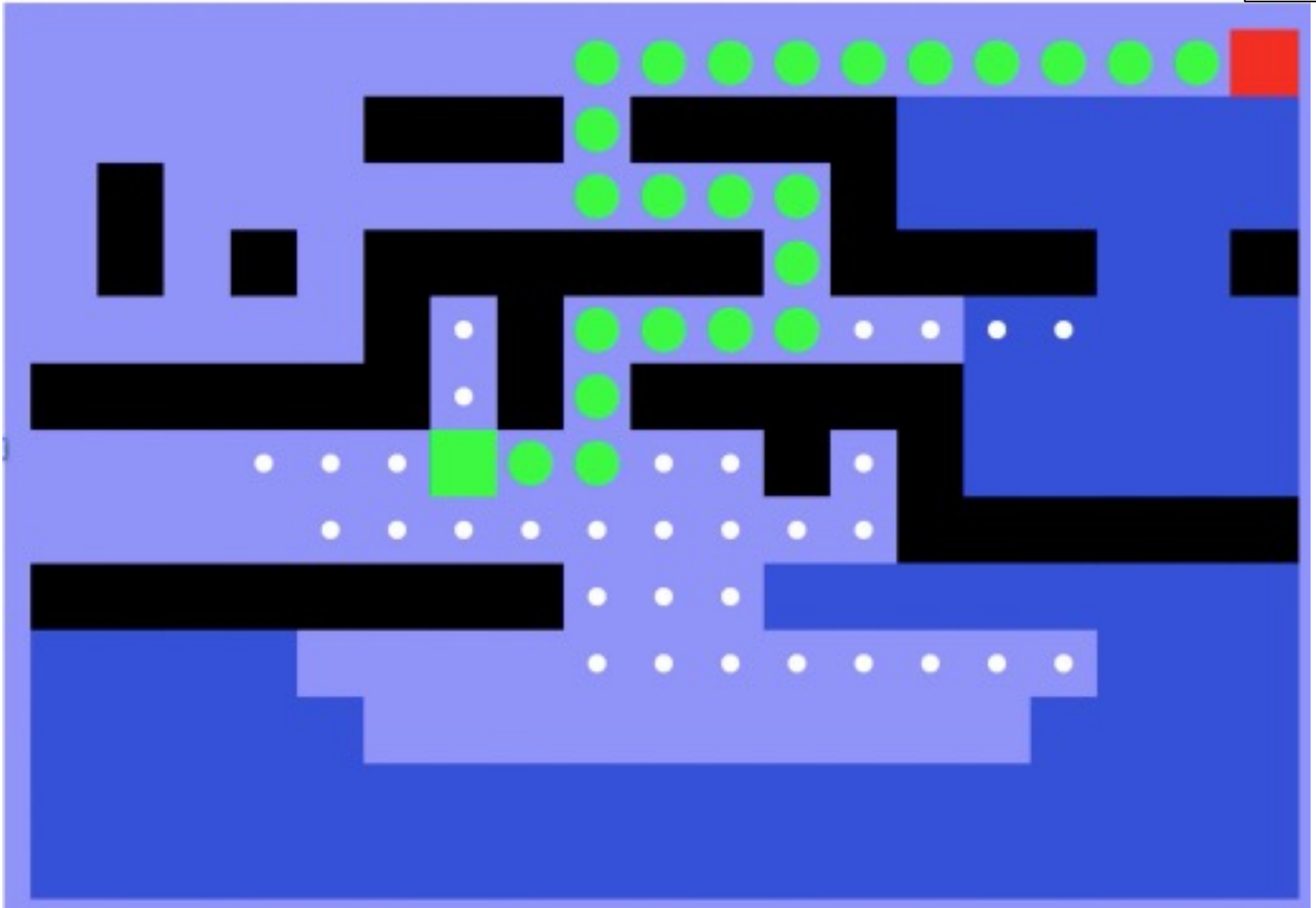


# Labirint – demo A\*



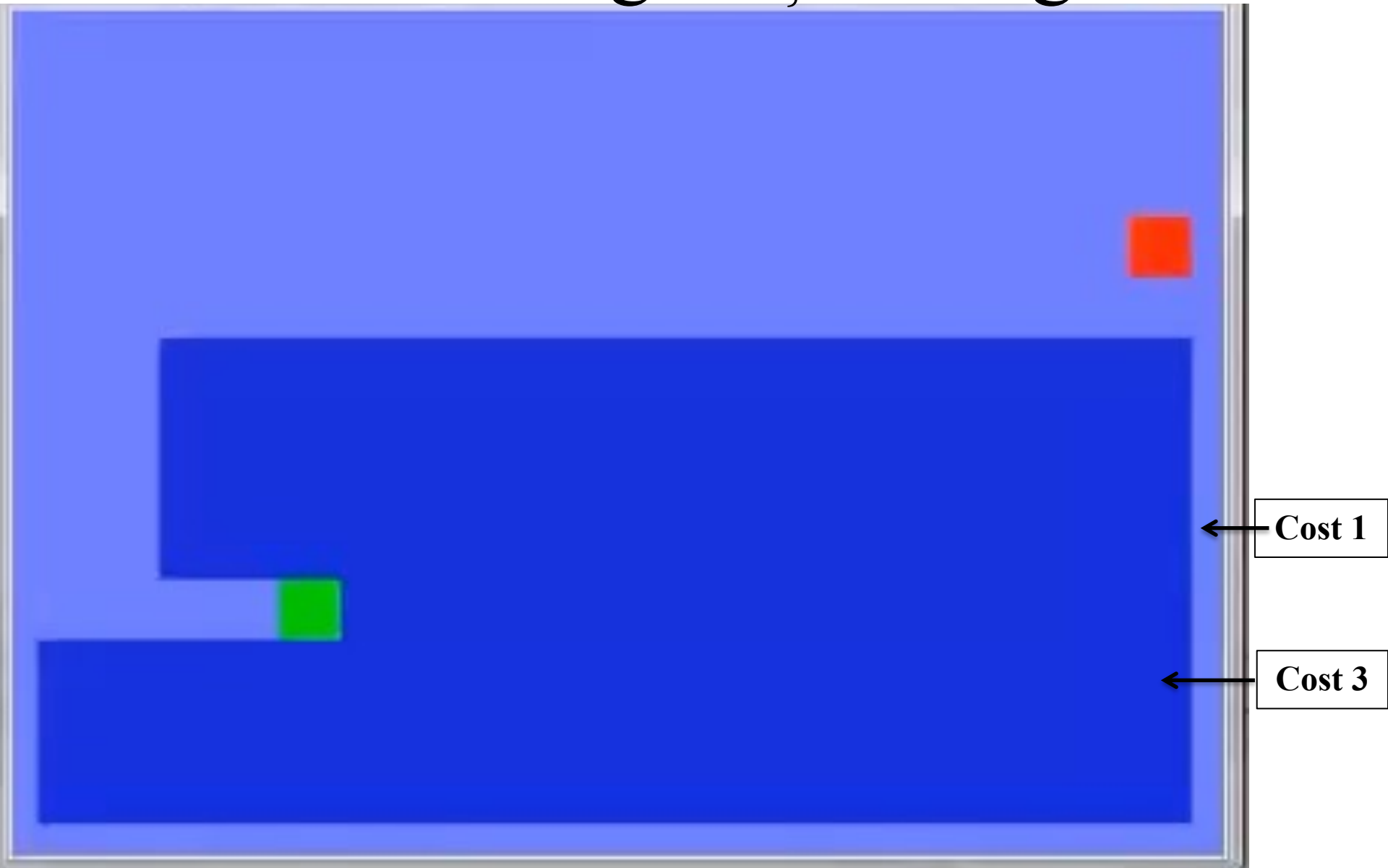
# Labirint – demo A\*

Cost 24

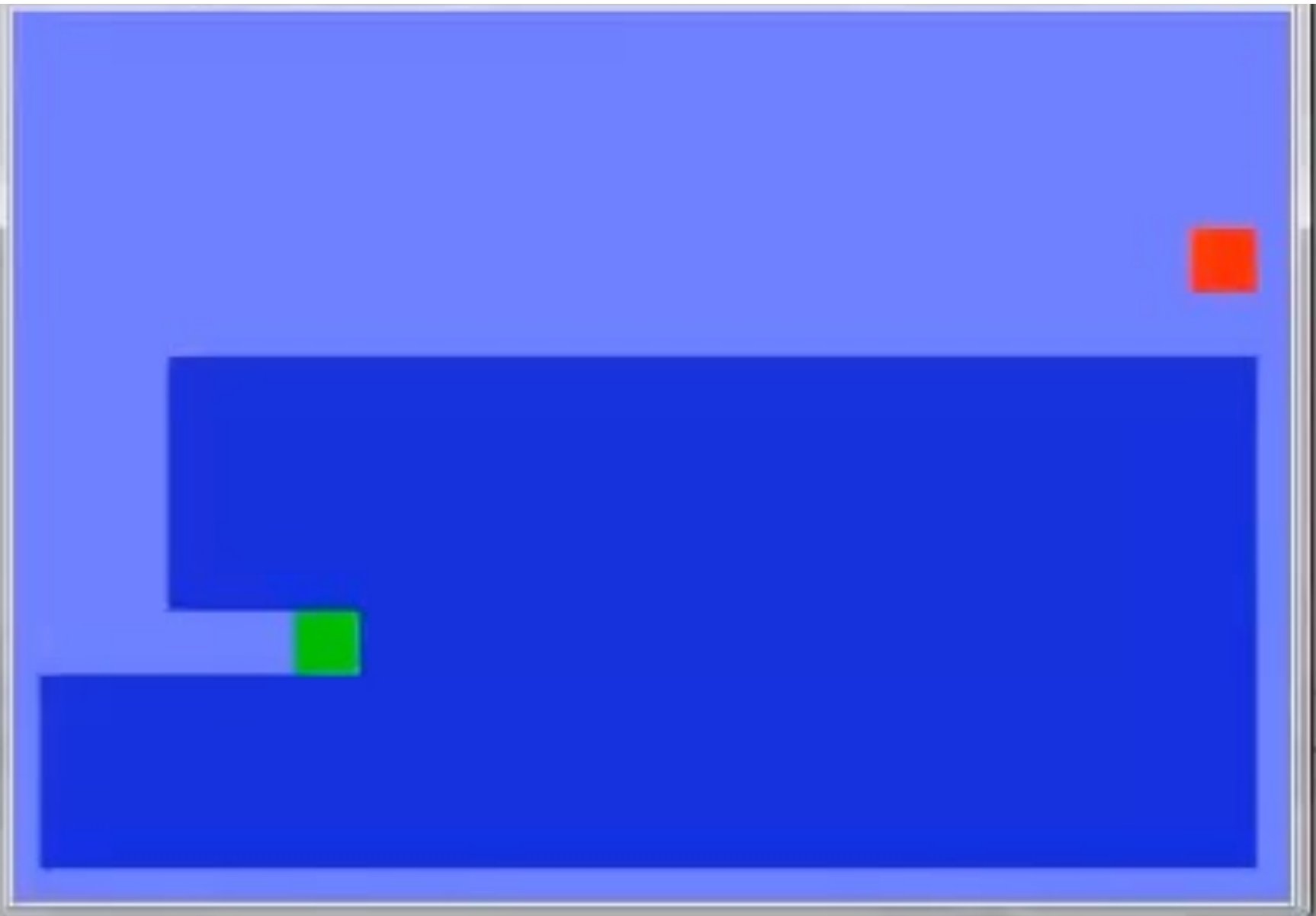




# Labirint – ghiciți strategia



# Labirint – BF



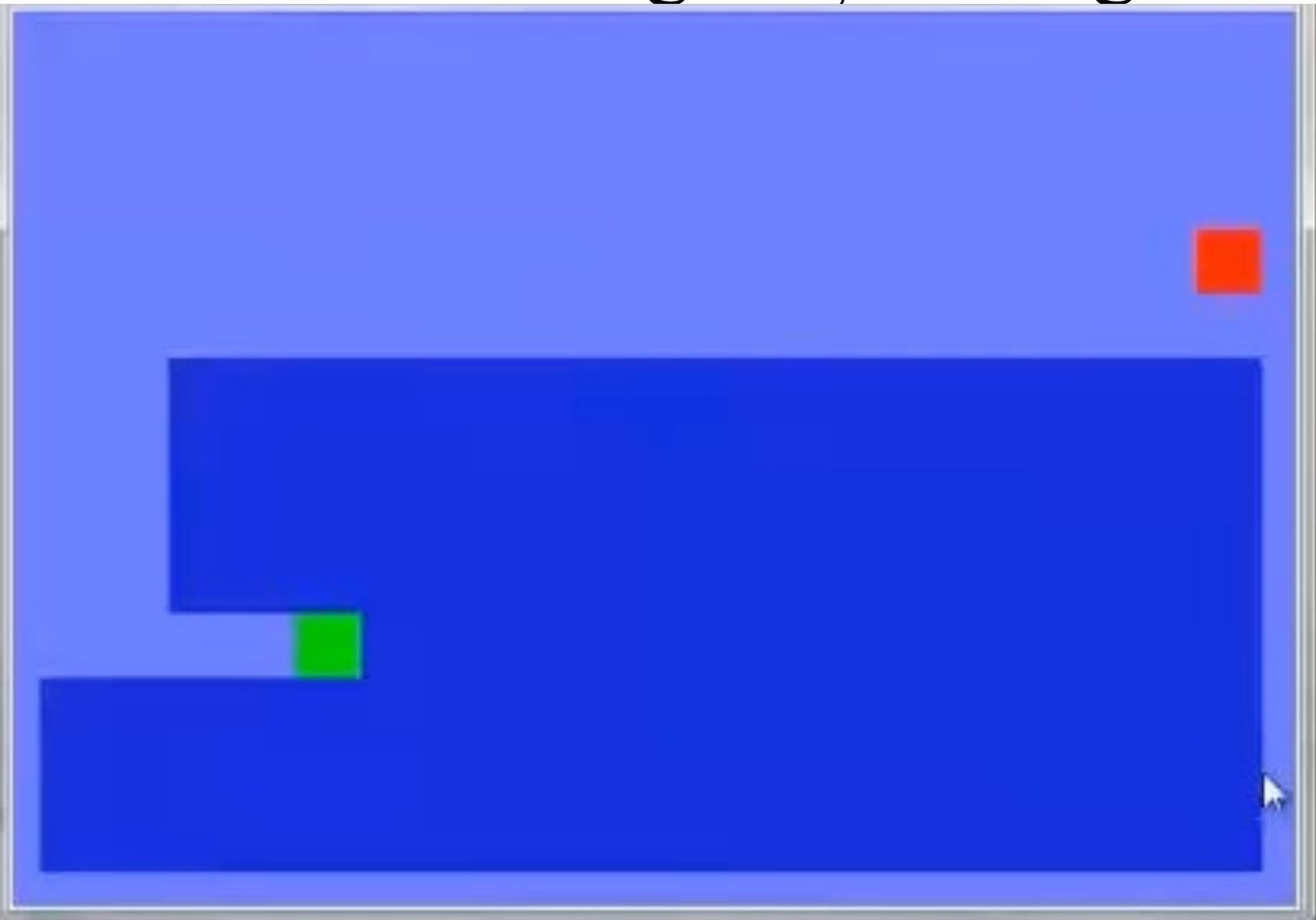
# Labirint – ghiciți strategia



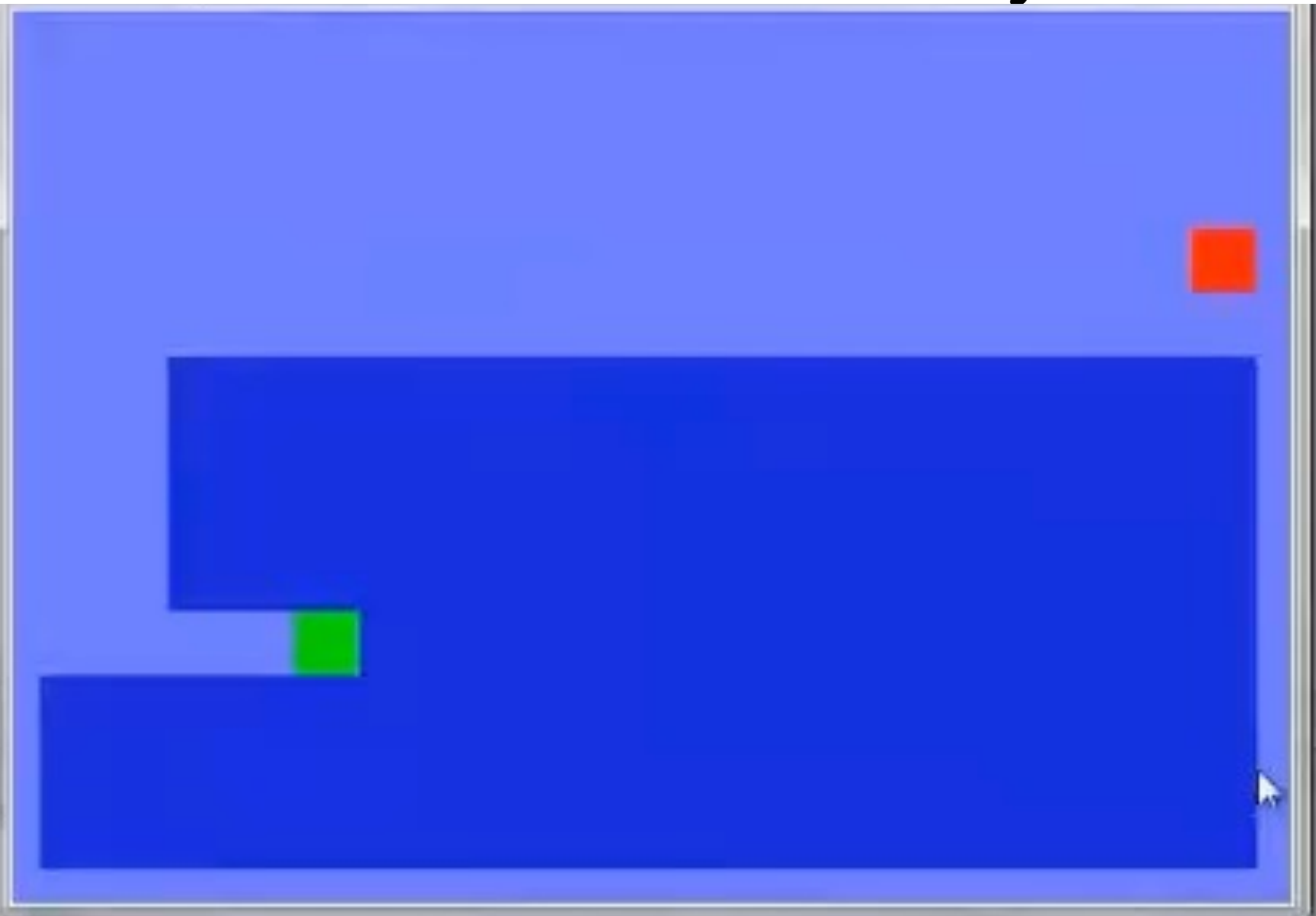
# Labirint – UCS



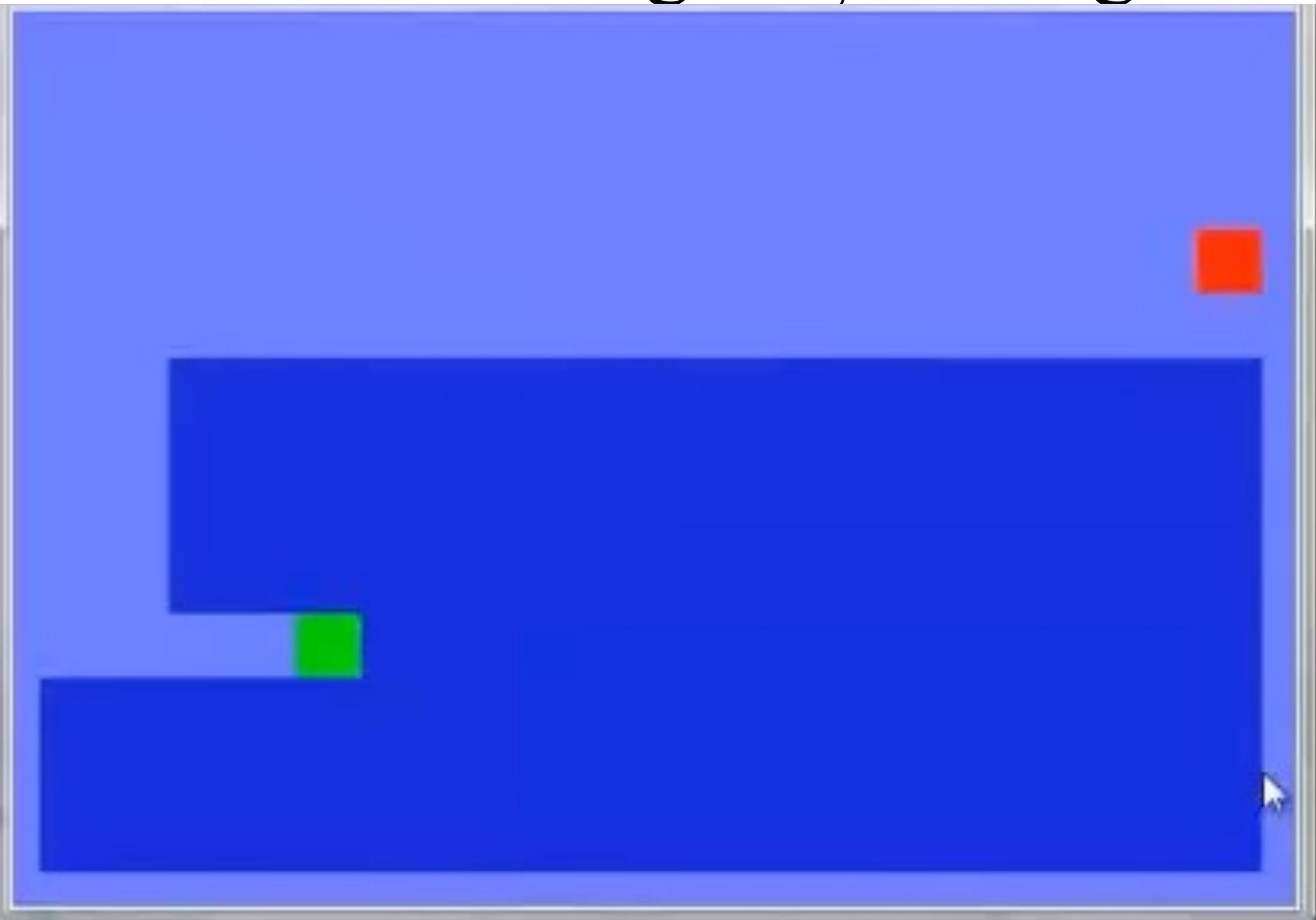
# Labirint – ghiciți strategia



# Labirint – Greedy



# Labirint – ghiciți strategia



# Labirint – DF





# Labirint – ghiciți strategia

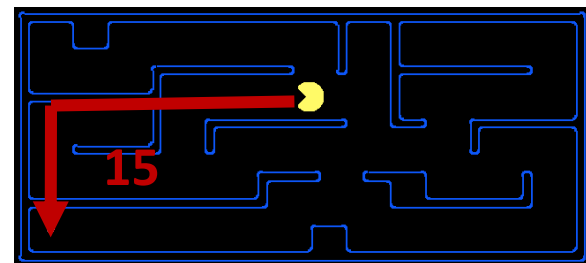
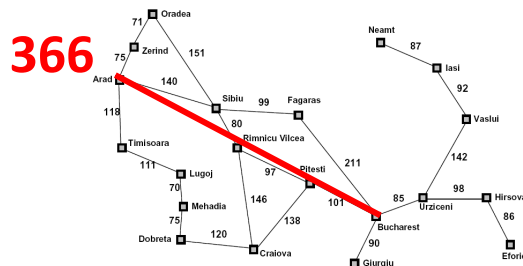


# Labirint – A\*



# Crearea de euristici admisibile

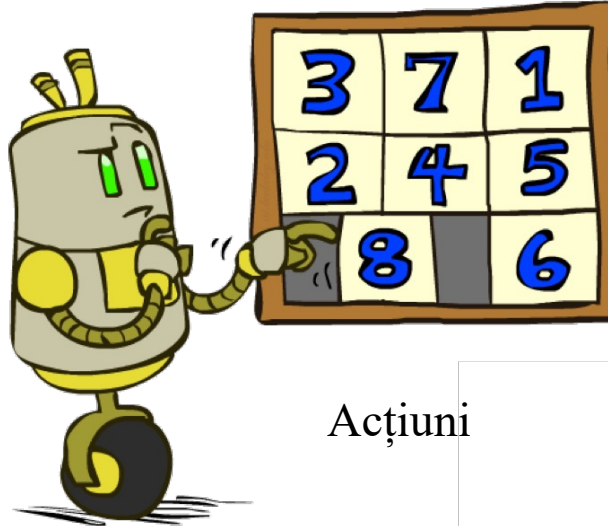
- Un ingredient fundamental în rezolvarea optimă a problemelor de căutare este găsirea unor euristici admisibile
- Deseori, euristicile admisibile sunt soluții pentru *probleme relaxate* (obținute din problema inițială prin relaxarea unor condiții), când sunt disponibile noi acțiuni



# Exemplu – 8 Puzzle

|   |   |   |
|---|---|---|
| 7 | 2 | 4 |
| 5 |   | 6 |
| 8 | 3 | 1 |

Starea inițială



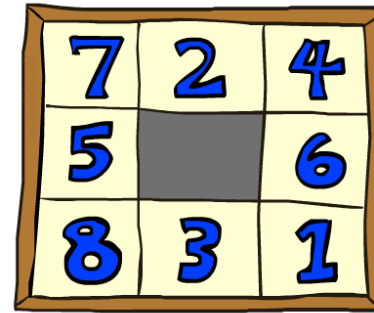
Acțiuni

|   |   |   |
|---|---|---|
|   | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

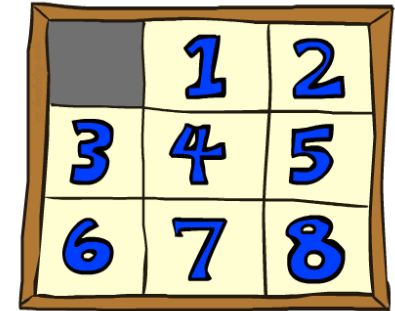
Starea finală

# Exemplu – 8 Puzzle

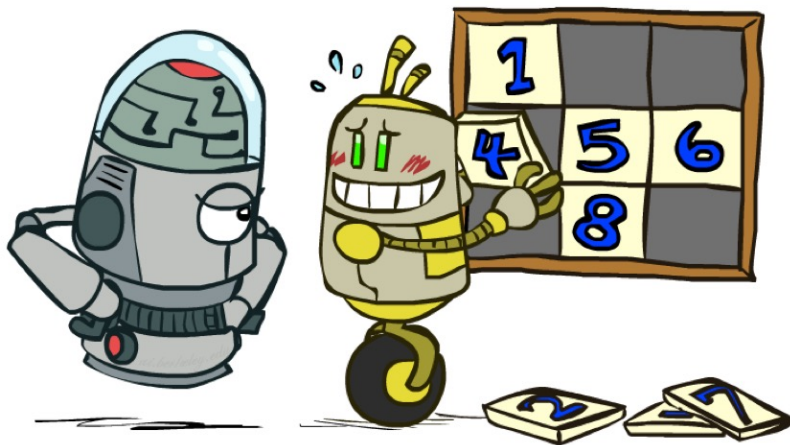
- Euristică: numărul de piese puse greșit
- Este euristică admisibilă?
- $h(\text{stare inițială}) = 8$
- Aceasta este o euristică pentru o *problema relaxată*



Starea inițială



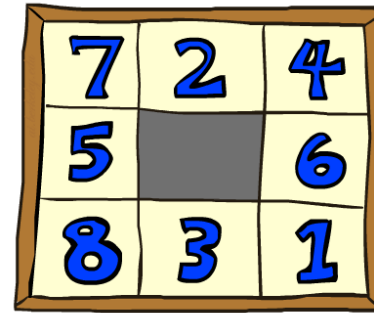
Starea finală



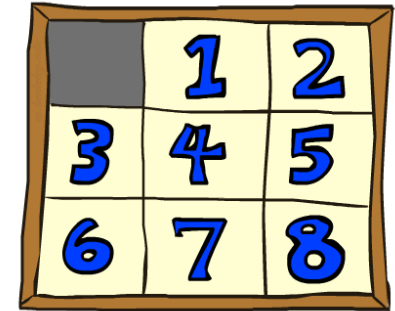
| Numărul de noduri explorate<br>când soluția optimă are... |           |           |                   |
|-----------------------------------------------------------|-----------|-----------|-------------------|
|                                                           | ...4 pași | ...8 pași | ...12 pași        |
| UCS                                                       | 112       | 6,300     | $3.6 \times 10^6$ |
| A* + #TILES                                               | 13        | 39        | 227               |

# Exemplu – 8 Puzzle

- Dacă considerăm problema 8-puzzle modificată astfel încât fiecare piesă poate fi glisată stânga/dreapta/sus/jos ignorând celelalte piese?



Starea inițială



Starea finală

- Distanța *Manhattan*
- Este euristică admisibilă?

- $h(\text{stare inițială}) = 3 + 1 + 2 + \dots = 18$

| Numărul de noduri explorate când soluția optima are... |           |           |            |
|--------------------------------------------------------|-----------|-----------|------------|
|                                                        | ...4 pași | ...8 pași | ...12 pași |
| A* + #TILES                                            | 13        | 39        | 227        |
| A* + MANHATTAN                                         | 12        | 25        | 73         |